

Who Holds the Jar?

Pumulo Sikaneta

Book Two — Who Holds the Jar?

CHAPTER WHJ-1

The Pebble and the Problem

On the clay tokens of ancient Mesopotamia and the birth of computation

The Pebble and the Problem

On a diver who surfaced screaming about corpses, and why the most powerful machine in history began with a handful of clay tokens in a jar.

Around Easter of 1900, a storm blew a small boat full of sponge divers off course in the Aegean. Captain Dimitrios Kontos, from the island of Symi, had been heading for the sponge beds off the North African coast. Instead he found himself anchored in the lee of a rocky island called Antikythera, between Crete and the Greek mainland, waiting for the wind to drop. Kontos was not the kind of man who wasted time. While the storm held, his crew would dive. Elias Stadiatis pulled on the newfangled diving suit — a canvas shell with a copper helmet and a rubber hose feeding air from a hand pump on deck — and dropped into forty-five metres of dark water.

He was down for less than a minute.

When they hauled him to the surface, he was frantic.

He described a scene on the seafloor that made no sense: heaps of rotting corpses and dead horses, scattered among the rocks, staring up at him. The crew thought he was delirious. Nitrogen narcosis at that depth could make a diver see things. Kontos did what any captain would: he suited up and went down himself.

He stayed longer than Stadiatis had.

When they pulled him back aboard, he was holding the arm of a bronze statue. The corpses were not corpses. The horses were not horses.

They were sculptures — marble and bronze figures from a Roman cargo ship that had gone down more than two thousand years earlier, their surfaces encrusted with centuries of marine growth, their features blurred into something that looked, in the dim green light of forty-five metres, exactly like the dead. Kontos kept the wreck's

location secret.

He sailed to Athens, showed the bronze arm to the authorities, and negotiated: his divers would do the recovery work. Greece was in financial trouble and political turmoil; a haul of ancient treasures — reminders of past glories — was exactly what the government needed. The Hellenic Navy dispatched ships. Recovery began in November 1900 and stretched into the following year. The work was brutal.

One diver, Georgios Kritikos, died from decompression sickness.

Two others were paralyzed.

But the team brought up dozens of bronze and marble statues, pottery, jewellery, glassware, and coins from the first century BCE. Among the treasures was a lump of corroded bronze about the size of a shoebox.

It looked like nothing.

Nobody paid it any attention.

It was cataloged, stored, and forgotten.

It was the most sophisticated piece of technology built in the ancient world.

The Machine at the Bottom of the Sea

In May 1902, a Greek archaeologist named Valerios Stais was examining the Antikythera haul in a workroom at the National Archaeological Museum in Athens when he noticed something odd about the corroded lump. As the wood casing had dried in the open air, it had cracked apart, revealing bronze fragments with tiny inscriptions and what appeared to be the precision gears of a mechanical clock.

It took the better part of a century to understand what Stais had found. In the 1950s, a British historian of science named Derek de Solla Price became obsessed with the fragments. Working with a Greek radiographer, he took the first X-rays of the device.

In 1974 he published his findings: the corroded lump was a mechanical computer, built around 100 BCE, containing at least thirty interlocking bronze gears, precisely machined, that could predict the positions of the sun and moon, the timing of eclipses, and the schedule of the ancient Olympic Games. A person in the first century BCE — a world with no electricity, no printing press, no telescope — could turn a hand crank and know, with considerable accuracy, where the sun and moon would be on any given date in the future.

The machine stored astronomical data in the arrangement of its gears.

It retrieved the data through the position of its dials.

It compared celestial cycles through the ratios of its gear trains. And it transformed inputs — turning the crank — into outputs: positions on its display. Once the gears were set, the machine ran on its own logic. The human turned the crank. The mechanism did the rest. Nothing of comparable complexity would appear in the historical record for over a thousand years. In most histories of computing, this is where the narrative moves on. But the Antikythera mechanism teaches two lessons that will echo through every chapter of this book.

What the Mechanism Teaches

The first lesson is about power. In the ancient world, predicting celestial events was not an academic exercise.

It was political and religious authority. A priest who could announce, weeks in advance, that the moon would vanish from the sky held enormous influence over a population that understood eclipses as omens from the gods. An astrologer who could tell a king the most auspicious day to launch a military campaign wielded real power — not because the stars actually governed outcomes, but because the king believed they did, and acted accordingly. The Antikythera mechanism was, among other things, a power tool. Whoever operated it could predict what others could not.

And the gap between what you can predict and what the people around you can predict is the oldest form of strategic advantage.

It is the same advantage held by every institution, government, and corporation that has controlled computation in the centuries since.

“The gap between what you can predict and what the people around you can predict is the oldest form of strategic advantage. Computation has never been neutral.” The second lesson is about loss. The Antikythera mechanism was built in a world that, within a few centuries, would lose the ability to build it. The precision gear-cutting techniques, the astronomical knowledge encoded in the gear ratios, the manufacturing skill required to mesh thirty gears in a casing the size of a shoebox — all of this knowledge vanished.

It did not fade gradually.

It was lost in the collapse of the Greco-Roman world and the fragmentation of the networks that had sustained it.

When the mechanism was pulled from the sea in 1901, no scholar alive could explain how it worked.

It took another hundred years of research to understand what a craftsman in 100 BCE had built with hand tools. Technological progress is not a ratchet.

It does not only go forward.

It can be lost — through war, through civilizational collapse, through the simple failure to transmit knowledge from one generation to the next. The Antikythera mechanism is a reminder that what we have built is not guaranteed to survive. That is a truth worth holding as we trace the story forward through substrates of increasing power and fragility.

The Shepherd and the Jar

But the Antikythera mechanism, extraordinary as it was, is not where this story begins.

It begins much earlier, much simpler, and much closer to the ground. Somewhere around five thousand years ago, in what is now southern Iraq, a shepherd faced a problem that his brain could not reliably solve.

He had a flock. The flock left in the morning. The flock returned in the evening. And every evening, he needed to answer a simple question: did they all come back? The human mind is a remarkable instrument.

It can recognize a face it has not seen in twenty years, compose a song from nothing, and sense danger from the faintest change in another person's expression.

What it cannot do, with any reliability, is count to sixty-three and remember the result twelve hours later. Not every day. Not without error. Not when the sheep look alike and you are tired and it is getting dark. The shepherd's solution was elegant. Each morning, as the sheep filed out through the gate, he dropped a small clay token into a jar — one token for each animal. Each evening, as they returned, he removed a token for each one that came back.

If the jar was empty at the end, the flock was complete.

If tokens remained, sheep were missing. The shepherd did not need to count.

He did not need to remember a number.

He needed to compare two quantities, and the jar did it for him. That jar of clay tokens is a computer. Not metaphorically. Functionally.

It stores information — the count of the flock.

It has an input mechanism — dropping a token in.

It has an output mechanism — the tokens that remain. And it has a purpose: answering a question that the human mind could not reliably answer on its own.

Every machine we have built since — from the abacus to the smartphone in your pocket to a quantum processor manipulating individual atoms — is an elaboration of that jar. I begin with a shepherd and a handful of pebbles, because I believe the history of computing is almost always told from the wrong starting point. Most accounts begin with Charles Babbage in the 1830s, or with the electronic computers of the 1940s, as though computation were an invention of the industrial age.

It is not. Computation is one of the oldest human activities — older than writing, older than agriculture, possibly older than spoken language as we know it. And the reason it is so old is that it solves the most fundamental problem any organism faces: the gap between what the mind can imagine and what the mind can reliably execute. That gap is where every computer lives.

It is where every computer has always lived. And the story of how we have tried to close it — with pebbles, with gears, with vacuum tubes, with silicon, and soon with the laws of quantum physics and the molecules of life itself — is, I believe, the most consequential story in human history.

Older Than Writing

The oldest mathematical artifact ever found is a small, dark bone roughly the length of a pencil, discovered in 1960 in what is now the Democratic Republic of Congo.

It is called the Ishango bone, and it is approximately twenty thousand years old. Carved along its surface are three columns of notches — grouped in patterns that have fascinated and divided mathematicians for decades.

One column contains groups of 11, 13, 17, and 19 — the prime numbers between 10 and 20. Another contains groups that could represent a doubling sequence. Whether this is intentional mathematics or coincidental grouping is still debated.

What is not debated is that someone, twenty thousand years ago, was scratching a systematic record of quantities onto a bone.

They were storing information outside their own mind.

They were computing. Twenty thousand years ago, there were no cities, no agriculture, no written language, no metal tools. Humans lived in small groups, hunted with stone-tipped spears, and wore animal skins. And somebody, in that world, felt the need to record numerical information in a durable medium. The urge to compute is not a product of civilization.

It is a precondition for it. The Mesopotamian shepherds came much later — around 3500 to 3000 BCE — and their counting tokens grew into something more

sophisticated. Archaeologists have found clay tokens of different shapes: spheres for grain measures, cones for small amounts of oil, discs for livestock. The tokens were placed inside hollow clay balls called *bullae*, which were then sealed. To know what was inside without breaking the seal, accountants began pressing the tokens into the surface of the wet clay before enclosing them — creating a visible record on the outside of what was stored on the inside.

Some scholars believe this is how writing was born. The impressions on the outside of the *bullae* became standardized. The tokens inside became unnecessary. The marks alone carried the meaning.

If this theory is correct — and the evidence is compelling — then writing itself is a byproduct of computation. Humans did not first learn to write and then use writing for accounting.

They first learned to account, and the marks they made became writing. Which means that the most transformative information technology in human history — the written word — may be a side effect of humanity's need to count.

The Reckoning Frame

The abacus appeared in Sumeria sometime between 2700 and 2300 BCE, and it represents a genuine leap. The counting tokens and tally marks that preceded it could store and compare, but they were slow and clumsy at transformation. To add two groups of tokens, you physically combined them. To subtract, you physically removed. For anything more complex, the system collapsed under its own weight. The abacus changed this by introducing a crucial idea: positional representation. A bead's value depended on which column it occupied. A bead in the rightmost column meant one.

The same bead, moved to the next column, meant ten. The next column, a hundred. This is the same idea behind the numbers you use every day — the digit 3 means three in the ones place, thirty in the tens place, three hundred in the hundreds place. The abacus was the first machine to exploit this principle. How effective was it?

In 1946, in a widely publicized contest reported across American and Japanese media, a Japanese postal clerk named Kiyoshi Matsuzaki sat across a table from an American soldier operating an electric calculating machine — the best electronic arithmetic technology the United States military had.

They competed across five categories of calculation: addition, subtraction, multiplication, division, and a mixed category combining all four. Matsuzaki, using an

abacus, won four of the five. The ancient tool, in trained hands, was still faster than twentieth-century electronics. The abacus also appeared independently in China, in Rome, in Mesoamerica, and in several other civilizations that had no contact with each other. This independent invention is significant.

It means the abacus is not a cultural artifact of a particular society.

It is a convergent solution — the way that wings evolved independently in birds, bats, and insects because flight solves a problem that many different organisms face. The abacus is what computation looks like when human hands are the only available moving parts.

But there is one critical thing the abacus cannot do.

It cannot operate without a human.

Every bead that moves, moves because a finger pushes it.

The machine has no autonomy.

It performs no operation on its own.

It is, in the most precise sense, a tool — an extension of the human hand and mind, incapable of independent action.

The first evidence that a machine could act on its own came from the wreck at the bottom of the Aegean that opened this chapter.

The Antikythera mechanism, once its gears were set, ran on its own logic. Turn the crank and the output followed from the input with no further human judgment required.

That is automation.

And the gap between a tool that requires a human hand on every bead and a mechanism that runs on its own logic is the gap that every subsequent chapter of this book will explore — because it is the same gap that exists today between a spreadsheet and an AI system that makes decisions without being asked.

The Substrate Principle

This chapter has described four different computing technologies: clay tokens, tally marks on bone, the abacus, and the Antikythera mechanism.

They span roughly twenty thousand years.

They were built by cultures that had no knowledge of each other. And yet they are all doing the same thing — storing, retrieving, comparing, and transforming information, performed on different physical materials. The physical material matters. Clay tokens can store and compare but cannot transform quickly. The abacus can

transform rapidly but cannot operate without a human hand. The Antikythera mechanism can operate automatically but is limited to a single domain — astronomy — hardwired into its gears. Each material enables certain capabilities and constrains others.

The material is what I will call, throughout this book, the *substrate* — the physical medium on which computation runs.

Here is the pattern that the rest of this book will trace: every substrate eventually reaches a ceiling. A physical limit, imposed not by human imagination but by the laws of nature, beyond which the material cannot be pushed. Clay tokens cannot perform multiplication. An abacus cannot operate autonomously. Gears cannot compute faster than the speed of their mechanical action.

When the ceiling is reached, progress does not stop. The substrate changes. And when the substrate changes, everything changes — not just how fast we compute, but who can compute, what gets computed, and who controls the results. The change is not gradual. Not the same machine getting better.

It is a phase transition — like water becoming steam. The old substrate does not get incrementally better.

It is replaced by something operating on fundamentally different physics. And each time this happens, the consequences ripple far beyond the machine itself, into economics, politics, warfare, and the basic structure of daily life.

We are approaching one of these transitions right now. Actually, we are approaching two simultaneously — quantum computing and biological computing — which is unprecedented in the history of computation.

But before we can understand what is coming, we need to understand what came before, and why each transition followed the pattern it did.

Who Holds the Jar

There is one more pattern, and it is the one that gives this book its title. The shepherd's jar was his.

He made it, he filled it, he read the result. The computation served the person who performed it.

But the Antikythera mechanism was not a shepherd's tool.

It was expensive, complex, and rare. Whoever owned it had access to predictions that nobody else could make. The mechanism's power was not merely technical.

It was political — the power to see what others could not, to anticipate what others could not, to act on knowledge that others did not have. Between the shepherd's jar and the Antikythera mechanism, something shifted. Computation went from personal to institutional. From the hand of the person with the problem to the hand of the person with the resources to build the machine. This shift — the movement of computational power from the many to the few, and occasionally back again, in a cycle that repeats in every era this book will document — is the central thread of the story that follows.

Call it the *distribute-then-recapture* cycle. A new substrate arrives.

It distributes computational capability more widely than before — to more people, more cheaply, more accessibly. And then, over time, the control of that capability reconcentrates in the hands of a few. The tokens changed. The jar changed.

The question did not: who holds it?

*“A new substrate arrives.

It distributes computational capability more widely than before. And then, over time, the control reconcentrates in the hands of a few. The tokens changed. The jar changed.

The question did not.”* Elias Stadiatis surfaced screaming about corpses and died with no fame that anyone recorded. Georgios Kritikos died of decompression sickness recovering the treasures. The lump of corroded bronze sat in a museum storeroom for two years before anyone noticed the gears inside it, and for another seventy before anyone understood what they meant. The most sophisticated computer of the ancient world was invisible to the people who found it.

They were looking for statues. This is another pattern worth remembering. The most consequential technology in any era is often the one that nobody recognizes at first — because it does not look like what people expect power to look like.

It looks like a corroded lump.

It looks like a jar of pebbles.

It looks like a toy. Until it isn't. The next chapter moves forward three thousand years, to a different kind of computing, a different kind of builder, and a different kind of ambition. A nineteen-year-old in France builds a machine to save his father from drowning in arithmetic. A philosopher in Germany invents a number system the world will ignore for 250 years. And a woman looks at a machine made of brass and sees something no one else can see: a future in which machines would not merely calculate, but create.

CHAPTER WHJ-2

The Clock and the Calculation

On Pascal's calculator, Babbage's engines, Ada Lovelace, and the rise and limits of mechanical computation

The Clock and the Calculation

On a nineteen-year-old who built a calculator to save his father from drowning in arithmetic.

In 1639, in the French city of Rouen, a sixteen-year-old named Blaise Pascal watched his father die by arithmetic. Not literally. Not yet.

But the trajectory was visible to anyone who looked. Étienne Pascal had recently been appointed the king's tax commissioner for upper Normandy — a prestigious position that came with a soul-crushing obligation. The work required endless columns of addition and subtraction, tallying tax receipts across hundreds of parishes, checking figures that had been checked before, searching for errors that crept in every time a human hand copied a number from one ledger to another. The work was important.

It was also grinding the man down. Not dramatically, not the way wars or plagues kill. Slowly. Through the monotony of arithmetic that the human brain can perform but was never designed to sustain. The boy watched. And then the boy decided to build a machine that could do it instead.

He was nineteen when the first working model was complete.

He called it the Pascaline — a brass box, roughly the size of a large book, with a row of numbered wheels on top. You dialled in a number by turning the wheels with a stylus, the way you might dial a rotary telephone. Internal gears carried the tens — when a wheel turned past nine, it advanced the wheel to its left by one, exactly the way an odometer in a car rolls over from 999 to 1000.

The machine could add and, through a method of complements that Pascal devised, subtract.

It was not fast.

It was not always reliable. The gears sometimes jammed.

But it worked. Pascal would build roughly fifty Pascalines over the next decade, each a small refinement of the last. Only about twenty were completed and fewer still were sold.

The machines were expensive — far more costly than hiring a clerk to do the same arithmetic — and clerks, unlike brass gears, could also answer questions, carry messages, and make coffee. The Pascaline was a commercial failure.

It was also one of the most important machines ever built. Not for what it did — adding and subtracting are modest accomplishments — but for what it proved. A mechanical device, built from gears and springs, could perform an operation that had previously required a human mind. The operation was simple. The principle was not. The Pascaline demonstrated that arithmetic was not an act of thought.

It was a physical process — a sequence of mechanical steps that could be performed by anything capable of executing those steps in the correct order. The mind was not necessary. The mechanism was sufficient. That insight — that a mental operation can be reduced to a mechanical one — is the foundation of every computer that has ever existed. Pascal did not articulate it in those terms.

He was trying to save his father.

But the principle he demonstrated would take three hundred years to reach its full expression, and when it did, it would change everything.

“The Pascaline demonstrated that arithmetic was not an act of thought. It was a physical process. The mind was not necessary. The mechanism was sufficient.”

The Philosopher and the Coin

Fifty years after Pascal, a German philosopher and mathematician named Gottfried Wilhelm Leibniz built a more ambitious machine. His Step Reckoner, completed in prototype around 1694, could not only add and subtract but also multiply and divide.

It used a clever mechanism called a stepped drum — a cylinder with teeth of varying lengths — that could engage different numbers of gear teeth depending on how far it was rotated.

The machine was ingenious but unreliable. The manufacturing precision of the seventeenth century could not consistently produce gears fine enough to mesh without jamming.

But Leibniz's most consequential contribution to computation was not a machine at all.

It was an idea. In 1679, he wrote a paper describing a complete arithmetic system that used only two digits: zero and one.

He called it the binary system. Our everyday number system is base-10 — it uses ten symbols, 0 through 9, and when you run out of symbols, you carry to the next column.

We use ten because we have ten fingers.

If we had eight fingers, we would use base-8. The base is a convention, not a law. Leibniz proposed base-2. Only two symbols: 0 and 1. The number after 1 is 10 (which means “two,” not “ten”).

Then 11 (three), 100 (four), 101 (five).

It is cumbersome for humans — the number 255 requires eight binary digits instead of three decimal ones.

But Leibniz was not thinking about human convenience.

He was thinking about elegance. In binary, every question reduces to the simplest possible choice: yes or no, true or false, something or nothing. Leibniz, who was deeply interested in Chinese philosophy, saw a connection between binary arithmetic and the *I Ching*, with its patterns of broken and unbroken lines.

He believed he had found a number system that reflected the fundamental structure of logic itself.

He was right.

But the world would not care for another 250 years. Binary arithmetic was published, admired as a philosophical curiosity, and filed away.

There was no machine that could use it. Mechanical calculators worked with gears, and gears naturally represent base-10 — a wheel with ten positions is straightforward to build. Binary was a solution waiting for a problem that would not arrive until the twentieth century, when a completely different kind of machine — one built from electrical switches instead of gears — would need a number system based on the simplest possible physical states: on and off. I will return to Leibniz in Chapter 3. For now, remember his two digits.

They are patient.

They will wait.

The Stick That Built the World

Before Charles Babbage, I want to spend a moment on a machine that almost no history of computing mentions, despite the fact that it was the most widely used computational tool in the world for three and a half centuries. The slide rule was invented by William Oughtred around 1622.

It is an analogue calculator.

It does not count discrete units the way an abacus or a Pascaline does. Instead, it represents numbers as physical distances along a logarithmic scale, and performs multiplication and division by sliding one scale against another.

Picture two rulers laid side by side, where the markings are compressed at the higher numbers. Slide one ruler along the other to the right distance, and the answer appears at the point where the scales align. No gears. No moving parts beyond the sliding motion itself. A twelve-inch stick that could multiply six-digit numbers in seconds. For 350 years, the slide rule was the instrument of engineering and science.

Every bridge designed before roughly 1975 was calculated on a slide rule.

Every ship's course charted by the Royal Navy.

Every artillery trajectory in two world wars.

Every structural beam, every electrical circuit, every chemical process designed in the industrial age. The engineers who built the Panama Canal used slide rules. The engineers who designed the Apollo spacecraft used slide rules. An entire civilization's physical infrastructure was computed on an instrument that could fit in a shirt pocket. The slide rule died in the mid-1970s, killed almost overnight by the electronic pocket calculator. Hewlett-Packard introduced the HP-35 in 1972 — the first scientific calculator that could fit in a pocket — and within five years the slide rule was extinct.

Not declining. Extinct. Manufacturing ceased. Training stopped. A tool that had been the universal instrument of technical professionals for longer than the United States had existed disappeared in half a decade. The slide rule's death is the purest example in computing history of what Chapter 1 called a substrate transition. The slide rule did not get worse.

It did not fail at its job.

It was replaced by a machine that operated on fundamentally different physics — electronics instead of mechanical sliding — and the new physics was so much faster, so much cheaper to reproduce, and so much easier to use that the old substrate became instantly obsolete. This is what a phase transition looks like: not a gradual decline, but a cliff.

The pattern will repeat with vacuum tubes, again with mainframes, and again — soon — with silicon itself.

The Engine That Could Not Be Built

Charles Babbage was, by any measure, one of the most remarkable minds of the nineteenth century.

He was also, by nearly every account, one of the most difficult human beings in London — a city that was not short of difficult human beings.

He quarrelled with organ grinders.

He quarrelled with Parliament.

He quarrelled with the craftsmen building his machines, the funders supporting his work, and the colleagues who admired him.

He was brilliant, obsessive, tireless, and possessed of a temperament so abrasive that he managed to alienate nearly everyone who tried to help him. His obsession began with errors. In the early nineteenth century, the British economy, its military, and its global navigation all depended on printed mathematical tables — logarithmic tables, trigonometric tables, astronomical tables, actuarial tables. These tables were computed by hand, by workers who were literally called “computers” — human beings whose job was to perform calculations according to fixed procedures and record the results.

The work was divided into simple steps, each performed by a different person, in what amounted to an assembly line for arithmetic.

The results were then typeset, printed, and distributed. At every stage — calculation, transcription, typesetting — errors crept in.

And these errors had consequences.

A wrong number in a navigation table could send a ship onto rocks.

A wrong number in an actuarial table could bankrupt an insurance company. Babbage, who had spent hours verifying tables and finding mistakes in nearly every volume he checked, reportedly exclaimed: “I wish to God these calculations had been executed by steam.” That wish became the Difference Engine — a mechanical device designed to compute polynomial functions automatically and stamp the results directly into copper printing plates, eliminating human error at every stage from calculation to publication.

The British government, recognising the strategic value of accurate tables for the Admiralty, funded the project with £17,000 — roughly £2 million today.

It was the largest government investment in a computing project in history. The Difference Engine was never completed. Babbage’s designs required gears machined to tolerances that the best craftsmen in England could not consistently achieve. Parts

that should have meshed smoothly jammed. Components that should have aligned precisely drifted. Babbage, characteristically, blamed the craftsmen. The craftsmen blamed Babbage's impossible specifications. The government, having spent £17,000 with nothing to show for it, stopped funding and never got its money back.

But by the time the Difference Engine collapsed, Babbage had already conceived something far more ambitious. Something that would not be fully understood for another century.

The General-Purpose Machine

The Difference Engine was a specialist.

It could compute polynomial functions and nothing else.

The machine Babbage began designing in 1834 was a generalist.

He called it the Analytical Engine, and it was, on paper, the first general-purpose computer ever conceived. The parallels to a modern computer are so precise they border on the eerie. The Analytical Engine had a *mill* — the part that performed arithmetic operations.

In a modern computer, this is the processor.

It had a *store* — a set of columns of gears that held numbers for use in calculations.

In a modern computer, this is memory.

It received instructions from punch cards — directly inspired by the Jacquard loom, a French weaving machine that used sequences of punched cards to control which threads were raised and lowered, allowing complex patterns to be woven automatically.

In a modern computer, the punch cards are the program. And it had an output mechanism that could print results or punch them into cards for later use. Processor. Memory. Program. Output. These are the four components of every computer built in the twentieth and twenty-first centuries. Babbage designed all four in the 1830s, using nothing but mechanical gears and the principle of the Jacquard loom.

He even designed the Engine to support conditional branching — the ability to change its sequence of operations based on intermediate results. In modern terms, he designed “if-then” logic. In the 1830s. The Analytical Engine was never built either. The same manufacturing limitations that defeated the Difference Engine made the far more complex Analytical Engine even more impractical. Babbage spent the rest of his life refining the design, filling notebooks with increasingly detailed plans for a machine that the technology of his era simply could not construct.

He died in 1871, having built only fragments.

The machine that contained every essential element of a modern computer existed for forty years as nothing but drawings and frustration.

The Poet's Daughter

In 1833, at a London dinner party, a seventeen-year-old named Ada Byron met Charles Babbage.

She was the daughter of Lord Byron, the most famous poet of his era, though she had never known him — her parents separated weeks after her birth and Byron died when she was eight. Her mother, determined that Ada would not inherit her father's temperament, had her educated rigorously in mathematics and science.

It was an unusual program for a young woman in the 1830s.

It produced an unusual mind. Babbage showed her the portion of the Difference Engine he had built — a working section of gleaming brass gears — and she was captivated. Not by the spectacle, which impressed most visitors as a kind of mechanical parlour trick, but by the principle.

She understood, immediately, what the gears were doing. And she wanted to know what else they might do. The relationship that followed — sustained over years of letters, conversations, and shared intellectual ambition — would produce one of the most important documents in the history of computing. In 1842, an Italian military engineer named Luigi Menabrea published a paper in French describing the Analytical Engine, based on lectures Babbage had delivered in Turin. Ada, now married and known as Ada Lovelace, was asked to translate the paper into English.

She did — and then added a set of notes that were more than twice the length of the original paper.

It is these notes that secured her place in history. In Note G, Lovelace described a detailed step-by-step procedure for using the Analytical Engine to compute Bernoulli numbers — a sequence of values important in number theory. The procedure specified the exact sequence of operations the machine should perform, the order in which cards should be fed, and the way intermediate results should be stored and reused.

It is recognized today as the first computer program ever written. Lovelace wrote software for a machine that did not exist, and would not exist for another hundred years.

But her most important insight was not the algorithm.

It was a conceptual leap that Babbage himself never made. Babbage saw the Analytical Engine as a mathematical tool — a machine for computing numbers with perfect accuracy. Lovelace saw something larger.

She wrote that the Engine could act upon “other things besides number,” provided that those things could be expressed in relationships the machine’s operations could capture. Music, she suggested, could be composed by a machine if the fundamental relationships of harmony and composition were expressed in a form the Engine could process. Patterns, logic, symbols of any kind — not just quantities — were within its reach. This was the first articulation of general-purpose computing. Not a machine that calculates. A machine that manipulates symbols according to rules.

The symbols can represent numbers, but they can also represent words, musical notes, images, logical propositions, or anything else that can be formally described. The power of the machine lies not in what it calculates but in what it can represent. Lovelace saw this in 1843, looking at a machine made of brass that had never been fully built, and described a future that would not arrive for over a century.

She also saw the limits. In the same notes, she wrote what may be the most prescient sentence in the history of artificial intelligence: “The Engine has no pretensions whatever to *originate* anything.

It can do whatever we know how to order it to perform.” The machine does not think.

It executes. The intelligence remains with the person who writes the instructions.

We will return to this sentence later, when the machines begin to do something that looks, to some observers, remarkably like originating.

*“The Engine has no pretensions whatever to originate anything.

It can do whatever we know how to order it to perform.” — Ada Lovelace, 1843*

The Power in the Tables

Return to the reason Babbage started building in the first place: the tables. The printed mathematical tables of the nineteenth century were not academic documents.

They were instruments of national power. The British Admiralty’s *Nautical Almanac*, published annually, contained the astronomical and navigational data that every Royal Navy captain needed to determine his ship’s position at sea. Accurate tables meant accurate navigation. Accurate navigation meant ships arrived where they intended, trade goods reached their markets, and naval fleets could be coordinated

across oceans. Errors in the tables meant ships ran aground, cargoes were lost, and battles were fought in the wrong place. Britain's global empire was, in a very real sense, a computational achievement.

The nation that computed best navigated best. The nation that navigated best traded best. The nation that traded best projected power farthest. The link between computation and geopolitical dominance was not metaphorical.

It was as direct as the link between a printed number in a table and the course a helmsman set at dawn. The power thread from the first chapter extends here. The shepherd who could count his flock had an advantage over the shepherd who could not. The priest who could predict an eclipse held power over those who could not. And the nation that could compute accurate navigational tables held dominion over seas that other nations could not safely cross. In every era, the machine belongs to whoever holds it.

And whoever holds it extends their reach — across a pasture, across a sky, across an ocean. The reach has grown longer with every chapter of this story. In the chapters that follow, it will grow longer still.

The Ceiling

The mechanical era of computation — from Pascal's brass box in 1642 to the last slide rule manufactured in the 1970s — spanned over three centuries. In that time, humanity built machines of astonishing ingenuity.

But every one of them ran into the same wall: the speed of metal. A gear can only turn so fast before friction defeats it. A spring can only store so much energy before it breaks. A cam can only execute so many steps before its complexity exceeds what can be manufactured. A simple calculation on the Analytical Engine, had it been built, would have taken seconds or minutes. A complex computation — the kind that modern computers perform billions of times per second — might have taken weeks.

The substrate had reached its ceiling. The next leap required something that moved faster than metal, something that could switch states in millionths of a second rather than tenths, something that did not depend on the physical movement of objects through space.

It required electricity. And when electricity arrived, it would bring with it a question that Leibniz had answered 250 years earlier, though nobody had been paying attention: what is the simplest possible language a machine can speak?

Two digits. Zero and one. On and off. The patient binary had finally found its machine.

PART II — THE ELECTRONIC REVOLUTION

Chapter 3 — The Switch · Chapter 4 — The Crystal

* * *

CHAPTER WHJ-3

The Switch

On Claude Shannon, the electronic computer, and why every machine speaks binary

The Switch

On the moment computation became electric, and the answer to a question most people have never thought to ask.

In the autumn of 1937, a twenty-one-year-old graduate student at the Massachusetts Institute of Technology submitted a master's thesis with a title that sounded almost comically dry: "A Symbolic Analysis of Relay and Switching Circuits." The student's name was Claude Shannon.

He had spent the summer working at Bell Telephone Laboratories, where the relay — an electrical switch operated by an electromagnet — was the fundamental building block of the telephone network.

He had also been studying Boolean algebra, the nineteenth-century system that reduced logic itself to mathematics using only two values: true and false.

Two different worlds.

Two different disciplines.

Nobody had connected them. Shannon connected them. A relay has two states: open and closed. Boolean algebra has two values: true and false. A circuit with two relays wired in series — one after the other — completes only if both are closed. That is an AND gate. A circuit with two relays wired in parallel — side by side — completes if either is closed. That is an OR gate. A relay that is normally closed and opens when current is applied is a NOT gate.

Boolean logic, the abstract mathematics of true and false, could be physically built from switches and wire.

It sounds simple when stated plainly.

It was not simple. George Boole had published *The Laws of Thought* in 1854 — eighty-three years earlier — showing that logical reasoning could be written as a system of equations using only two values. For eighty-three years, nobody could think of anything practical to do with it. Relays had been everywhere for decades — in telephone exchanges, industrial controls, telegraph systems. Everyone understood how they worked.

But nobody had looked at a relay and seen a truth table.

Nobody had looked at a logic textbook and seen a circuit diagram. Shannon, at twenty-one, saw both at the same time.

“Shannon saw what no one before him had seen: Boolean logic and electrical circuits were the same thing. Abstract truth and physical switches obeyed the same rules.” This was the bridge between Leibniz’s patient binary and the physical world. Leibniz had shown, in 1679, that any number could be represented with two digits. Boole had shown, in 1854, that any logical statement could be expressed with two values.

Shannon showed, in 1937, that any circuit of electrical switches could implement those logical statements physically.

Three insights, spanning 258 years, clicked together like the gears Babbage could never quite get to mesh.

Why Binary Won

A question almost nobody asks, despite the fact that the answer shapes virtually every piece of technology on Earth: why does every computer speak a language of only two words? Your phone, your laptop, the servers behind every website you visit, the systems that manage air traffic and stock exchanges and hospital ventilators — all of them, without exception, represent every piece of information as sequences of ones and zeros.

Every photograph you have ever taken, every song you have ever streamed, every message you have ever sent — all of it has been translated, at the machine’s most fundamental level, into an astronomically long string of two symbols. Binary is not a law of nature.

It is not mathematically superior to other number systems. The number ten can be represented just as accurately in base-10, base-2, base-8, or base-16. No base is more correct than any other. So why did binary win?

The answer is not mathematics.

The answer is physics. A switch is either open or closed. A vacuum tube is either conducting or not conducting. A transistor is either on or off.

Two states.

Two digits.

One and zero. Binary won because it matched the physics of the available substrate.

If the available substrate had had ten reliable states, we would use base-10 computing. In fact, there were early experiments with ternary computers — the Soviet Setun, built in 1958, used three states — but the engineering advantages of binary in electronic circuits were so overwhelming that alternatives could not compete. Binary is a concession to atoms, not to mathematics. This will matter enormously when we encounter a machine whose fundamental unit is not a switch with two states but a quantum object that can exist in a continuous superposition of both states simultaneously.

When the physics changes, the language may change with it. Binary is not forever.

It is for now.

The Machine in the Country House

In January 1944, in a Victorian country estate called Bletchley Park, about fifty miles northwest of London, a machine called Colossus began operating.

It was built to solve a specific and desperately urgent problem: breaking the Lorenz cipher, a German encryption system used for high-level military communications between Hitler and his generals. The man who built it was Tommy Flowers, a General Post Office engineer. His proposal for a high-speed electronic computing machine had been rejected by the Bletchley Park leadership as impractical. Flowers built it anyway, partly with his own money.

He was not a wealthy man.

He was a telephone engineer who believed that vacuum tubes were more reliable than the Bletchley leadership understood, and who was willing to stake his own salary on being right.

He was right.

The first Colossus used approximately 1,500 vacuum tubes.

By the war's end, ten Colossus machines were in operation at Bletchley Park, the most advanced containing 2,500 tubes.

They were, by a considerable margin, the most complex electronic devices in the world. Colossus could read encrypted messages from punched paper tape at five thousand characters per second — the tape flew through the reader so fast it sometimes caught fire. The intelligence produced by Colossus contributed materially to the Allied war effort. Some historians believe it shortened the war by months. And then, at the war's end, the British government classified everything. The Colossus machines were dismantled. The blueprints were burned. The engineers and operators were sworn to secrecy under the Official Secrets Act.

Tommy Flowers was told to destroy his records.

He could not tell anyone what he had built.

He could not put it on his curriculum vitae.

He could not publish.

He could not even discuss the work with colleagues who had been in the same building. For thirty years — until the mid-1970s — the existence of Colossus was unknown to the public, to the academic computer science community, and to most of the government itself. The American ENIAC, which became operational about a year later, received credit as the first large-scale electronic computer. The British contribution was erased from the record. Flowers spent the rest of his career as a telephone engineer.

He never received significant recognition or financial reward for building the machine that helped win the war.

When he applied for a bank loan in the 1950s, he could not explain the gap in his professional record or the skills he had demonstrated. The most consequential work of his life was a secret he was forbidden to tell.

“The value of Colossus was not that it could compute. It was that the enemy did not know it could compute. Those who possess advanced computational capability do not announce it.” This pattern — classification, secrecy, the deliberate concealment of computational advantage — did not end with the war.

It continued through the Cold War, through the digital age, and it continues today.

It will become the central concern of this book's final chapters.

The Room That Drank a City's Power

While Colossus operated in secrecy in the English countryside, a very different machine was being built in the open, at the University of Pennsylvania's Moore School of Electrical Engineering.

It was funded by the United States Army, which needed a faster way to calculate artillery firing tables — the ballistic data that told gunners how to aim their weapons under varying conditions of range, wind, temperature, and elevation.

The machine was called ENIAC — the Electronic Numerical Integrator and Computer — and when it was completed in late 1945, it was a spectacle.

It contained 17,468 vacuum tubes.

It filled a room fifty feet long and thirty feet wide.

It weighed thirty tons.

It consumed 150 kilowatts of electricity — enough to power roughly a hundred American homes. There is an often-repeated story, likely apocryphal but revealing of the machine's reputation, that when ENIAC was switched on, the lights in an entire section of Philadelphia dimmed.

But ENIAC could perform five thousand additions per second. A human computer — the kind that had been calculating firing tables by hand — could perform roughly one addition every fifteen seconds. ENIAC was seventy-five thousand times faster than the human it replaced. The gap between human computation and machine computation, which had been modest in the age of mechanical calculators, had just become an abyss. Programming ENIAC was a physical act. There was no keyboard, no screen, no programming language. To set up a new calculation, operators had to physically reconnect cables between panels — essentially rewiring the machine for each new problem.

This work was done by six women: Kay McNulty, Betty Jennings, Betty Snyder, Marlyn Meltzer, Fran Bilas, and Ruth Lichterman.

They had been recruited from a pool of human computers — women who calculated firing tables by hand — and were given the job of figuring out how to make the machine do the same calculations electronically.

They received no manual, because none existed.

They learned the machine's logic by studying its circuit diagrams and essentially teaching themselves to program by reverse-engineering the hardware.

When ENIAC was unveiled to the public in February 1946, the six women who had programmed it were not introduced.

They appeared in photographs but were identified only as “refrigerator ladies” — models posing with the machine for the cameras. Their names were not included in

press materials. Their contribution was not acknowledged by the Army or the university.

It would be decades before historians recognized them as the first professional programmers of a general-purpose electronic computer. This is data, not an aside. At every stage of computing history, the people who do the foundational work of translating human intent into machine behavior are undervalued relative to the people who design the hardware or fund the project. This pattern is as persistent as any in this book, and it has consequences that extend far beyond historical credit.

It shapes who enters the field, who stays, and whose perspective informs the next generation of machines.

The Living Room in Berlin

There is one more machine from this era that deserves attention, not because it changed the world — it did not get the chance — but because it illustrates a truth from Chapter 1 that is easy to forget: technological progress can be lost.

In 1936, a young German engineer named Konrad Zuse, working in his parents' living room in Berlin, began building a programmable computing machine. By 1941, he had completed the Z3 — a device that used 2,600 electromechanical relays to perform floating-point binary arithmetic, read programs from punched film, and execute a set of instructions that included addition, subtraction, multiplication, division, and square root. Many computer historians consider the Z3 the first functional programmable digital computer. Zuse built it with almost no institutional support. The German military, consumed by the war effort, showed intermittent interest but provided little funding.

He sourced parts from wherever he could find them. His parents' apartment became a workshop. His friends helped assemble components in their spare time. In December 1943, an Allied bombing raid destroyed the Z3 and much of Zuse's workshop. His subsequent machine, the Z4, survived the war only because Zuse evacuated it to a village in the Bavarian Alps as Berlin collapsed around him.

He reportedly transported it in a military truck he talked his way into borrowing, hiding the machine among refugees fleeing the advancing Soviet army. Zuse's work was largely unknown outside Germany during the war and for years afterward. The country was divided, its scientific community scattered, its records fragmented.

By the time Zuse's contributions were widely recognized, the narrative of computing history had already been written around ENIAC, Colossus, and the

Anglo-American machines. An entire branch of the evolutionary tree had been bombed, buried, and rediscovered too late to change the story as it was told. Zuse's story is haunting for a specific reason.

It is a reminder that the history of computing is not a smooth line of progress.

It is a landscape shaped by war, by politics, by who gets funded and who gets forgotten, by which machines survive and which are destroyed. The Z3 was, by several important measures, ahead of its time.

It did not matter. The bombs did not care.

The Ceiling of Glass

The vacuum tube era lasted, as the dominant computing substrate, for roughly fifteen years — from the mid-1940s to the early 1960s. In that time, it transformed computation from a theoretical concept into an operational reality. Machines like ENIAC, UNIVAC, and the IBM 701 proved that electronic computation worked, that it was fast, and that it could be applied to problems ranging from ballistic trajectories to census tabulation to weather prediction.

But the vacuum tube had a problem that no amount of engineering could solve: it burned out. A vacuum tube works by heating a filament until it glows, releasing electrons that can be controlled to switch current on and off. That filament, like the filament in an incandescent light bulb, has a finite lifespan. A typical tube lasted a few thousand hours before failing.

In a machine with seventeen thousand tubes, this meant that on average, a tube failed roughly every few hours. The operators of early computers spent more time locating and replacing burned-out tubes than they spent running programs. There was also the heat. Seventeen thousand tiny furnaces in a single room generated enormous amounts of thermal energy. ENIAC required an industrial air conditioning system that consumed almost as much power as the computer itself.

And there was the size. Each tube was a discrete glass component that had to be individually wired into a circuit. You could not shrink a vacuum tube below a certain size without losing the vacuum or the electrical properties that made it function. A more powerful computer meant more tubes, which meant a bigger room, more wiring, more cooling, more failures, and more operators crawling through the machine with replacement tubes and soldering irons. The vacuum tube had reached its ceiling. Like the mechanical gears before it, the glass bulb had taken computation as far as its physics would allow.

To go further — faster, smaller, more reliable, less power-hungry — required a completely different kind of switch. Not a filament in a glass envelope. Something solid. Something that did not burn. Something that could be made very, very small. On December 23, 1947, in a laboratory in Murray Hill, New Jersey, three men found it.

What This Era Made Clear

The electronic era — from Shannon’s thesis through the twilight of the vacuum tube — established several truths that would govern every subsequent chapter of computing history.

The first is that the choice of physical substrate determines everything. Binary is not a mathematical necessity.

It is a physical one. The two-state switch was the simplest, most reliable component available, and the entire architecture of modern computing was built to match it.

When we encounter a substrate with different physical properties — and we will — the architecture will change accordingly. The substrate is not a detail.

It is the foundation.

The second is that computation and secrecy are deeply intertwined. Colossus was classified because the advantage of breaking enemy codes was too valuable to reveal. ENIAC’s first major task after the war was hydrogen bomb calculations. From the very beginning of the electronic era, computation was understood by governments as a strategic asset — not just a tool for science or business, but a source of military and intelligence advantage. This understanding has never changed.

It has only intensified.

The third is that the people who build and program the machines are consistently undervalued relative to the machines themselves. Tommy Flowers paid for Colossus partly out of his own pocket and was told to destroy his records. The six women who programmed ENIAC were unnamed in the press coverage. Konrad Zuse built the Z3 in his parents’ living room with scavenged parts and had it destroyed by a bomb.

The pattern is unmistakable: the human contribution to computing is perpetually discounted. Machines are celebrated. The people who make them work are footnotes. This pattern will not improve in the chapters that follow.

It will get worse. And by the time we reach the final chapters of this book, the question of who is credited, who is compensated, and who is forgotten will have

expanded from a question about historical fairness to a question about whether humans are necessary at all. And the fourth truth, the one that closes this chapter and opens the next: every substrate reaches a ceiling, and the ceiling is always physical. Gears could not spin fast enough. Vacuum tubes could not stay cool enough.

The next substrate would be a crystal of semiconductor material — something that did not heat, did not burn, did not shatter, and could be made unimaginably small.

It would become the most manufactured object in the history of the human species, and the story of how it came to exist begins with a piece of germanium, a strip of gold foil, a bent paper clip, and a scientist who was furious that he had not been in the room when it happened.

CHAPTER WHJ-4

The Crystal

On the transistor, the semiconductor revolution, and the moment computation became personal

The Crystal

On the most manufactured object in human history, and the approaching moment when we run out of room.

On the afternoon of December 23, 1947, in a laboratory at Bell Telephone Laboratories in Murray Hill, New Jersey, two physicists named John Bardeen and Walter Brattain performed an experiment that would reshape the world more thoroughly than any single invention of the twentieth century.

They pressed two thin strips of gold foil against a small crystal of germanium — a semiconductor, a material that conducts electricity under some conditions and blocks it under others — and observed that a small electrical signal applied to one gold contact could control a much larger current flowing through the other. The crystal could amplify.

They called it a transistor. The device was crude — a slab of crystal, two strips of foil, and a bent paper clip holding the assembly together.

It looked like something a talented child might build from a science kit.

It was, in function, a switch — the same kind of switch that a vacuum tube was, performing the same on-off binary operation that Shannon had mapped to Boolean logic a decade earlier.

But it was a switch made of solid material instead of heated gas in a glass envelope. No filament to burn out. No vacuum to maintain. No enormous heat to dissipate. A switch that was small, cool, reliable, fast, and — most consequentially — could be manufactured in bulk. There is a person notably absent from this moment: William Shockley, Bardeen and Brattain's supervisor. Shockley had directed the semiconductor research group with the explicit goal of finding a solid-state

replacement for the vacuum tube.

He was not in the room when the breakthrough happened.

He had contributed critical theoretical work, but the key experimental insight belonged to Bardeen and Brattain. And Shockley, a man of ferocious intellect and equally ferocious ego, could not accept that.

The Ego and the Valley

What happened next is one of the most consequential sequences of human pettiness in the history of technology. Shockley, determined to reclaim intellectual ownership of the transistor, essentially locked himself in a hotel room in Chicago for several weeks in early 1948 and developed an entirely different transistor design: the junction transistor, which sandwiched a thin layer of one type of semiconductor between two layers of another.

It was more elegant, more reliable, and more manufacturable than Bardeen and Brattain's point-contact design.

It was also, incontestably, Shockley's.

He made sure everyone knew it. All three men shared the 1956 Nobel Prize in Physics. The collaboration, such as it had been, did not survive the award. Shockley's behavior had already driven Bardeen to leave Bell Labs for the University of Illinois, where he would win a second Nobel Prize — making him one of only five people in history to win two Nobels. Brattain was reassigned within Bell Labs and spent his later career in relative obscurity. Shockley, meanwhile, decided to commercialize the transistor himself.

In 1956, he founded Shockley Semiconductor Laboratory.

He chose to locate it in Mountain View, California, near Palo Alto, partly because Stanford University had a growing engineering community, and partly — by several accounts — because his aging mother lived nearby. The most consequential industrial location decision of the twentieth century was influenced, at least in part, by a son who wanted to be close to his mum. Shockley recruited brilliantly.

He assembled a team of the best young semiconductor physicists and engineers in the country. And then he managed them catastrophically.

He was paranoid, abusive, and dictatorial.

He subjected employees to polygraph tests.

He publicly berated their work.

He refused to pursue the silicon-based research his own team believed was the path forward, insisting on a quixotic pursuit of a four-layer diode that almost nobody else thought was commercially viable.

In 1957, less than a year after the company's founding, eight of Shockley's top researchers walked out. Shockley called them "the traitorous eight." They called themselves free. With funding from Sherman Fairchild, they founded Fairchild Semiconductor — and in doing so, created the seed from which virtually the entire modern semiconductor industry would grow.

Two of the eight, Robert Noyce and Gordon Moore, would leave Fairchild in 1968 to found Intel. Another, Eugene Kleiner, would co-found Kleiner Perkins, one of the most influential venture capital firms in Silicon Valley history. By one widely cited estimate, more than sixty-five semiconductor companies can trace their lineage back to Fairchild.

"The most consequential industrial cluster in modern history exists partly because one man was brilliant and impossible to work with. Silicon Valley is the geography of a defection." I linger on this story because it illustrates something about the evolution of computing that the purely technical narrative misses.

Technologies do not evolve in a vacuum.

They evolve through human institutions, and human institutions are shaped by the personalities, decisions, and dysfunctions of the people who run them. Shockley's genius was real. His toxicity was also real. And the Silicon Valley that exists today — with its culture of defection, startup formation, and venture-funded risk-taking — is, in meaningful ways, a response to the experience of working for William Shockley. The valley's founding trauma shaped its founding culture.

The Switch That Does Not Burn

Set the human drama aside for a moment and consider what the transistor actually is, because its properties explain why it did not merely improve computing but fundamentally transformed it. A vacuum tube heats a filament to release electrons. A transistor controls electron flow through a solid crystal using electrical fields. No filament means no burnout. The most common failure mode of the vacuum tube — the thing that made ENIAC's operators spend their days crawling through the machine with replacement parts — simply does not exist in a transistor.

A vacuum tube generates substantial heat. A single transistor generates almost none. This means you can pack transistors far more densely without the machine melting itself. A vacuum tube switches in microseconds — millionths of a second. A transistor switches in nanoseconds — billionths of a second. A thousand times faster. And as transistors shrink, they get faster still, because the electrons have less distance to travel.

But the property that changed the world was not speed, not reliability, not size.

It was manufacturability.

A vacuum tube is a discrete handmade component.

Each one must be individually manufactured, tested, and wired into a circuit by a human being.

A transistor can be fabricated as part of a batch process — thousands, then millions, then billions at a time, etched onto a single slab of silicon using photographic techniques. Building a vacuum tube computer was like building a cathedral: every stone placed by a skilled craftsman, every joint fitted by hand, every finished structure unique. Building a transistor computer would become like printing a newspaper: a pattern defined once, reproduced millions of times at negligible marginal cost.

This is the shift from craft to industry, from artisanal to automated, from scarce to abundant.

And it is why, within a few decades, computational power would go from being the rarest resource on Earth to one of the most ubiquitous.

Putting It All on One Chip

The transistor was a revolution. The integrated circuit was the revolution's revolution.

By the late 1950s, transistors were being used widely, but each one was still a discrete component — a tiny object that had to be individually wired to other tiny objects on a circuit board. A computer with ten thousand transistors required ten thousand physical connections, each one a potential point of failure. Engineers called this the “tyranny of numbers.” In 1958, Jack Kilby, a newly hired engineer at Texas Instruments, had an idea so simple it was almost absurd: build the entire circuit — transistors, resistors, capacitors, and the connections between them — on a single piece of semiconductor material.

Kilby was alone in the lab.

He was a new hire who had not yet accrued vacation time, so he worked through the company holiday while everyone else was away.

He built a prototype in September 1958 using germanium.

It was rough — the components were connected by tiny gold wires bonded by hand — but it worked. A few months later, at Fairchild Semiconductor, Robert Noyce independently developed a different approach to the same idea. Noyce's version used a planar process — building the circuit in flat layers on a silicon wafer, with connections formed by depositing metal traces directly onto the surface. No hand wiring at all. Everything photographic, everything reproducible, everything scalable. Noyce's approach won, for a reason that by now should feel familiar: it matched the manufacturing substrate better. A planar process could be automated.

It could be scaled.

It could put not just dozens but eventually millions and then billions of components on a single chip. The integrated circuit did for transistors what the printing press did for letters — it turned individual handcrafted elements into a mass-reproducible pattern. Kilby and Noyce both received credit.

They never litigated against each other personally, though their companies fought over patents for years. Kilby received the Nobel Prize in Physics in 2000. Noyce, who died in 1990, did not — the Nobel is not awarded posthumously. Kilby, in his Nobel lecture, acknowledged Noyce's contribution.

It was a more gracious outcome than Shockley's generation had managed.

The Observation That Became a Prophecy

In 1965, Gordon Moore published a short paper in *Electronics* magazine.

He noted that the number of components on an integrated circuit had been doubling approximately every year since the integrated circuit was invented.

He projected the trend would continue.

In 1975, he revised the estimate to a doubling every two years. This observation became known as Moore's Law. Moore's Law is not a law of physics.

It is an observation about the rate of improvement in semiconductor manufacturing — an empirical trend, not a theoretical necessity. And yet it held, with remarkable accuracy, for over half a century. The reason is partly self-fulfilling: the entire semiconductor industry organized its research, its investment, and its product roadmaps around maintaining the pace. The prophecy sustained itself because the industry believed in it. The numbers this sustained effort produced are staggering. Intel's first commercial microprocessor, the 4004, was released in 1971 with 2,300 transistors.

In 1989, the Intel 486 contained 1.2 million.

In 2005, a dual-core Pentium contained 291 million.

In 2012, an eight-core Intel processor contained 2.3 billion.

In 2023, Apple's M2 Ultra chip contained 134 billion transistors. From 2,300 to 134 billion in fifty-two years. A factor of more than 58 million.

If automobiles had improved at the same rate as semiconductors over the same period, a car today would travel at approximately six million miles per hour, get 100,000 miles per gallon, and cost less than a penny. The comparison is absurd, which is precisely the point. No other technology in human history has improved by a factor of 58 million in half a century. Nothing else is close.

The Most Manufactured Object in Human History

The total number of transistors manufactured by the human species, across all chips, all devices, all years of production, is estimated at roughly 13 sextillion. That is 13 followed by 21 zeros. For comparison, the estimated number of grains of sand on Earth is roughly 7.5 sextillion.

We have manufactured nearly twice as many transistors as there are grains of sand on the planet. More than bricks. More than nails. More than any single component of any structure humans have ever built. And each one of those transistors is a switch. On or off.

One or zero. Leibniz's two digits, replicated 13 sextillion times, performing the binary logic that Shannon proved could be implemented in circuits, executing the computations that Babbage dreamed of and Lovelace foresaw, at speeds that would have struck every person in this book's first three chapters as indistinguishable from magic.

The Geography of Power

Manufacturing advanced semiconductors is one of the most complex industrial processes ever devised.

The machines that fabricate modern chips — extreme ultraviolet lithography systems, built exclusively by a Dutch company called ASML — cost approximately \$380 million each, weigh 180 tons, and require multiple Boeing 747 cargo flights to deliver.

They project patterns onto silicon wafers using light with a wavelength of 13.5 nanometres — light so energetic it must travel through a vacuum because air would

absorb it. The precision involved is often compared to hitting a golf ball on the moon from Earth, and even that analogy undersells it. Only three companies on Earth can manufacture the most advanced chips: TSMC in Taiwan, Samsung in South Korea, and Intel in the United States. Of these three, TSMC dominates overwhelmingly — fabricating over 90 percent of the world’s most advanced semiconductors.

Apple, NVIDIA, AMD, Qualcomm, and dozens of other companies design their chips but depend on TSMC to build them. TSMC’s fabrication plants are located primarily on the island of Taiwan. Taiwan sits in the Taiwan Strait, 100 miles off the coast of mainland China, in one of the most geopolitically contested bodies of water on Earth. The Chinese government considers Taiwan a breakaway province. The United States has maintained a policy of strategic ambiguity regarding Taiwan’s defense for decades. Military analysts on both sides of the Pacific have gamed scenarios in which a conflict in the Strait disrupts or destroys TSMC’s fabrication capacity.

If that happened, the global supply of advanced semiconductors would effectively cease. Not decline. Cease. No company could replace TSMC’s output in less than several years, if then.

Every smartphone, every server, every AI training cluster, every military system that depends on advanced chips would be affected. The global economy would experience a shock that would make the 2020 supply chain crisis look like a minor inconvenience. The power thread of the transistor era: the physical substrate of modern civilization is manufactured by one company, on one island, in one of the world’s most dangerous geopolitical flashpoints. The fragility is literal.

The machine that the transistor built — the entire global computational infrastructure — has a single point of failure, and that point is guarded by the ambiguous commitments of great powers who may not agree on its defense.

“The physical substrate of modern civilization is manufactured by one company, on one island, in one of the world’s most dangerous geopolitical flashpoints. This is not a metaphor for fragility. It is fragility.”

The Wall

We are running out of room. The most advanced transistors manufactured today are approximately 3 nanometres in their smallest dimension. A human hair is about 80,000 nanometres wide. A strand of DNA is about 2.5 nanometres across. A single silicon atom is approximately 0.2 nanometres in diameter. The transistors on a

cutting-edge chip are roughly 15 silicon atoms wide. Fifteen atoms. That is the distance between the current state of the art and the point at which transistors can no longer be made smaller. At this scale, electrons begin to exhibit quantum mechanical behavior.

They tunnel through barriers that should block them.

They leak from one transistor to its neighbour. The binary distinction between on and off — the foundation of every computer described in this book — becomes unreliable. The concern is not theoretical.

It is an engineering reality being confronted right now. The techniques being used to push beyond 3 nanometres — gate-all-around transistors, backside power delivery, new materials layered at atomic scales — are extraordinary feats of engineering.

But they are engineering around a physical wall, not through it. Each new process node costs more, takes longer, and delivers smaller improvements. Moore's Law is not ending because the industry lost its ambition.

It is ending because the substrate is approaching its physical limit. Silicon, like gears before it and vacuum tubes before that, is reaching its ceiling.

Every substrate reaches a ceiling.

Every ceiling triggers a search for a new substrate. The laws of physics do not negotiate.

What Comes Next

The transistor era gave us something no previous era of computing had produced: abundance. For the first time in the five-thousand-year history of computation, the machines became cheap enough, small enough, and reliable enough to escape the institution. Computation moved from government labs and corporate data centers into offices, then homes, then pockets. That story — the personal computer, the internet, the smartphone — is the subject of the next two chapters.

It is the most visible chapter of computing history, the one most people have lived through, and the one whose lessons are most immediately relevant to the question this book is building toward: what happens when computation stops getting cheaper and starts getting more concentrated?

Return one last time to that laboratory in Murray Hill, New Jersey, on December 23, 1947.

Two men, a piece of germanium, a strip of gold foil, and a bent paper clip. A device so small and unimpressive that few people in the room grasped its significance.

And a supervisor who was not present, whose wounded pride would set in motion a chain of defections, startups, and innovations that would create the most powerful industrial cluster in human history. The transistor did not arrive through careful planning.

It arrived through physics, craftsmanship, ego, and accident. The valley it spawned was not designed.

It grew — the way organisms grow, through adaptation, competition, and the occasional mutation that turns out to be exactly what the environment required. Thirteen sextillion copies later, the crystal shows no sign of surrendering its dominance.

But the wall is visible now. Fifteen atoms wide. And on the other side of that wall, the physics changes. The switches stop being simply on or off.

They become something stranger — a machine that can be both at once.

But that machine is still several chapters away. First, the transistor's abundance must do what abundance always does: escape the laboratory, reach the public, and change everything.

PART III — PERSONAL & CONNECTED

Chapter 5 — The Escape · Chapter 6 — The Web

* * *

CHAPTER WHJ-5

The Escape

On the personal computer, the democratization of computational power, and the garage mythology

The Escape

On the moment the computer stopped belonging to institutions, and the great redistribution that lasted about thirty years before it reversed.

In late 1974, in a small office in Albuquerque, New Mexico, an engineer named Ed Roberts was staring at the end of his company. MITS — Micro Instrumentation and Telemetry Systems — had been selling calculator kits by mail order, but Texas Instruments had entered the market with finished calculators at prices Roberts could not match. The company was \$300,000 in debt. Roberts had one idea left: a computer kit, built around Intel's new 8080 microprocessor, cheap enough that electronics hobbyists might buy it.

He called it the Altair 8800, named by the twelve-year-old daughter of one of his co-workers after a planet in *Star Trek*.

He expected to sell a few hundred. The January 1975 issue of *Popular Electronics* put the Altair on its cover.

Within weeks, MITS received thousands of orders. Roberts could not fill them fast enough.

The machine itself was, by any objective measure, useless — a blue metal box with toggle switches and blinking lights, no keyboard, no screen, no software. You programmed it by flipping switches one bit at a time.

It could not do anything a mainframe could not do millions of times faster. And it changed the world, because it was yours. To understand why a useless machine mattered, you need to understand the world it entered.

The Priesthood

For the first thirty years of the electronic era, a computer was something you visited. You did not own one. You did not carry one. You went to it, in a specially constructed room with a raised floor and controlled air, and you submitted your work to the people who operated it.

Then you waited. The dominant machine of this era was the mainframe, and the dominant company was IBM.

By the mid-1960s, IBM controlled roughly seventy percent of the computer market worldwide. Its machines were expensive — hundreds of thousands to millions of dollars — physically enormous, and operated by a specialized class of professionals who controlled access to computation the way medieval clergy controlled access to scripture.

If you needed the computer to do something, you wrote your instructions on coding sheets, handed them to a keypunch operator, submitted your deck of cards through a window, and came back hours or days later. You did not touch the machine. This was not an accident of technology.

It was an architecture of control. Mainframes were designed to be shared resources, centrally managed, with every action logged, every user authenticated, every program approved by an administrator.

In 1964, IBM introduced the System/360 — a family of compatible computers that could all run the same software. The 360's compatibility meant that once a customer invested in IBM software, they were locked in. The switching cost was enormous. The ecosystem was self-reinforcing. The priesthood had its cathedral. The Altair shattered the cathedral's window. For the first time in the history of electronic computing, an individual could own a computer. Not time-share on an institutional machine. Not submit punch cards to a priesthood. Own it.

Put it on a desk. Program it however you wanted. No permission required. No gatekeeper. No waiting.

The Garage

The Homebrew Computer Club is one of those historical institutions whose influence is so wildly disproportionate to its appearance that the story sounds invented.

It met in a garage, then a community centre, then an auditorium at the Stanford Linear Accelerator Center. Its members were engineers, students, and hobbyists who shared a common obsession with making computers small, cheap, and personal. There

were no investors. No business plans. No pitch decks. Just people who wanted to build things and show each other what they had built. Among the regular attendees was a twenty-four-year-old engineer named Steve Wozniak, who worked at Hewlett-Packard during the day and attended Homebrew meetings at night.

Wozniak was, by all accounts, a gifted engineer — one of those people who can see the elegant solution to a complex problem the way a musician hears a melody.

When he saw the Altair 8800, he saw not what it was but what it could become. Over the next several months, Wozniak designed and hand-built a computer that was fundamentally different from the Altair. Where the Altair required toggle switches, Wozniak's machine had a keyboard. Where the Altair displayed blinking lights, Wozniak's machine could display text on a television screen.

He called it the Apple I. His friend Steve Jobs saw a business. Jobs had neither Wozniak's engineering talent nor his technical knowledge.

What Jobs had was an instinct — nearly unerring, though often infuriating to those around him — for how technology should feel to the person using it. Wozniak built machines for engineers. Jobs wanted to build machines for everyone. The Apple II, released in 1977, proved Jobs's instinct right.

It came in a moulded plastic case — not a bare circuit board, not a metal box with toggle switches.

It had a keyboard, a color display, and expansion slots.

It looked like something that belonged in a home, not a laboratory.

The Killer App

In 1979, a Harvard Business School student named Dan Bricklin was sitting in a lecture, watching his professor erase and recalculate numbers on a blackboard.

It was the same kind of tedious, error-prone arithmetic that had driven Pascal to build a calculator three centuries earlier. Bricklin's insight was the same as Pascal's: a machine should do this instead. Bricklin owned an Apple II. This was not incidental. The Apple II was one of the only personal computers in 1978 with enough memory — 32 kilobytes, expandable — to hold a meaningful grid of numbers.

It had a screen capable of displaying rows and columns of text.

It had Wozniak's floppy disk interface, so you could save your work. And it had those expansion slots that Wozniak had insisted on — the ones that let you add more memory as your spreadsheets grew. Wozniak had built a machine more capable than it needed to be for hobbyist tinkering. That excess capability is what made VisiCalc

possible. Jobs made the Apple II look approachable. Wozniak made it powerful enough to matter. With his friend Bob Frankston, who wrote the actual code in 6502 assembly language, Bricklin built VisiCalc — the first electronic spreadsheet.

It displayed a grid of rows and columns. Type numbers and formulae into the cells. Change one number, and every cell that depended on it recalculated instantly. VisiCalc was released for the Apple II in October 1979, and within a year it had become the reason people bought Apple IIs. Not the hardware. Not the color graphics. The spreadsheet. Accountants, financial analysts, and small business owners — people who had never considered owning a computer — bought one because VisiCalc ran on it.

The pattern would repeat so reliably that the industry gave it a name: the killer app. VisiCalc for the Apple II. Lotus 1-2-3 for the IBM PC. Desktop publishing for the Macintosh. Email for the internet. The hardware creates a capability. The software translates that capability into a problem that ordinary people recognize. The human need, not the technical feat, drives adoption. And here, once again, is the echo of our oldest story. Bricklin, watching his professor erase and recalculate on a blackboard, was seeing exactly what Pascal saw watching his father grind through tax arithmetic.

The problem is the same problem it has always been: the human mind imagines the answer but cannot reliably execute the calculation.

The machine fills the gap. The substrate changes. The gap remains.

The Man Who Should Have Been Bill Gates

In 1974, a computer scientist named Gary Kildall, working from a converted tool shed near the Naval Postgraduate School in Monterey, California, created CP/M — Control Program for Microcomputers.

It was, by any reasonable definition, the first commercially successful personal computer operating system. By 1981, CP/M ran on over three thousand different computer models. Kildall's company, Digital Research, was generating millions in annual revenue.

He had also invented the concept of the BIOS — the Basic Input/Output System — the thin layer of software between the operating system and the hardware that every computer still uses today.

In 1980, IBM decided to enter the personal computer market and needed an operating system. IBM's team approached Bill Gates, whose company Microsoft at that point primarily sold a BASIC programming language interpreter. Gates did not

have an operating system to offer.

He directed IBM to Kildall.

What happened next has been mythologized and distorted for over four decades. The popular version — that Kildall was off flying his private plane while IBM waited in his lobby — comes primarily from accounts Gates later gave to reporters. Kildall and his associates disputed this version for the rest of his life. The documented facts are these: IBM representatives arrived at Digital Research. Negotiations between IBM and Dorothy Kildall, Gary's wife and co-founder of the company, stalled — not over a missed meeting, but over the terms of the deal.

Digital Research's lawyer found IBM's non-disclosure agreement unacceptable. And on the substantive question of licensing, the two sides were fundamentally misaligned: Kildall wanted per-copy royalties — the compensation model of a creator who believed his work had ongoing value. IBM wanted a flat fee with no per-copy obligations. IBM returned to Gates. This time, Gates did not redirect them.

He acquired an operating system called QDOS — the Quick and Dirty Operating System, built by Tim Paterson at Seattle Computer Products by studying CP/M's published design — for \$75,000, renamed it MS-DOS, and licensed it to IBM. Critically, Gates retained the right to license MS-DOS to other manufacturers.

When IBM's PC became the industry standard and dozens of companies began producing compatible machines, every one of them needed MS-DOS. Gates had not invented the operating system.

He had captured the distribution channel. IBM did eventually reach an agreement with Kildall to offer CP/M alongside MS-DOS on the IBM PC.

But IBM priced CP/M at \$240 and MS-DOS at \$40. The market chose accordingly. CP/M faded. Microsoft became the most valuable software company in the world. And Gary Kildall, the man who invented the personal computer operating system and the BIOS, spent the rest of his life fielding questions about the day he allegedly went flying instead of meeting with IBM.

He died in 1994 at the age of 52. I tell Kildall's story not as a cautionary tale about business strategy, though it serves as one. I tell it because it reveals something about the power dynamics of computing's distribution phase that the licensing dispute makes precise. Kildall wanted royalties — the model that says the creator's contribution has ongoing value. Gates wanted the channel — the model that says whoever controls distribution controls the market. The royalty model rewards invention. The flat-fee-plus-distribution model rewards control.

In the personal computer era, control won. The inventor held the jar. The distributor held the gate. And the gate, as it turned out, was worth more than the jar.

“Kildall wanted royalties — the model that says the creator’s work has ongoing value. Gates wanted the channel — the model that says whoever controls distribution controls the market. The gate was worth more than the jar.”

Making the Invisible Visible

In the early 1970s, at Xerox’s Palo Alto Research Center — known as PARC — a team of researchers led by Alan Kay developed a radically different way for humans to interact with computers. Instead of typing commands into a blank screen, they created a visual world on the screen. Documents looked like documents. Folders looked like folders. You moved a pointer using a small device that rolled on your desk, and selected things by clicking a button.

They called it a graphical user interface, or GUI. The significance of the GUI is not aesthetic.

It is cognitive. Before the GUI, interacting with a computer required you to learn the computer’s language — a set of commands, each with specific syntax, that you typed into a prompt. You had to remember that “dir” listed files, that “cd” changed directories, that “del” deleted and there was no undo. The computer’s internal world was invisible, and you navigated it by memorising incantations. The GUI made the invisible visible. Files became objects you could see. Actions became gestures you could perform — dragging, dropping, clicking, resizing.

The computer’s internal state was represented as a spatial world that human spatial intuition could navigate. You did not need to memorize commands. You looked at the screen and saw what was there. The GUI did not make computers faster or more powerful. In purely technical terms, a command-line interface is more efficient for an expert user.

What the GUI did was lower the barrier to participation.

It transformed the computer from a tool for people who spoke its language into a tool for people who did not.

It was one of the largest expansions of who could compute since the invention of the abacus. Xerox, in one of the most celebrated failures of corporate strategy in business history, did almost nothing with this invention. PARC researchers built the Alto — a complete personal computer with a GUI, a mouse, ethernet networking, and

a laser printer — in 1973. Xerox management declined to commercialize it aggressively.

In 1979, Steve Jobs visited PARC, saw the Alto in action, and was captivated. Apple released the Macintosh, with its GUI and mouse, in 1984. Microsoft released Windows in 1985. Xerox remained a copier company.

The Great Redistribution

Consider what happened between roughly 1975 and 2005, because it is the most important power shift in the history of computing. For the entirety of computing history prior to the personal computer — from the Antikythera mechanism to the IBM mainframe — computation was an institutional resource.

It belonged to temples, governments, militaries, corporations, and universities. Access was controlled by gatekeepers. The personal computer shattered this. Between 1977 and 2005, the cost of a capable computer fell from several thousand dollars to several hundred. Moore's Law put exponentially increasing power into every successive generation. The GUI made them usable by non-specialists. Software gave ordinary people compelling reasons to use them.

By the early 2000s, roughly a billion people had personal computers. A teenager in her bedroom had more computational power than the entire United States government possessed in 1960. A small business owner with a \$500 computer could perform analyses that would have required a team of accountants and a mainframe twenty years earlier. Computation had escaped the institution.

It belonged, for the first time in five thousand years, to the individual. This was the most radical redistribution of a strategic capability in human history. The power that had been concentrated in institutions since the first priests computed eclipses was suddenly, within a single generation, scattered across a billion desks.

The Recapture

The personal computer put a machine on your desk that you controlled. You chose what software to install. You stored your data on your own hard drive. You could use the machine without an internet connection, without an account, without anyone's permission. The computer was, in the most meaningful sense, yours. The smartphone changed this. Not immediately, not explicitly, and not in a way that most people noticed.

But the shift was structural, and its consequences are still unfolding.

When Apple released the iPhone in June 2007, it was marketed as a phone, a music player, and an internet communicator.

It was all three, and much more.

It was a computer more powerful than the desktops of a decade earlier, small enough to fit in your pocket, connected to the internet at all times, and equipped with sensors — GPS, accelerometer, camera, microphone — that made it aware of its physical environment in ways no previous consumer device had been.

It was also, in a way that the personal computer never was, tethered. The iPhone's software came from Apple. The apps you could install were approved by Apple. Your data was stored on Apple's cloud. Your identity was managed by Apple. On Android, the same was true of Google. The smartphone looked personal, felt personal, and lived in your pocket.

But its fundamental architecture was a client-server relationship — your device was the client, and a platform controlled by a very large corporation was the server. The device in your hand was a window into someone else's system. The personal computer had given you a machine whose software you chose, whose data you stored locally, whose operation required no network connection and no account. The smartphone reversed every one of these properties. Your software came from an app store controlled by the platform. Your data was stored on the platform's cloud.

Your identity was managed by the platform. Your device was, architecturally, a terminal — a thin client connected to someone else's server, running someone else's software, storing your life on someone else's infrastructure. The liberation that the personal computer had achieved was being quietly unwound by a device that felt more personal than anything that had come before. Cloud computing completed the recapture. In the personal computer era, your data lived on your machine. In the cloud era, it migrated to remote servers operated by Amazon, Google, Microsoft, and a handful of other providers.

Your documents, your photographs, your emails, your financial records, your medical data — all of it moved from hardware you owned to infrastructure you rented. The economics were compelling: cloud storage was cheaper, more reliable, and accessible from anywhere. The strategic consequence was less visible: your data was no longer yours in any meaningful physical sense.

It existed on someone else's servers, subject to someone else's terms of service, accessible to someone else's employees, and discoverable by someone else's government.

“The personal computer put computation in your hands. The smartphone put it in your pocket. The cloud put it back in someone else’s building. The great redistribution lasted about thirty years.” By 2025, three companies — Amazon Web Services, Microsoft Azure, and Google Cloud — control the majority of the world’s cloud computing capacity.

The pattern is clear: computation starts concentrated. A new substrate makes it abundant. The abundance distributes power. And then, through economics, network effects, and the natural tendency of platforms to consolidate, power reconcentrates in new hands. Name this pattern: distribute, then recapture. The personal computer distributed. The cloud recaptured. The internet distributed. The platforms recaptured. Mobile distributed reach. App stores and data collection recaptured control.

Every cycle of apparent democratization has been followed by a reconcentration of power in fewer, larger, more opaque institutions. Not conspiracy. Economics. Platforms benefit from scale. Scale creates network effects. Network effects create lock-in. Lock-in concentrates power. The mechanism is so reliable that I have come to treat it as something close to a law of computational economics: abundance distributes capability, but economics recaptures control.

The Number in Your Pocket

By 2025, there were approximately 6.8 billion smartphone subscriptions worldwide, in a global population of roughly 8 billion. No previous invention — not electricity, not the automobile, not the telephone, not television — reached this fraction of the human population this quickly. Each of those smartphones is more computationally powerful than the machines that guided the Apollo missions to the moon. Each one contains more transistors than the entire global installed base of computers in 1970. Each one can access, via the internet, more information than was contained in the Library of Alexandria.

And each one reports to a platform. The duality of the personal computing era is plain.

It distributed computational power to nearly every human being on Earth.

It also created a surveillance and data infrastructure of unprecedented scope, concentrated in the hands of a few corporations and the governments that can compel them.

Every search you make, every location you visit, every message you send, every photograph you take — all of it flows through infrastructure you do not control, to

databases you cannot inspect, analyzed by algorithms you cannot see. The deeper question — who really benefits from a computer in every pocket — is more complicated than the hobbyists of the Homebrew Computer Club imagined.

They dreamed of liberation.

They achieved distribution. Whether distribution and liberation are the same thing is a question that the subsequent chapters will increasingly force us to confront.

What Comes Next

The personal computer gave individuals access to computation.

But a computer in isolation — disconnected, unable to communicate with other machines — is limited in the same way that a person who speaks but cannot listen is limited. The power of computation grows not just with the speed of a single machine but with the number of machines it can reach. The next chapter is the story of what happened when the machines learned to talk to each other.

It begins with a crashed message between two California universities, passes through a memo that a supervisor described as “vague but exciting,” and arrives at the most radical transformation of human communication since the printing press — along with the most radical concentration of informational power since the invention of the secret police. The internet was designed to be free.

What it became is a different story entirely.

CHAPTER WHJ-6

The Web

On the internet, distributed architecture, and the ideology of ungoverned connection

The Web

On the network that connected every computer on Earth, and the lesson that matters most for everything that follows.

On the evening of October 29, 1969, a graduate student named Charley Kline, sitting at a computer terminal in a lab at the University of California, Los Angeles, attempted to send a message to a computer at the Stanford Research Institute, 350 miles to the north. The two machines were connected by a dedicated telephone line and a pair of newly built devices called Interface Message Processors — refrigerator-sized boxes that handled the work of chopping data into packets and reassembling them at the other end.

The setup was called ARPANET, funded by the Advanced Research Projects Agency of the United States Department of Defense, and this was its first test. Kline's task was simple: type the word "LOGIN" and see if it appeared at Stanford.

He typed the L.

He called the Stanford operator on a separate telephone line. "Did you get the L?" "Got the L." He typed the O. "Did you get the O?" "Got the O." He typed the G.

The system crashed.

The first message ever transmitted on what would become the internet was two letters: L-O. "LO." As in: lo and behold.

They fixed the bug, successfully logged in about an hour later, and the event passed without fanfare. No press coverage. No celebration. Leonard Kleinrock, the UCLA professor who supervised the lab, later noted that they had not prepared any commemorative statement — unlike, say, the team behind the first telegraph message, who had the good sense to transmit "What hath God wrought?" This is, I have come to

believe, a reliable indicator of genuinely transformative technology. The people building it are usually too close to the problem to see the magnitude of the solution.

The significance arrives later, and it is always identified by people who were not in the room.

The Myth and the Reason

The most persistent myth about the internet is that it was designed to survive a nuclear war. This is half true and entirely misleading. In the early 1960s, a researcher at the RAND Corporation named Paul Baran published a series of papers on distributed communications networks. Baran was indeed concerned about nuclear survivability — the vulnerability of centralized telephone networks to a Soviet first strike.

He proposed a network architecture with no central hub: if any node was destroyed, traffic could route around the damage through surviving nodes.

But ARPANET was not built to implement Baran's proposal. ARPANET was funded by ARPA — a defense research agency, certainly, but its purpose was not nuclear survivability. Its purpose was resource sharing. In the 1960s, computers were enormously expensive, and different universities had different machines with different capabilities. ARPANET's original goal was to let researchers at one university use computers at another university remotely, without leaving their desks. The nuclear myth persists because it makes for a better origin story. A network born to survive apocalypse sounds dramatic and inevitable.

A network born to let academics share expensive machines sounds bureaucratic and contingent.

But the mundane story is more revealing, because it illustrates a pattern we have seen before: the first use of a computing breakthrough is almost never its important use. ARPANET was built for resource sharing.

It became the infrastructure of global communication, commerce, entertainment, surveillance, and warfare.

Nobody in that UCLA lab in 1969 anticipated any of this.

The Rules of the Road

In 1983, ARPANET adopted a protocol suite called TCP/IP — Transmission Control Protocol and Internet Protocol — designed primarily by Vint Cerf and Bob Kahn. TCP/IP did something deceptively simple: it established a universal standard for how data would be packaged, addressed, routed, and reassembled across any network,

regardless of the underlying hardware. A message from a computer in Tokyo could reach a computer in London, even if the two machines were completely different models running completely different software, as long as both spoke TCP/IP.

This was the internet's equivalent of the railroad gauge standardization. Before TCP/IP, computer networks were like the railroads of 1865 — dozens of incompatible systems, each optimized for local use, unable to interoperate. TCP/IP was the standard gauge.

It made every network part of one network. And just as railroad standardization unleashed an explosion of commerce across the American continent, TCP/IP unleashed an explosion of communication across the world.

But there is a critical design choice embedded in TCP/IP that deserves attention. TCP/IP is decentralized. There is no master server. There is no central authority that routes all traffic. There is no single point whose failure brings down the network.

Every node is, architecturally, equal. A message from a teenager's laptop and a message from the Pentagon traverse the same network using the same rules. The protocol does not know or care who is sending the message, what it contains, or how important the sender believes it to be.

It treats every packet the same way. This was a philosophical choice as much as an engineering one. The internet's architects — many of them academics steeped in the open, collaborative culture of university research — built a network that reflected their values: open, flat, egalitarian, resistant to central control. The architecture was, in a meaningful sense, democratic. No gatekeeper. No censor. No king. Remember this.

It will matter in about ten paragraphs.

The Memo

The internet and the World Wide Web are not the same thing, and the confusion between them obscures one of the most important inventions in the history of communication. The internet is the infrastructure — the physical cables, the routers, the protocols that allow computers to talk to each other.

It existed by the mid-1980s and was used primarily by academics, government researchers, and the military.

It was powerful but essentially invisible to ordinary people, because using it required technical knowledge and a tolerance for cryptic addresses and error messages. The World Wide Web is what made the internet usable by everyone. In March 1989, a British physicist named Tim Berners-Lee, working at CERN — the

European Organization for Nuclear Research, in Geneva, Switzerland — submitted a proposal to his supervisor titled “Information Management: A Proposal.” Berners-Lee’s problem was mundane. CERN employed thousands of researchers from dozens of countries, each using different computer systems, and the institutional knowledge was scattered across incompatible databases.

He wanted a system where any document could link to any other document, regardless of where it was stored, and anyone on the network could follow those links freely. His supervisor’s response, handwritten on the cover page of the proposal, has become one of the most famous marginal notes in the history of technology: “Vague but exciting.”

“Berners-Lee’s supervisor wrote on his proposal: ‘Vague but exciting.’ It is possibly the most understated assessment of a world-changing idea ever committed to paper.” Over the next two years, Berners-Lee built the three technologies that constitute the web.

HTML — Hypertext Markup Language — was the format for creating documents with embedded links. HTTP — Hypertext Transfer Protocol — was the set of rules for requesting and delivering those documents. And the URL — Uniform Resource Locator — was the addressing system that gave every document on the network a unique, human-readable name. None of these technologies was individually revolutionary.

What Berners-Lee did was combine them into a coherent system, build it on top of the existing internet infrastructure, and — critically — give it away.

He did not patent the web.

He did not license the technologies.

He released them freely, for anyone to use, extend, and build upon. This decision, more than any technical innovation, is why the web became universal.

It had no owner.

It had no gatekeeper. Anyone could create a website. Anyone could link to anyone else.

In 1993, a team at the University of Illinois released Mosaic — the first web browser that combined text and images in a single window and was easy enough for a non-technical person to use. The lead developer, Marc Andreessen, would later co-found Netscape, whose IPO in August 1995 — the stock nearly tripled on its first day of trading, reaching a market cap of roughly \$2.9 billion — marked the moment the business world recognized that the web was not a curiosity.

It was an economy.

The Explosion

What happened between 1995 and 2005 was, by any historical standard, an explosion. The number of websites grew from roughly 23,000 in 1995 to over 64 million by 2005. Global internet users grew from approximately 16 million to over a billion. Email replaced letters. Search engines replaced encyclopaedias. Streaming replaced record stores. The web did not merely add a new channel to existing industries.

It restructured them. The web spread as fast as it did because of two structural properties, both inherited from its design.

The first was zero barrier to entry. Creating a website required no license, no approval, no capital investment beyond a cheap computer and a connection. A teenager could create a website that reached the same global audience as a Fortune 500 company. A blogger in Jakarta and a newspaper in New York competed on the same network under the same rules. The web was, in its first decade, the most level playing field the world of information had ever seen.

The second was the link. A link costs nothing to create and nothing to follow.

Every link makes both the source and the destination more valuable. The more links, the more connections, the more valuable the network. This is what economists call a network effect — a system whose value increases with the number of participants. The telephone is the classic example: one phone is useless, two phones make a connection, a million phones make an indispensable infrastructure. The web worked the same way, except that the cost of adding a new node was essentially zero, so the network grew at a rate that no physical infrastructure could match.

But network effects have a dark twin, and it was hiding in plain sight.

The Capture

Network effects create value.

They also concentrate it. The mechanism is simple.

In a network, the nodes that have the most connections attract more connections — because being connected to a well-connected node is more valuable than being connected to an isolated one. Sociologists call this preferential attachment. You might know it by a simpler name: the rich get richer.

In a network with zero barriers to entry and strong network effects, the distribution of connections follows a power law — a tiny number of nodes accumulate an

enormous share of the connections, and the vast majority have very few. In the late 1990s, the web was genuinely decentralized. Millions of small websites, each independently operated. Search engines like AltaVista, Yahoo, Excite, and Lycos competed roughly equally. E-commerce was fragmented. Social interaction happened on thousands of independent forums and message boards. No single company dominated any major category.

By 2010, the landscape had consolidated into something almost unrecognisable. Google handled over 90 percent of global search. Facebook had over 500 million users and was the dominant social platform in most of the world. Amazon was the default starting point for online shopping. YouTube, acquired by Google in 2006, was the dominant video platform. A handful of platforms — fewer than ten — accounted for the majority of time spent online. The architecture of the internet remained decentralized. The economics of the internet had become radically concentrated.

The protocol still treated every packet equally. The market treated Google, Facebook, and Amazon as essential infrastructure and everything else as optional. The pipes were still democratic. The destinations were an oligarchy.

The lesson is the most important lesson in the history of the networked era: architectural democracy does not guarantee political democracy. You can build a system with no central authority, no gatekeepers, and no barriers to entry, and it will still consolidate into a small number of dominant players. The consolidation is not a corruption of the system.

It is a consequence of the system's own rules — network effects, preferential attachment, and the economics of scale. The open architecture creates the conditions for the very concentration it was designed to prevent. The internet is not the first open system in computing history to be captured by a small number of gatekeepers, and it will not be the last.

If the pattern holds — and I have seen nothing in my career that suggests it will not — then every open, democratic, decentralized computing system will eventually concentrate.

The question is not whether it will happen.

The question is who ends up holding the power when it does.

The New Gatekeepers

The platform companies that emerged from the web's consolidation are not like the mainframe gatekeepers of the 1960s.

They are more powerful, in ways that the mainframe companies never imagined. IBM controlled access to computation. Google, Facebook, and Amazon control access to attention. In an economy where attention is the scarcest resource, controlling its allocation is a form of power that no previous technology company ever wielded. A search engine that decides what you see first when you ask a question has more influence over public opinion than any newspaper editor, any television network, any government information ministry. Not because it tells you what to think — it is more subtle than that — but because it decides what you encounter.

The curation is the power.

Consider what the major platforms know. Google knows what you search for, which results you click, what you watch on YouTube, where you go via Google Maps and Android location services, what you write in Gmail, and what events are on your calendar. Facebook knows who your friends are, what you like, what you share, what you read, what you photograph, and where you check in. Amazon knows what you buy, what you browse but do not buy, what you read on Kindle, what you ask Alexa, and what you watch on Prime Video.

Apple knows what apps you use, what music you listen to, what messages you send, and the biometric data your watch collects from your wrist.

No institution in human history has ever possessed this depth and breadth of information about this many individual human beings. Not the Catholic Church at the height of the confessional. Not the Soviet KGB at the height of its surveillance apparatus. Not any government, any military, any intelligence agency in the history of the world.

The major technology platforms have achieved, through the voluntary adoption of consumer products, a degree of informational awareness that the most ambitious totalitarian state could only dream of — and they have achieved it without coercion, without legislation, without anyone in particular deciding that it should happen.

It happened because the services were useful.

It happened because the terms of service were long and nobody read them.

It happened because the cost was zero in monetary terms, and the price paid in data was invisible.

It happened because network effects made the dominant platforms indispensable, and leaving an indispensable platform carries social costs that most people are unwilling to bear.

It happened, in short, because the system worked exactly as designed. The design just produced consequences that the designers did not foresee and the users did not

notice until it was too late to easily reverse.

“No institution in human history has possessed this depth of information about this many individual human beings. The platforms achieved it without coercion. They achieved it because the services were useful and the price paid in data was invisible.”

The Vulnerability

There is one more consequence of the networked era that must be named, because it introduces a vulnerability that did not exist in any previous era of computing. A standalone computer can only be compromised by someone who has physical access to it. You have to be in the room. You have to touch the machine. The security perimeter is the walls of the building. A networked computer can be compromised by anyone, anywhere, at any time, from any distance. The security perimeter is the entire internet.

And the internet, by design, has no perimeter.

It was built to connect everything to everything. Security — the act of deciding what should not connect to what — was not a design consideration.

It was an afterthought. Not a criticism of the internet’s designers.

They were building a research network for a community of academics who trusted each other. Security against hostile actors was not a relevant concern in 1969.

But the network they built was so useful, so powerful, and so adaptable that it escaped the research community and became the infrastructure of global civilization — without ever being redesigned for that purpose. The internet that carries your banking transactions, your medical records, your military communications, and the control signals for your power grid is, at its core, the same trusting network that Charley Kline used to send two letters to Stanford in 1969. The security consequences have been exactly what you would expect from a system designed for trust and deployed in a world of adversaries.

Data breaches exposing hundreds of millions of records. Ransomware attacks that shut down hospitals and pipelines. State-sponsored cyber operations that penetrate government agencies and critical infrastructure. An entire ecosystem of cybercrime, cyber-espionage, and cyber-warfare that exists because the foundational infrastructure of the digital economy was built without a lock on the door. The networked era created two new forms of asymmetric power that did not exist before.

The first is informational: the platforms know everything about you and you know nothing about them.

The second is offensive: anyone with sufficient skill can reach into any networked system on Earth and extract, alter, or destroy its contents. The same openness that made the internet revolutionary made it vulnerable. The same architecture that resisted central control resists central defense.

The Pattern, Restated

The internet was a new substrate for computation — not in the physical sense (it ran on the same silicon that preceded it) but in the architectural sense.

It transformed the unit of computation from a single machine to a network of machines. A computer connected to the internet is categorically more powerful than the same computer in isolation, in the same way that a cell in a multicellular organism is categorically more capable than the same cell floating alone. This new substrate followed the pattern we have traced in the preceding chapters.

It expanded who could compute — from institutions with mainframes to anyone with a browser.

It produced an unexpected transformative application — the web, which no one anticipated when ARPANET was designed for resource sharing.

It hit a governance crisis — security, privacy, platform concentration, misinformation. And it concentrated power in new gatekeepers who look nothing like the old gatekeepers but hold even more influence. The distribute-then-recapture pattern played out in textbook fashion. The web distributed the power to publish. The platforms recaptured the power to be found. The internet distributed the power to communicate. The platforms recaptured the power to curate. Open architecture, concentrated economics. Democratic pipes, oligarchic destinations.

If this pattern holds — and I have found no reason in the historical record to believe it will not — then the question for every subsequent computing substrate is not whether concentration will occur, but how fast, how deep, and who benefits. The next chapter is about what happens when the machines on this network begin to learn.

When computation stops being something humans program and starts being something that emerges from data, patterns, and a process that even the engineers who design it cannot always explain.

The story of artificial intelligence is eighty years old.

It has died three times. The fourth life looks different from the first three — not because the ambition has changed, but because the substrate has finally caught up. And the power implications of a machine that learns are unlike anything in the five thousand years of computing history we have traced so far.

Because a machine that calculates extends the human mind. A machine that learns begins, however imperfectly, to resemble it.

PART IV — THE INTELLIGENT ERA

Chapter 7 — The Neuron · Chapter 8 — The Sensor

* * *

CHAPTER WHJ-7

The Neuron

On artificial neural networks, deep learning, and the substrate that learns from data

The Neuron

On machines that learn, three deaths and four lives of artificial intelligence, and the exiles who kept believing.

Every machine described in this book so far has one thing in common: it does what it is told. The abacus moves beads where your fingers push them. Babbage's Engine executes the operations specified by its punch cards. ENIAC performs the calculations its operators wire into its panels. A laptop runs the software a programmer wrote.

The machines have grown incomprehensibly faster, smaller, and more powerful across five thousand years of evolution.

But the fundamental relationship between human and machine has not changed: the human specifies.

The machine executes.

This chapter is about the moment that relationship began to shift. The idea is simple to state and extraordinarily difficult to implement: instead of telling a machine exactly what to do, show it examples of what you want and let it figure out the rules for itself. Instead of programming, teach. Instead of specifying, demonstrate. Instead of writing instructions, provide data and let the machine find the patterns. This idea is called machine learning, and the particular approach that has come to dominate the field is inspired — loosely, imperfectly, but genuinely — by the structure of the human brain.

It is called a neural network. And the story of how it went from a theoretical curiosity in 1943 to the most transformative technology of the 2020s is one of the strangest in the history of computation, because it died three times before it finally lived.

The First Life: The Perceptron

In 1943, a neurophysiologist named Warren McCulloch and a logician named Walter Pitts published a paper that proposed a mathematical model of a biological neuron. The real neuron in your brain is a cell that receives electrical signals from other neurons through connections called synapses.

If the combined strength of those incoming signals exceeds a threshold, the neuron fires — it sends its own signal out to other neurons.

If the incoming signals are too weak, the neuron stays silent. McCulloch and Pitts's insight was that this behavior could be modeled as a mathematical function. Inputs come in. Each input has a weight — a number that represents how important that input is. The weighted inputs are added together.

If the sum exceeds a threshold, the output is 1 (fire).

If not, the output is 0 (silent).

One artificial neuron. A tiny mathematical machine that takes numbers in and puts a number out. A single artificial neuron is not intelligent.

It is barely a calculator.

But McCulloch and Pitts showed that networks of these simple units, connected to each other in layers, could in theory compute anything — the same *anything* that Babbage's Analytical Engine and Turing's universal machine could compute. The power was not in the individual neuron.

It was in the connections. On July 8, 1958, the *New York Times* ran a story about a machine at the Cornell Aeronautical Laboratory in Buffalo, New York. The article described it as a device that the Navy expected would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence.

The machine was called the Perceptron.

It had been built by a psychologist named Frank Rosenblatt, and it could be *trained* — not programmed, trained — to recognize simple visual patterns. You showed it an image. You told it whether the image was, say, a square or not a square. And the machine adjusted the weights on its connections, strengthening the ones that led to correct answers and weakening the ones that did not.

After enough examples, the Perceptron could classify images it had never seen before. This was genuinely remarkable. For the first time in history, a machine had learned something from experience rather than being explicitly told. The weights — the knowledge — were not designed by a human.

They emerged from the data.

But the headline bore little resemblance to reality, and the hype it generated would prove catastrophic.

The First Death

In 1969, Marvin Minsky and Seymour Papert, two of the most respected computer scientists at MIT, published a book called *Perceptrons*.

They proved mathematically that a single-layer perceptron had fundamental limitations.

It could not learn certain simple logical functions. The most famous example was XOR — exclusive or — a function that outputs true when exactly one of its inputs is true, but not both. A trivially simple operation that any conventional computer could perform in a single step was provably impossible for a single-layer perceptron to learn. Minsky and Papert's proof was correct. Their conclusion — that neural networks were a dead end — was not. Multi-layer networks could solve XOR and far more complex problems.

But the book did not emphasize this possibility, and the field read the book as a death sentence. Funding agencies, looking for reasons to cut budgets, seized on the result. Government grants for neural network research dried up almost entirely. Academic positions disappeared. Researchers who continued working on neural networks found themselves marginalized within their own departments. Frank Rosenblatt, the man who had built the Perceptron, died in a boating accident on Chesapeake Bay on July 11, 1971.

He was forty-three years old.

He did not live to see his work vindicated. This was the first AI winter. Not a gradual cooling but a sudden freeze. The research did not slow down.

It stopped. The community did not shrink.

It scattered.

*“The first AI winter was not a gradual cooling.

It was a sudden freeze. The community did not shrink.

It scattered. And the few who continued believed in something the field had abandoned.”*

The Second Life and the Second Death

In 1986, a paper by David Rumelhart, Geoffrey Hinton, and Ronald Williams demonstrated a practical method for training multi-layer neural networks. The technique was called backpropagation — a way of calculating how much each weight in the network contributed to an error, and adjusting those weights accordingly, working backward from the output to the input. The idea was not entirely new — versions of it had been described by several researchers in the 1960s and 1970s — but the 1986 paper presented it clearly enough, and demonstrated it convincingly enough, that the field paid attention.

Imagine teaching a child to throw a ball at a target. The child throws. The ball lands three feet to the left. You say “a little more to the right.” The child adjusts. The ball lands one foot to the left. “A bit more.” Eventually, the throws converge on the target. Backpropagation is this process, automated. The network makes a prediction. The prediction is compared to the correct answer.

The difference is propagated back through the network, and every weight is nudged in the direction that would have reduced the error. Repeat this millions of times, with millions of examples, and the weights converge on a configuration that produces accurate predictions. Backpropagation revived the field. Neural networks could now be built with multiple layers, trained on real data, and applied to practical problems like handwriting recognition and speech processing.

The second life had begun.

But parallel to this revival, a different approach to artificial intelligence was absorbing most of the field’s energy and nearly all of its commercial investment.

They were called expert systems — programs that encoded human expertise as a set of if-then rules.

If the patient has a fever AND a cough AND a rash, THEN consider diagnosis X.

If the stock price drops below Y AND trading volume exceeds Z, THEN execute sell order. The knowledge was handcrafted by humans, not learned from data.

Every rule was explicitly written by an expert. Expert systems were the darling of the 1980s AI industry. Companies invested hundreds of millions of dollars. Japan launched a national Fifth Generation Computer project. Corporations bought specialized hardware to run expert systems. The hype was enormous. And once again, the technology could not deliver what the hype promised. Expert systems were brittle — they worked well within their narrow domain but failed catastrophically when they encountered situations their rules did not cover.

They were expensive to build, because every rule had to be extracted from a human expert and hand-coded.

They were difficult to maintain, because as knowledge changed, every affected rule had to be manually updated.

By the early 1990s, the expert systems market had collapsed. Japan's Fifth Generation project was quietly wound down. The specialized hardware became expensive paperweights. This was the second AI winter. The entire field of artificial intelligence had once again promised more than it could deliver. And for neural network researchers, the winter was doubly cold: not only had their field's broader umbrella lost credibility, but their specific approach was still regarded by the mainstream as a sideshow. The real work, the establishment believed, was in symbolic AI — hand-coded rules, logic, knowledge representation.

Neural networks were messy, opaque, and theoretically dubious.

The Exile

What happened during the second AI winter is, I think, one of the most instructive episodes in the history of computing, and it connects directly to the power dynamics this book has been tracing. A small number of researchers continued working on neural networks through the 1990s and early 2000s, largely without major funding, without mainstream academic recognition, and often in institutional settings far from the elite departments that had declared the approach a dead end. Geoffrey Hinton worked at the University of Toronto in Canada.

Yann LeCun worked at Bell Labs and later at New York University. Yoshua Bengio worked at the University of Montreal. These three, later called the “godfathers of deep learning,” shared a conviction that deep networks could learn representations of extraordinary complexity, if only the right training techniques and sufficient data could be brought to bear.

They were right.

But being right during a winter is a lonely experience. Hinton has described submitting papers to major conferences during this period and having them rejected repeatedly. The community was not interested. The approach was unfashionable. For a decade and a half, the deep learning researchers were academic dissidents — believers in a paradigm the mainstream had dismissed, sustained by a handful of government grants, a few supportive colleagues, and the stubborn conviction that the mathematics was sound.

The pattern that will recur in the final chapters: *innovation disperses during winters and concentrates during springs*.

When funding collapses, researchers scatter — to smaller universities, to foreign countries, to the margins of their field.

They work independently, quietly, cheaply. The ideas diversify because there is no dominant institution enforcing orthodoxy.

But when the breakthrough comes — when spring arrives — the largest institutions move immediately to capture the talent, the technology, and the strategic advantage. The springs are harvests. The winters are planting seasons. And the people who endure the winter are rarely the ones who profit most from the spring.

The Third Life

The spring arrived in 2012, and it arrived with a bang.

Every year, the computer vision community held a competition called the ImageNet Large Scale Visual Recognition Challenge. Research teams submitted systems that attempted to classify one million photographs into a thousand categories. Winning teams typically improved on the previous year's best result by a fraction of a percentage point, the way track records fall by hundredths of a second.

In 2012, a team from the University of Toronto submitted a deep neural network called AlexNet, designed by Alex Krizhevsky under the supervision of Geoffrey Hinton. AlexNet did not improve on the previous year's result by a fraction.

It demolished it. The error rate dropped by over ten percentage points — a margin of victory so large that it was as if an athlete had broken the world record by thirty seconds.

The result was not incremental.

It was categorical.

Two things made AlexNet possible that had not been available during the winter.

The first was data. ImageNet contained over fourteen million labeled images — photographs that humans had annotated by hand. Neural networks are hungry.

They learn from examples, and the more examples they see, the better they learn. Fourteen million was an ocean no previous generation had possessed.

The second was hardware. AlexNet was trained not on a conventional processor but on graphics processing units — GPUs. A GPU is a chip originally designed to render video game graphics.

It excels at performing the same mathematical operation on millions of data points simultaneously — precisely what training a neural network requires. A single GPU could perform in hours the training that would have taken weeks on a conventional

processor. The substrate had caught up to the algorithm. Hinton's ideas in the 1990s were sound. His algorithms were correct.

What he lacked was the data and the hardware to make them work at scale. The concept was right. The atoms — or rather, the transistors — were not yet sufficient.

When the transistors caught up, the ideas that had been dormant for twenty years erupted. The spring that followed was the most intense in AI history. Google hired Hinton in 2013. Facebook hired LeCun the same year. Bengio's lab in Montreal became a global magnet for talent.

Within five years, deep learning had been applied to speech recognition, language translation, drug discovery, protein structure prediction, autonomous navigation, game playing at superhuman levels, and dozens of other domains. Each success attracted more funding, which attracted more talent, which produced more success.

In 2018, Hinton, LeCun, and Bengio shared the Turing Award — computing's highest honor — for their work on deep learning.

The Transformer

In 2017, a team of eight researchers at Google published a paper with a title that was, in retrospect, breathtakingly accurate: “Attention Is All You Need.” To understand why it mattered, you need to understand the problem it solved. Language is sequential.

A sentence unfolds word by word, and the meaning of each word depends on the words that came before it and sometimes on the words that come after.

The word “bank” means one thing in “river bank” and a different thing in “bank account.” To understand language, a machine must understand context — the relationships between words, even words that are far apart in a sentence or a paragraph.

Previous neural network architectures for language processed text one word at a time, in order, passing information forward through a chain. This was slow and lossy — by the time the network reached the end of a long sentence, it had partially forgotten the beginning. The transformer replaced this sequential processing with something called *attention* — a mechanism that allows the network to look at every word in a sentence simultaneously and learn which words are most relevant to which other words, regardless of how far apart they are.

Think of reading a mystery novel. A sequential reader processes each page in order and tries to hold everything in memory. An attentive reader, reaching a crucial clue on page 200, can flip back to page 30 and recognize its significance. The transformer is

the attentive reader.

It does not process language word by word.

It processes it all at once, attending to the relationships between every pair of words in the input. This architecture turned out to be extraordinarily powerful, extraordinarily scalable, and — the detail that changed the world — extraordinarily responsive to being made larger.

When you train a transformer with more data, on more hardware, with more parameters, it does not just improve incrementally.

It develops capabilities that were not present at smaller scales.

It begins to summarize text, answer questions, write code, translate between languages, and perform tasks it was never explicitly trained to do. These emergent capabilities are among the most debated phenomena in modern computer science.

Nobody fully understands why they appear.

The Fourth Life

On November 30, 2022, a San Francisco company called OpenAI released a product called ChatGPT.

It was a large language model — a transformer trained on a vast corpus of text from the internet, books, and other sources — wrapped in a simple chat interface that anyone could use. You typed a question.

It typed an answer. In plain English. About anything. ChatGPT reached one hundred million users in approximately two months. No consumer technology in history had been adopted that fast. Not the telephone. Not television. Not the internet. Not the iPhone.

It was not adopted because it was perfect — it was not.

It made errors, fabricated facts, and could be manipulated into producing harmful content.

It was adopted because, for the first time, a machine could participate in a conversation in natural language, at a level of fluency and apparent comprehension that felt, to the human on the other side, genuinely responsive. I am choosing my words carefully here. I said “apparent comprehension” and “felt responsive.” Whether a large language model actually understands anything is one of the most contested questions in contemporary science and philosophy. Some researchers argue that LLMs are performing a very sophisticated form of pattern matching — “statistical parrots,” in one critic’s phrase.

Others argue that the distinction between “real” understanding and a sufficiently detailed model of understanding is unclear, and may not be meaningful. I do not propose to resolve this question.

But I want to reframe it, because I think the debate over machine consciousness distracts from the question that actually matters.

The question that matters is not whether the machine thinks.

The question is what the machine enables. The printing press did not think.

It transformed civilization. The steam engine did not think.

It industrialized the world. The relevant question about any tool is not what is happening inside it.

It is what becomes possible because it exists. And what becomes possible because large language models exist is a transformation of human productivity, human creativity, and human access to information that is still being measured, still being debated, and still unfolding.

The Cost of the Frontier

The power thread tightens to its narrowest point so far. Training a frontier large language model — the kind of model that produces the most capable AI systems available — requires three things in quantities that effectively exclude all but a handful of organizations on Earth.

The first is data. Frontier models are trained on trillions of tokens of text — a significant fraction of everything that has ever been written on the internet, supplemented by books, academic papers, code repositories, and other sources. Acquiring, cleaning, and preparing this data is a massive engineering operation.

The second is hardware. Training a frontier model requires thousands of the most advanced GPUs available — chips that cost tens of thousands of dollars each, manufactured almost exclusively by Nvidia using fabrication capacity at TSMC. The training process runs continuously for weeks or months, consuming enough electricity to power a small city.

The scarcity has a second consequence, beyond cost. Because the most advanced chips are the chokepoint — designed by one company, fabricated by one company, in quantities the whole world competes for — they have become the primary instrument of AI statecraft. By the mid-2020s, access to them was governed less by markets than by governments: export controls that restrict which nations may buy the fastest chips, imposed, loosened, and re-tightened as policy shifted. The details change from year to

year, and any specific rule will be obsolete before this book is. The structural fact does not change. The one place the world has found real leverage over artificial intelligence is not the model or the data but the silicon — which is to say, the narrowest part of the jar.

The third is money. Credible estimates of the cost of training a single frontier model range from \$100 million to well over \$1 billion, and the costs are rising with each generation. This is not the cost of running the model once it is trained. This is the cost of creating it. And each new generation typically requires several times more investment than the previous one. Count the organizations that can spend a billion dollars on a single experiment. The list is short.

It includes a few of the largest technology companies — Google, Microsoft, Meta, Amazon, Apple — and a handful of well-funded startups backed by those same companies and their investors.

It includes the national AI programs of the United States, China, and a few other nations.

It does not include most universities, most companies, most governments, or any individual. A new phenomenon in the history of computation, and it inverts the pattern we traced in earlier chapters. The personal computer era distributed computational capability because the cost fell. The AI era is concentrating computational capability because the cost of staying at the frontier is rising exponentially.

The machine in your pocket is more powerful than ever.

But the machine that matters most — the frontier model, the one that defines the boundary of what AI can do — is less accessible to you than any computing technology since the mainframe.

“The personal computer era distributed capability because the cost fell. The AI era is concentrating capability because the cost of the frontier is rising exponentially. This inverts the pattern of every previous era.”

What the Winters Teach

The three AI winters reveal a pattern that is directly relevant to what is happening now and what will happen next. In biological evolution, mass extinction events — the end-Permian, the end-Cretaceous — are catastrophic in the short term and creative in the long term. The Permian extinction killed roughly 90 percent of all species on Earth.

But it cleared the ecological space for the dinosaurs. The Cretaceous extinction killed the dinosaurs, but it cleared the space for mammals — and eventually for us. AI winters worked the same way.

The first winter killed the Perceptron hype but cleared space for backpropagation.

The second winter killed expert systems but cleared space for deep learning.

The third, quieter winter of the 1990s forced the field's most stubborn believers into exile, where they developed the ideas that would explode when the substrate caught up. Each winter killed the weak ideas and the opportunistic funding. Each spring validated the ideas that had survived. And each spring was more powerful than the last, because the surviving ideas had been refined during the winters by people who continued working when there was no reward for doing so.

After each previous winter, the spring was relatively open. New companies formed. New labs contributed. The barrier to entry was low enough that a graduate student with a good idea and access to a university computer could make a significant contribution. AlexNet, the breakthrough that launched the current era, was built by a PhD student and his professor. The current spring is different. The cost of entry has risen so dramatically that the barrier is not intellectual but financial. A brilliant researcher with a transformative idea but no access to a billion-dollar compute cluster cannot test that idea at the frontier.

The spring is open in principle and closed in practice. The ideas are free. The substrate is not.

The pattern from earlier chapters — distribute, then recapture — playing out at the level of the AI substrate itself. The algorithms are published. The papers are open. The code is often available.

But the ability to run the code at the scale that matters is concentrated in a handful of the wealthiest organizations on Earth. Open knowledge, closed capability.

It is the internet's story retold: democratic architecture, oligarchic economics.

Lovelace's Question, Revisited

Earlier in this book, I quoted Ada Lovelace's observation that the Analytical Engine "has no pretensions whatever to originate anything.

It can do whatever we know how to order it to perform." A large language model complicates this statement in ways that Lovelace could not have anticipated.

Nobody "ordered" ChatGPT to write a sonnet in the style of Shakespeare about quantum physics.

Nobody programmed a specific rule for combining those concepts. The model learned patterns from the vast corpus of text it was trained on, and it combines those patterns in ways that its creators did not specify and often cannot predict. Whether this constitutes “origination” is a question for philosophers. That it produces outputs no human explicitly designed is a matter of observation. I raise this not to settle the question but to mark the moment. For the first time in the five-thousand-year history of computation traced in this book, the relationship between human intent and machine output has become genuinely ambiguous.

Every previous machine did exactly what it was told. This machine does something in the vicinity of what it is told, in ways that emerge from its training rather than its instructions. The human is still in the loop — the prompt shapes the output.

But the machine’s contribution is no longer merely execution.

It is, at minimum, interpolation. Whether interpolation is a form of creativity is a debate for another book. That it changes the fundamental dynamic between humans and machines is the point that matters here. And with that changed dynamic comes a changed power structure.

When the machine merely executes, the human who gives the instructions holds all the power.

When the machine contributes something of its own — however we define that contribution — the question of who controls the machine becomes more complex. And when that machine is available only to those who can afford the substrate, the answer to who controls it becomes, increasingly, the answer to who holds the future.

What Comes Next

The neural network, in its current form as the large language model, is the most capable computing technology humanity has ever produced.

It is also the most concentrated.

It is also the most opaque — even its creators cannot fully explain why it produces the outputs it does. And it is also the most hungry, consuming data, hardware, and energy at a scale that increases with every generation.

But the neural network, for all its power, is still running on the same substrate we have been using since 1947: transistors on silicon, switching between one and zero. The current era of AI is pushing that substrate harder than any previous application. The largest training runs consume the output of entire power plants. The demand for GPUs has created global shortages and reshaped international trade policy. The

substrate is groaning. The next chapter is about what the computer opened its eyes to see — the smartphone and its fifteen sensors, the military signal the state gave away for free, and the surveillance infrastructure that six billion people adopted voluntarily.

Then, the final chapters of this book trace two substrate transitions that are arriving simultaneously: a machine that computes using the quantum behavior of subatomic particles, and a machine built from the molecules of life itself. Both raise a question that no previous substrate has forced us to confront: what happens when the next generation of computational power is so expensive, so complex, and so strategically consequential that it does not distribute at all?

CHAPTER WHJ-8

The Sensor

On IoT, edge computing, and the substrate that perceives the physical world

The Sensor

On the moment the computer opened its eyes, and the surveillance infrastructure that six billion people adopted voluntarily.

In 2009, a German politician named Malte Spitz asked his mobile phone carrier, Deutsche Telekom, for all the data they had collected about him.

They refused.

He sued.

After a court settlement, Deutsche Telekom delivered a spreadsheet.

It contained 35,831 rows. Each row represented a single instance when Spitz's phone had communicated with the network over a six-month period — from August 31, 2009 to February 28, 2010. Working with the German newspaper *Die Zeit* and a team of data visualization specialists, Spitz turned the spreadsheet into an interactive map. Press play, and you watch six months of a human life unfold in real time.

Every train ride.

Every night at home.

Every trip to another city.

Every time he lingered in one location or rushed through another. The map showed when he slept, when he traveled, when he worked, and where he spent Christmas.

It showed who he called and how often, though Spitz chose to redact those names to protect the privacy of the people in his life. Thirty-five thousand data points. Six months. A portrait of a human life more detailed than anything the person living it could have reconstructed from memory. And this was 2009 — before smartphones carried most of the sensors they carry today. Before GPS tracking was continuous. Before fitness trackers measured heartbeats and sleep cycles. Before facial recognition

and LiDAR and always-on microphones.

What Deutsche Telekom collected in 2009, with the comparatively primitive technology of that era, was the equivalent of a rough pencil sketch.

What your phone collects today is a high-resolution photograph, updated every second, of every dimension of your physical existence. Take out your phone. Hold it in your hand.

It weighs roughly six ounces. You probably think of it as a device for communication and entertainment. That is how it was marketed.

It is not what it is.

What you are holding is the most densely instrumented device in the history of the human species.

Fifteen Instruments

Inside that thin slab of glass and aluminium are, at minimum, the following sensors. An accelerometer, which measures how fast and in which direction the phone is moving. A gyroscope, which measures the phone's orientation and rotation in three-dimensional space. A magnetometer, which detects magnetic fields and functions as a digital compass. A barometer, which measures atmospheric pressure and can determine your altitude — which floor of a building you are on. A proximity sensor. An ambient light sensor. A GPS receiver, which triangulates your position on Earth to within a few metres by listening to signals from satellites orbiting twenty thousand kilometres overhead.

At least two cameras — front and rear — capable of capturing still images and video at resolutions that exceed what professional film cameras could produce twenty years ago. A microphone sensitive enough to pick up a whisper across a quiet room. A fingerprint scanner or facial recognition system that maps the geometry of your body. An NFC chip for contactless payments. Bluetooth, Wi-Fi, and cellular radios that continuously scan for nearby devices and networks. Some newer models add a LiDAR scanner that builds three-dimensional maps of the space around you using pulses of infrared light.

Some include ultra-wideband chips that can locate other devices to within centimetres. Some, paired with a watch on your wrist, measure your heart rate, blood oxygen, skin temperature, and sleep patterns. Fifteen instruments. Many of them running continuously. All of them generating data. And nearly seven billion of these devices are in active use worldwide.

The Blind Calculator

For the first sixty years of the electronic era, computers were blind, deaf, and numb.

They processed information, but they did not perceive anything. An ENIAC operator typed numbers in and got numbers out. A mainframe processed punch cards. A personal computer received keystrokes and mouse clicks.

Every piece of information that entered the machine was put there deliberately by a human being. The computer knew nothing about the world that the human did not explicitly tell it. A computer in 1995 could calculate the trajectory of a satellite but did not know whether it was sitting in a building or a field.

It could process a million financial transactions per second but did not know whether its operator was standing or sitting, happy or distressed, alone or in a crowd.

The machine was powerful and completely disconnected from physical reality. The smartphone changed this so completely that it is difficult, from the vantage point of 2026, to appreciate how radical the change was. The computer did not merely become mobile.

It became aware.

It knows where it is.

It knows how fast it is moving.

It knows what it can see and hear.

It knows who is holding it.

It exists, for the first time in the history of computation, in the physical world — not as an abstract processor but as an entity with senses.

*“For sixty years, computers were blind, deaf, and numb. The smartphone did not merely make the computer mobile.

It gave the computer senses.

It made the machine aware.”*

The Senses, Mapped

The parallel to biological evolution is so direct that I want to draw it explicitly, because it illuminates what is happening and what it means. For billions of years, the earliest life on Earth had no senses at all. Single-celled organisms drifted in the ocean, reacting only to direct chemical contact.

They were, in the terms of this book, blind calculators — processing information at the point of contact, with no awareness of what lay beyond their immediate surface. Then, over hundreds of millions of years, sensory organs evolved. Photoreceptor cells appeared — primitive light-sensitive patches that could distinguish brightness from darkness. These evolved into eyes of increasing sophistication, from the simple pit eyes of flatworms to the compound eyes of insects to the camera-like eyes of vertebrates. Ears evolved from vibration-sensitive cells in fish to the complex sound-processing organs of mammals.

A vast proliferation of ways to perceive the world at a distance. The effect was not incremental.

It was transformative. An organism that can see a predator approaching from fifty metres occupies a fundamentally different ecological niche than one that detects the predator only at the moment of contact. Senses do not just provide information.

They provide time — time to react, time to plan, time to choose. The organism that senses becomes an organism that anticipates. And an organism that anticipates survives better than one that merely reacts. The smartphone gave the computer this same transition. GPS is proprioception — the sense of knowing where your body is in space. The camera is vision. The microphone is hearing. The accelerometer and gyroscope are the vestibular system — balance, orientation, a sense of movement. The barometer is environmental awareness. Bluetooth and Wi-Fi scanning are something like echolocation — detecting nearby objects by sending out signals and listening for responses.

A computer with senses is not the same category of machine as a computer without them. A blind calculator is a tool you use when you choose. A sensing computer is an environment you inhabit. You use a tool. You live inside an environment.

The Signal That Made It Possible

Every location-aware service on every smartphone on Earth — every map, every ride-hailing algorithm, every geofenced advertisement, every fitness tracker that logs your morning run, every dating app that shows you who is nearby — runs on a signal that the United States military gave away for free. The Global Positioning System was built by the Department of Defense, beginning in 1973, as a military navigation tool. A constellation of satellites, each broadcasting a precise time signal, allowing a receiver to calculate its position on Earth to within a few metres.

In 1983, after the Soviet shootdown of Korean Air Lines Flight 007 — which had strayed into Soviet airspace due to a navigation error — President Reagan directed that GPS be made available to civilian aviation.

In 2000, President Clinton ordered the removal of Selective Availability — the intentional degradation of the civilian signal — making full-precision GPS accessible to anyone with a receiver. This was, by any measure, a remarkable act of public infrastructure. The state built the constellation, launched the satellites, maintained the ground stations, and distributed the signal globally, without charge. GPS created enormous public value: safer aviation, precision agriculture, emergency response, accessible navigation for billions of people.

It is one of the most consequential public goods of the twentieth century. And it is the foundational infrastructure of modern surveillance capitalism.

Every one of Malte Spitz's 35,831 data points was anchored by GPS.

Every location trail that allows a mobile carrier, a social media platform, or an advertising network to map a human life to a geographical grid runs on the signal the state distributed for free. The platforms did not build the constellation.

They captured the value it created. The state distributed the signal. The platforms recaptured the intelligence. The distribute-then-recapture pattern in its purest form. The personal computer distributed because it was cheap; the cloud recaptured the data. The internet distributed because the protocol was free; the platforms recaptured the attention. GPS distributed because the state gave it away; the platforms recaptured the location. In each case, the infrastructure that enabled the distribution was open, democratic, and available to all. In each case, the value was captured by the few who understood that the real product was not the signal, but the pattern of life the signal revealed.

The Bargain

The smartphone's sensor suite was not a secret.

Every phone's specifications list its sensors openly.

Nobody hid the GPS receiver or the accelerometer. The information was available to anyone who cared to look. Almost nobody looked. The reason is a bargain — implicit, never formally negotiated, and deeply asymmetric. The sensors enable services that people genuinely value. GPS enables maps. The camera enables photography. The microphone enables voice assistants.

Every sensor has a consumer-facing function that provides real, tangible, daily value. In exchange, the sensors generate a continuous stream of data about the person carrying the device — where they are, where they have been, how fast they are moving, what they are looking at, what they are saying, who they are near, how they sleep, how their heart beats. This data flows to the platforms that operate the device's software: Apple, Google, the developers of every app that has been granted sensor access.

You get navigation, and the platform gets your location history. You get a voice assistant, and the platform gets a record of your questions. You get fitness tracking, and the platform gets your biometric data. You get convenience. The platform gets surveillance. Both sides receive something of value.

But the values are not symmetric. You receive a service that makes your day slightly easier. The platform receives a data point that, aggregated across billions of users over years, produces a model of human behavior more detailed and more comprehensive than any ever constructed.

The Asymmetry

The preceding chapters traced how computational capability translates into power. The priest who predicts the eclipse. The navy that navigates by superior tables. The government that classifies its codebreaking machines. The company that trains the frontier AI model. In each case, the power derives from an asymmetry: one party can compute something that the other party cannot. The sensor era introduces a new and particularly potent form of this asymmetry.

It is not an asymmetry of computation.

It is an asymmetry of awareness. The platform knows where you are. You do not know where the platform's servers are. The platform knows what you search for, what you buy, who you call, how long you sleep, and where you were at 2:47 AM last Thursday. You do not know what the platform does with this information, who has access to it, how long it is retained, or what inferences are drawn from it. The platform's algorithms analyze your behavior continuously. You cannot see the algorithms.

You cannot audit them. You cannot verify their conclusions. You cannot appeal their judgments. Not the asymmetry of a tool and its user.

It is the asymmetry of an observer and its subject. And the subject — you, me, the nearly seven billion people carrying sensor-laden devices — volunteered for the

observation.

We were not compelled.

We were not coerced.

We were offered maps and music and messaging and we said yes. The most comprehensive surveillance infrastructure in human history was built not by a totalitarian government but by consumer electronics companies, adopted not through mandate but through desire. I find this more unsettling than the alternatives. A surveillance state imposed by force can, at least in principle, be resisted. A surveillance infrastructure adopted voluntarily, because it is genuinely useful and genuinely pleasant to use, offers no such clarity. You cannot resist something you enjoy.

You cannot fight something you chose. The chains are services, and the prison is convenience, and most of the inmates do not recognize the architecture because the view from inside is so comfortable.

The Data That Sees You

Your phone's location data, collected over months or years, reveals where you live, where you work, where your children go to school, which doctor you visit, which church or mosque or synagogue you attend, which political rallies you go to, who you spend the night with, and how often.

This is not inference.

This is geometry — a device that records its GPS coordinates every few minutes produces a map of your life that is more detailed than anything you could reconstruct from memory.

Your search history reveals your fears, your desires, your medical concerns, your financial anxieties, your relationship troubles, and your secret interests — the things you would never say aloud to another person but will type into a search bar at midnight because you believe the query is private.

It is not private.

It is stored.

It is analyzed.

It is used to build a profile of your interests and vulnerabilities that is, in many cases, more accurate than your own self-assessment. Your biometric data — your face, your fingerprint, your heart rate, your gait — is uniquely identifying and, unlike a password, cannot be changed.

If your password is stolen, you create a new one.

If your facial geometry is compromised, you cannot create a new face. Biometric data is permanent, personal, and irrevocable. And billions of people have handed it to technology companies in exchange for the convenience of unlocking their phone by looking at it. Your microphone data — even when you are not actively using a voice assistant — has been the subject of persistent concern. Major technology companies have acknowledged that human contractors reviewed recordings from voice assistants to “improve accuracy,” meaning that fragments of conversations conducted in the presumed privacy of your home were heard by strangers in offices you have never seen.

None of this data is collected in secret. All of it is disclosed, technically, in terms of service and privacy policies that average between five thousand and ten thousand words per document.

One study estimated that if a person actually read every privacy policy they encountered in a year, it would consume roughly 76 working days. The disclosure exists. The informed consent does not.

The Instrument of Others

The sensor data collected by consumer devices does not remain exclusively with the company that collected it.

It flows.

It flows to advertisers, who use it to target messages with a precision that no previous advertising medium could approach. A billboard is seen by everyone who passes it. A targeted advertisement is seen only by people whose data profile matches specific criteria — age, income, location, browsing history, recent purchases, recent searches, proximity to a competitor’s store. The advertisement does not just reach you.

It reaches you at the moment your data suggests you are most receptive.

It flows to data brokers — companies whose entire business model is the collection, aggregation, and resale of personal data. These companies operate largely without public awareness. Most people have never heard of Acxiom, Oracle Data Cloud, or Experian Marketing Services, yet these companies hold detailed profiles on hundreds of millions of individuals, assembled from sensor data, purchase records, public records, and other sources.

It flows to governments. Law enforcement agencies in many countries routinely purchase location data from commercial brokers, obtaining surveillance capability that

would normally require a court order if collected directly. Intelligence agencies have acknowledged using commercially available data for national security purposes. The distinction between government surveillance and commercial data collection has, in practice, dissolved. The government does not need to build a surveillance apparatus.

It can buy access to one that already exists.

And it flows, in ways that are difficult to trace and nearly impossible to control, into the training data of artificial intelligence systems.

The large language models described in an earlier chapter were trained on vast corpora of text from the internet — text that includes personal information, private communications that were inadvertently made public, and data scraped from platforms that users believed was shared only with friends.

Your words, your photographs, your public behavior — filtered through sensors, processed through platforms, and fed into machines that learn to predict and generate human behavior — are part of the substrate on which the next generation of AI is built.

The Organism in Your Pocket

When organisms first evolved eyes, the competitive landscape of life changed permanently. Before vision, predators and prey interacted only at close range.

After vision, the world expanded. An arms race began that drove a massive acceleration in the complexity of life on Earth. Many evolutionary biologists believe that the development of the eye was a primary trigger of the Cambrian explosion — the period roughly 538 million years ago when virtually all modern animal body plans appeared in a geologically brief window. Vision did not just add a capability.

It restructured the entire ecosystem. The smartphone's sensor suite is doing the same thing to the computational ecosystem. Before sensors, computation operated in an abstract space.

After sensors, computation operates in physical reality — aware of location, movement, sound, light, proximity, and the biometric identity of the person holding the device.

The consequences are restructuring every domain it touches: commerce, communication, transportation, healthcare, security, politics, warfare, and the basic texture of daily human experience. The phone in your pocket is not a tool.

It is, in a functionally meaningful sense, an organism — a device that senses its environment, processes information, communicates with other devices, adapts its

behavior based on what it perceives, and operates continuously whether or not you are actively using it.

It is not alive.

But it is not passive in the way that any previous computing device was passive.

It is attentive. And it is paying attention, always, to you.

The Bridge to What Comes Next

The evolution of computing has been traced from a shepherd's jar of pebbles to a device in your pocket that knows where you are, what you are looking at, how your heart is beating, and what you whispered to your phone at midnight. Eight chapters.

Five thousand years. Four fundamental operations — store, retrieve, compare, transform — that have never changed. And a single thread that connects them all: the machine belongs to whoever holds it, and whoever holds it sees further than those who do not. Everything in the first eight chapters of this book has happened. The pebbles, the gears, the vacuum tubes, the transistors, the networks, the neural networks, the sensors — all of this is history. The next four chapters are about what is beginning to happen and has not yet reached its consequence.

We are approaching two substrate transitions simultaneously, and that has never happened before.

The first is quantum computing — a machine that abandons the binary switch and computes using the quantum-mechanical behavior of subatomic particles.

The second is biological computing — a machine built not from silicon but from the molecules of life. Each of these transitions, on its own, would represent the most significant change in computational substrate since the transistor. Together, they suggest something that none of the previous eight chapters has prepared us for: a future in which the machine does not merely extend the human mind but begins to merge with the natural world — and in which the power dynamics we have traced through pebbles and gears and vacuum tubes and silicon reach a concentration that has no precedent in human experience.

“Over eight chapters, the jar has grown from a handful of pebbles to a global infrastructure. The next chapters ask a harder question: what happens when only a handful of institutions on Earth can build it, afford it, or even see it?”

PART V — BEYOND SILICON

Chapter 9 — The Qubit · Chapter 10 — The Molecule

* * *

CHAPTER WHJ-9

The Qubit

On quantum computing, superposition, and the substrate that may never distribute

The Qubit

On the end of ones and zeros, and the invisible arms race that may already be underway.

In 1981, Richard Feynman stood at a blackboard at MIT and said something that sounded, to the physicists in the room, like a complaint, and to everyone who came after, like a prophecy. The problem was simulation. Feynman wanted to simulate quantum-mechanical systems — the behavior of atoms, molecules, subatomic particles — on a computer.

But a quantum system with fifty particles requires tracking an exponentially large number of possible states — more states than any classical computer could store, let alone process. The mathematics was clear. The physics was mocking him.

Every time he tried to model nature on a classical machine, the machine choked on the complexity that nature handled effortlessly. His conclusion was characteristically blunt: nature isn't classical, dammit.

If you want to simulate nature faithfully, you need a computer that operates by the same rules nature does. You need a quantum computer. Everything you have read in this book so far is about machines that speak two words: one and zero, on and off, yes and no. The abacus. The Analytical Engine. Colossus. ENIAC.

Every transistor in every chip in every device on Earth. The shepherd's pebble was binary in its own way — present or absent, in the jar or not. For five thousand years, every computing substrate has been built on a foundation of discrete states. Something is, or it is not.

This chapter is about a machine that refuses this premise.

The Spinning Coin

A quantum computer does not process ones and zeros.

It processes something for which there is no everyday analogy that is fully accurate, but for which the closest useful approximation is this: imagine a coin spinning in the air. While it spins, it is not heads and it is not tails.

It is, in a real and physical sense, both possibilities at once. Only when it lands — only when you look — does it become one or the other. A classical bit is a coin that has landed. Heads or tails.

One or zero. A qubit — a quantum bit — is a coin in the air.

It exists in a *superposition* of states, carrying information about both possibilities simultaneously, right up until the moment of measurement.

It is not that we don't know which state the qubit is in.

It is that the qubit is, according to the laws of quantum mechanics, genuinely in both states at the same time. The physics is unambiguous on this point, even though human intuition rebels against it. Earlier in this book, I told you that binary was a concession to atoms, not to mathematics, and I asked you to remember that sentence.

Here is where it pays off. Binary won because 1940s switches had two reliable states — on and off.

But qubits do not have two states.

They have a continuous range of states, represented mathematically as points on a sphere called the Bloch sphere. A qubit is not a simpler switch.

It is a richer one — a switch that carries exponentially more information than a classical bit. And a machine built from these switches does not compute faster in the same way.

It computes differently.

What Quantum Computing Actually Is

Quantum mechanics is the most accurately tested physical theory in the history of science, and it describes a reality that contradicts virtually every intuition you have about how the world works. I cannot make it intuitive.

Nobody can. Feynman himself is widely quoted as saying: “If you think you understand quantum mechanics, you don't understand quantum mechanics.” What I can do is describe the three properties that make quantum computing powerful, as clearly and honestly as the subject allows.

The first is *superposition*. A qubit can exist in a combination of 0 and 1 simultaneously.

Two qubits can represent four states simultaneously.

Three qubits, eight. Ten qubits, 1,024. Fifty qubits can represent over one quadrillion states simultaneously. The number of states grows exponentially with the number of qubits. This is not a faster classical computer.

It is a different kind of mathematical object entirely.

The second is *entanglement*.

Two qubits can be placed in a state where they are correlated — measuring one instantly determines the state of the other, regardless of the physical distance between them. Einstein famously called this “spooky action at a distance” and refused to believe it was real.

It is real.

It has been experimentally confirmed thousands of times. Entanglement allows quantum algorithms to explore correlations across an enormous space of possibilities in a coordinated way, rather than checking each possibility one at a time.

The third is *interference*.

When multiple quantum states are superposed, they can reinforce some outcomes and cancel others, the way two waves in a pond can combine to create a bigger wave or cancel each other to create still water. A quantum algorithm is, in essence, a carefully designed sequence of operations that arranges the interference so that wrong answers cancel out and the right answer is amplified. This is a different kind of computation entirely. A classical computer checks possibilities sequentially, or at best in modest parallel. A quantum computer uses superposition to represent all possibilities simultaneously, entanglement to coordinate information across them, and interference to filter for the correct answer.

The result, for certain classes of problems, is a machine that can solve in minutes what a classical computer could not solve in the lifetime of the universe. To be precise: *for certain classes of problems*. A quantum computer is not faster at everything.

It is not a better laptop.

It will not load your email faster or render your video games more smoothly.

It is powerful for specific kinds of problems — problems with a particular mathematical structure that quantum algorithms can exploit. And those specific problems happen to include some of the most consequential computational challenges on Earth.

“A quantum computer does not search through possibilities one by one. It uses the physics of interference to make the correct answer more probable than any incorrect one. For certain problems, it solves in minutes what classical machines could not solve in the lifetime of the universe.”

The Path Here

In 1985, David Deutsch at Oxford formalized Feynman’s intuition, describing the theoretical framework for a universal quantum computer.

In 1994, Peter Shor, a mathematician at Bell Labs, discovered an algorithm that would reshape the strategic landscape of the twenty-first century. Shor’s algorithm is a method for factoring large numbers into their prime components. This sounds abstract.

It is not. The security of nearly every encrypted communication on Earth — your banking transactions, your private messages, military communications, diplomatic cables, corporate secrets — relies on a mathematical assumption: that factoring very large numbers is computationally impractical. A 2048-bit encryption key is secure because factoring it on the best classical supercomputer would take longer than the current age of the universe. This is not security through secrecy.

It is security through mathematics — the assumption that the maths is simply too hard to break. Shor proved that a sufficiently powerful quantum computer could factor these numbers efficiently. Not in the age of the universe. In hours. Possibly minutes. The publication of Shor’s algorithm in 1994 changed quantum computing from a theoretical curiosity into a strategic imperative.

Every intelligence agency on Earth understood what it meant: a working, large-scale quantum computer would render all current public-key encryption obsolete.

Every secret protected by mathematical difficulty would become readable. Not eventually, in some distant future. As soon as the machine was built.

Where We Stand

As of 2026, quantum computers exist.

They are real machines, built by IBM, Google, Microsoft, IonQ, Rigetti, Quantinuum, and others, and available, in limited form, through cloud services.

In 2019, Google claimed “quantum supremacy” — demonstrating that its 53-qubit Sycamore processor could perform a specific calculation in 200 seconds that would take the world’s most powerful classical supercomputer approximately ten thousand

years. IBM disputed the classical estimate. The debate continues.

But the current generation are, in practical terms, prototypes. Qubits are extraordinarily fragile — disturbed by the slightest vibration, temperature fluctuation, or electromagnetic interference.

They lose their quantum state — a process called decoherence — in microseconds or milliseconds.

Every quantum computation is a race against decoherence, and most of the engineering effort in the field is devoted to extending the time a qubit can hold its state long enough to be useful. The spinning coin is powerful because it carries information about both states while it spins.

But the coin wants to land. The physical world conspires to make it land — air resistance, vibration, the table it's spinning on. The engineer's job is to keep the coin spinning long enough to extract the answer before it lands. Current quantum computers can keep the coin spinning for a very short time. Not long enough to break encryption. Not long enough to simulate complex molecules with commercial accuracy.

But long enough to prove the principle works. The engineering path forward involves quantum error correction — using many physical qubits to create a single, more reliable “logical” qubit, the way you might use many noisy voices singing the same note to produce a clearer tone than any individual voice. Current estimates suggest that breaking RSA-2048 encryption would require roughly four thousand logical qubits, which in turn might require millions of physical qubits, depending on the error rate. The largest quantum computers today have roughly one to two thousand physical qubits.

The gap is significant but not, most experts believe, permanent.

We are, in the terms of this book, roughly where classical computing was in the ENIAC era. The principle works. The engineering is brutal.

The machines fill rooms, require extreme environmental control, and break constantly.

But the trajectory is clear, the investment is enormous, and the people working on the problem are among the most talented physicists and engineers alive.

What It Will Do

When quantum computers reach sufficient scale and reliability — a milestone most researchers estimate is ten to twenty years away for the most demanding applications

— they will be capable of things that no classical computer, regardless of its speed, can achieve. Molecular simulation. The behavior of molecules is governed by quantum mechanics. Simulating a molecule’s behavior on a classical computer requires approximations that become exponentially more costly as the molecule grows larger. A quantum computer could simulate molecular interactions with native fidelity — because the computer and the molecule obey the same physics.

This means designing drugs by modeling how a candidate molecule interacts with a protein target, computationally, with a precision that no classical simulation can match.

It means designing new materials — room-temperature superconductors, better battery chemistries, more efficient solar cells — by simulating their atomic structure rather than testing thousands of variations in a laboratory. Optimization. Many of the hardest problems in logistics, finance, and engineering are optimization problems — finding the best solution among a vast number of possibilities. Route planning for delivery networks. Portfolio allocation across thousands of securities. Scheduling for airlines, hospitals, and military operations. These problems grow exponentially harder as the number of variables increases, and classical computers solve them with approximations and heuristics.

Quantum algorithms can, for certain problem structures, find optimal or near-optimal solutions exponentially faster. Cryptography. Shor’s algorithm breaks all current public-key encryption.

But quantum mechanics also enables a new kind of encryption — quantum key distribution — that is provably secure based on the laws of physics rather than the difficulty of a mathematical problem. An eavesdropper who intercepts a quantum-encrypted message physically disturbs it in a way that is detectable. You cannot copy a quantum state without altering it. This means that quantum computing does not just break the old lock.

It provides a fundamentally new one.

But the new lock is available only to those who have the quantum infrastructure to use it. Artificial intelligence. Quantum algorithms may accelerate specific machine learning operations — optimization of neural network weights, sampling from complex probability distributions, training on data with quantum correlations. The implications are still theoretical, but if quantum computing provides even a modest speedup in AI training, it would amplify the concentration dynamics already at work: the organizations with quantum hardware would train better models, faster, widening the gap between the frontier and everyone else.

The Invisible Arms Race

In 1944, Colossus was the most powerful computer in the world, and the British government classified its existence for thirty years.

The machine's value lay not just in what it could compute but in the fact that the enemy did not know it could compute. The asymmetry of knowledge — we can read your messages; you do not know we can read your messages — was more valuable than any single piece of intelligence the machine produced.

Now project this forward. A nation or entity that achieves a breakthrough in quantum computing — specifically, a machine capable of running Shor's algorithm against current encryption standards — would possess, for the period before the rest of the world catches up, the ability to read every encrypted communication on Earth.

Every diplomatic cable.

Every military order.

Every financial transaction.

Every corporate negotiation.

Every personal message that anyone believed was private. Would they announce this capability? The history of computing, traced through every chapter of this book, gives us the answer: they would not. Colossus was classified for thirty years. The NSA's capabilities have been consistently underestimated by the public until forced disclosures revealed their scope.

Every entity that has achieved a significant computational advantage has treated that advantage as a secret to be protected, not a capability to be disclosed. This creates a problem that is fundamentally different from the nuclear arms race, and in some respects more dangerous. Nuclear weapons are visible. You can see a uranium enrichment facility from a satellite. You can detect a nuclear test from the other side of the planet. The infrastructure required is large, distinctive, and observable. This visibility is what makes deterrence possible and arms control treaties enforceable.

Quantum computing is invisible. You cannot see a qubit from a satellite.

A quantum computation produces no detectable emission — no seismic signature, no radiation, no electromagnetic pulse.

The infrastructure — a cryogenic cooling system, a shielded chamber, control electronics — fits inside a single building and looks, from the outside, like any other research laboratory.

There is no “quantum test” that alerts the world to a breakthrough.

A nation could achieve the ability to break global encryption and the rest of the world might not know for years — until the consequences start appearing in negotiations that always seem to go one way, in financial markets that move in patterns that benefit the same actors, in military deployments that consistently anticipate the adversary's plans.

Harvest Now, Decrypt Later

There is a strategy, widely discussed in the intelligence community and known informally as “harvest now, decrypt later,” that makes the timeline even more urgent than the engineering milestones suggest. The strategy is simple.

Every encrypted communication that passes through the internet can be intercepted and stored — not read, because it is encrypted, but stored. Intelligence agencies have the capacity to capture and archive vast quantities of encrypted traffic.

They cannot read it today.

But if a quantum computer capable of breaking that encryption becomes available in ten or fifteen years, they will be able to decrypt the archive retroactively. Which means that every encrypted communication happening right now — every diplomatic negotiation, every corporate merger discussion, every military planning session, every private message between heads of state — has an expiration date on its secrecy. Not the date the message was sent. The date someone builds the machine that can read it. The United States National Institute of Standards and Technology has been leading a multi-year effort to develop and standardize post-quantum cryptography — encryption algorithms believed to be resistant to quantum attack.

The urgency of this effort reflects an assumption that the “harvest now, decrypt later” threat is real and that the transition to quantum-resistant encryption must happen before the quantum computers arrive, not after. The transition is underway.

It is also slow, expensive, and incomplete.

Every day that passes between now and its completion is a day when communications are being intercepted and stored by entities betting they will be able to read them eventually.

The Binary Question, Answered

A question has been running through this book since Leibniz proposed his two-digit system and Shannon showed how binary maps to switches.

Here is the payoff. Binary is not a permanent feature of computation.

It was an engineering choice, made because switches have two reliable states, and it served us extraordinarily well for eighty years.

But a qubit is not a switch.

It does not have two states.

It exists on a continuous sphere of possibilities, defined by two parameters — an angle and a phase — that together describe a point on the Bloch sphere.

A qubit carries more information than a bit, not because it is faster, but because it occupies a richer mathematical space.

And as the next chapter will show, DNA stores information in four bases — A, T, G, C — effectively base-4 rather than base-2.

The biological substrate of computation does not speak binary either.

The eighty-year reign of the binary switch may turn out to be a historical parenthesis — a solution perfectly adapted to one particular substrate, as dominant in its era as the slide rule was in its own, and potentially just as transient.

The next substrates compute in the language that their physics demands, and that language is not ones and zeros.

It is richer, stranger, and more powerful. The implication is striking: it means that the most fundamental assumption of modern computing — that everything reduces to bits — may not survive the century. The foundation that Shannon built, that Turing formalized, that the entire digital economy rests on, is an artefact of a particular era's physics.

When the physics changes, the foundation changes with it. And that means everything built on that foundation — every operating system, every programming language, every algorithm, every architecture — will eventually have to be rethought for a post-binary world.

The Power Question, Sharpened

Building a useful quantum computer requires infrastructure that is beyond the reach of individuals, most companies, and most nations.

The machines operate at temperatures near absolute zero — colder than outer space.

They require electromagnetic shielding that blocks every stray signal.

They require control electronics of extraordinary precision.

They require a supply chain of specialized components that no single company produces.

They require physicists, engineers, and mathematicians whose training takes a decade and whose numbers could fit in a lecture hall. The cost of a single useful quantum computing installation is measured in billions of dollars. The number of entities on Earth that can spend billions on a computation experiment is vanishingly small. This is not a garage technology.

It is not a hobbyist's project.

It is a sovereign capability — the computational equivalent of a nuclear program, accessible only to the wealthiest nations and the largest corporations. And unlike the personal computer, which distributed because it was cheap, or the internet, which distributed because the protocol was free, quantum computing has no obvious path to mass distribution. The physics itself resists it. Qubits must be isolated from the environment. The more you isolate them, the more expensive and complex the apparatus becomes. There is no Moore's Law for qubits — no observed trend of exponential improvement that suggests quantum computers will become cheap, small, and ubiquitous.

They may become available through cloud services, as some already are.

But cloud access is not ownership. Cloud access is access granted at the discretion of the owner. This means that the distribute-then-recapture pattern traced through the preceding chapters may not apply. There may be no distribution phase. The quantum computing era may begin concentrated and stay concentrated. The jar may never leave the hands of the few who can afford to build it.

“The personal computer distributed because it was cheap. The internet distributed because the protocol was free. Quantum computing has no obvious path to mass distribution. The physics itself resists it.”

What Comes Next

If quantum computing represents the extension of computation into the subatomic world — exploiting the physics of particles too small to see — then the next chapter represents something even stranger: the extension of computation into the living world. Biological computing — DNA storage, molecular logic gates, engineered cells that process information — does not just use a different material.

It uses a material that can grow, repair itself, copy itself, and evolve. A silicon chip must be manufactured by a billion-dollar fabrication plant. A DNA molecule can be replicated by the same cellular machinery that copies your genome every time a cell divides.

If quantum computing worries me because of its concentration, biological computing worries me for the opposite reason: its potential to propagate. A quantum computer is expensive and confined. A biological computing agent could, in principle, spread. The governance implications are unlike anything we have encountered in this book — because the governance frameworks humanity has built for computation are designed for machines. And a biological computer is not, in any traditional sense, a machine. The spiral of computing's evolution started with nature — Jacquard mimicking the weaver's hand, Babbage modeling human calculation, McCulloch and Pitts abstracting the neuron.

Then it diverged into pure engineering: electronics, silicon, integrated circuits.

Now it is curving back. Quantum computing exploits the physics that governs atoms. Biological computing exploits the chemistry that governs life.

The machine is returning to the substrate it originally tried to transcend. Whether this is a homecoming or an invasion depends entirely on who is steering.

CHAPTER WHJ-10

The Molecule

On biological computing, DNA storage, and the substrate that is alive

The Molecule

On one gram of DNA that could hold every piece of data on Earth, and the moment the machine stops being separate from the living world.

In 2012, a Harvard geneticist named George Church held up a small vial in his laboratory. The vial was smaller than a fingertip.

It contained synthetic DNA. And encoded in the sequence of nucleotide bases within that DNA was his entire book — 53,000 words, plus images — stored not in silicon, not on paper, not on magnetic tape, but in the molecules of life itself.

One gram. That is how much synthetic DNA you would need to store approximately 215 petabytes of data. A petabyte is a million gigabytes. 215 petabytes is roughly the estimated total data storage capacity of every major technology company on Earth combined — every email, every photograph, every video, every financial record, every medical file, every line of code, every piece of information that the digital economy has produced and preserved. All of it. In one gram. A quantity so small it would barely cover a fingertip.

That number sounds like science fiction.

It is chemistry. DNA is the most information-dense storage medium known to exist — not because it was engineered but because it was evolved, over 3.8 billion years, by a process that relentlessly optimized for exactly the properties that data storage requires: density, durability, copyability, and fidelity. In the preceding chapters, the computing substrate was something humans built from scratch: gears from metal, vacuum tubes from glass, transistors from purified silicon.

This chapter is about a substrate that existed before humans did. A substrate that nature spent billions of years perfecting. And a substrate that raises a question that no

previous computing chapter has had to confront — not just what the machine can do, but what happens when the machine and the person holding it become, at the molecular level, the same thing.

The Four-Letter Library

DNA uses a different encoding from binary. Instead of two symbols, it uses four: the nucleotide bases adenine (A), thymine (T), guanine (G), and cytosine (C). These four molecular letters are strung together in sequences that can be billions of characters long. Your genome — the complete set of instructions for building and operating your body — is approximately 3.2 billion base pairs long and is contained in nearly every cell you possess.

If binary is an alphabet with two letters, DNA is an alphabet with four. Each position in a DNA sequence carries twice as much information as a single binary digit.

But the density advantage goes far beyond the alphabet. A single transistor on a modern chip is about three nanometres. A single base pair in a DNA molecule occupies about 0.34 nanometres. DNA stores information at a scale roughly ten times smaller than the best silicon technology, and it does so in three dimensions rather than two — the double helix is a coiled, twisted, folded structure that packs an extraordinary amount of information into an extraordinarily small space. And DNA is durable. Under the right conditions — cool, dark, dry — DNA can survive for tens of thousands of years.

We have extracted and sequenced readable DNA from woolly mammoths frozen in Siberian permafrost for over forty thousand years. No manufactured storage medium comes close. A hard drive lasts perhaps five to ten years. Magnetic tape, perhaps thirty years. DNA, properly preserved, lasts millennia.

*“DNA is the most information-dense storage medium known to exist.

One gram can hold 215 petabytes.

It survives for tens of thousands of years. No manufactured medium comes close on either measure.”*

Writing Data to Life

The process of storing digital data in DNA works like this. Take a digital file. Convert the binary sequence into DNA bases using a mapping such as: 00 = A, 01 = T, 10 = G,

11 = C. Synthesize an actual DNA molecule with that sequence. You now have a physical molecule that encodes your digital file. To read the data back, sequence the molecule and convert back to binary. Church's 2012 experiment was the proof of concept.

In 2017, researchers at Columbia University and the New York Genome Center stored a full operating system, a movie, a computer virus, a Pioneer plaque image, and a \$50 Amazon gift card in DNA and retrieved them with perfect accuracy.

In 2019, researchers at the University of Washington and Microsoft demonstrated a fully automated system that could write data to DNA and read it back without human intervention. The writing process took approximately 21 hours for a small data set. Those timescales reveal both the promise and the current limitation. Twenty-one hours is glacially slow compared to silicon.

Nobody is going to stream a movie from a DNA molecule. The speed gap is a fundamental difference in physics: chemical bonds form in milliseconds; electronic switches operate in billionths of a second.

But for archival storage — data written once and read rarely, stored for decades or centuries — DNA's advantages are overwhelming. A warehouse of magnetic tape that costs millions to maintain and must be migrated every decade could be replaced by a vial that fits in a desk drawer, requires no power, and will be readable in a thousand years. The economics are converging. The cost of synthesising DNA has fallen by roughly a factor of ten thousand since the early 2000s.

The Computer That Was Always There

DNA storage is remarkable, but it is ultimately a use of biology as a passive medium — a very good hard drive. The far more radical idea is using biology as an active computer — a substrate that does not just store information but processes it. This idea is less exotic than it sounds, because living cells have been doing exactly this for 3.8 billion years.

Consider what a single cell does.

It stores information — the genome, 3.2 billion base pairs.

It retrieves information — specific genes read at specific times in response to environmental signals.

It compares information — the immune system distinguishes self from foreign, a feat of pattern recognition operating continuously across trillions of cells. And it transforms information — the genetic code translated into proteins that fold into

three-dimensional structures carrying out the physical work of the organism. Store. Retrieve. Compare. Transform. The four operations that opened this book — the operations that define what a computer is, from a shepherd's jar of pebbles to a quantum processor. A cell performs all four.

It has been performing all four since before multicellular life existed. The cell is, by the functional definition established at the beginning of this book, a computer.

It is the oldest computer on Earth. And by several important measures, it is the best. The human brain contains approximately 86 billion neurons connected by roughly 100 trillion synapses.

It performs the computational equivalent of billions of operations per second while consuming approximately 20 watts of power — roughly enough to run a dim light bulb. A modern GPU performing comparable operations consumes thousands of watts. The brain is, by a factor of hundreds or thousands, the most energy-efficient computer known.

It is also self-repairing.

When a transistor fails, it stays failed.

When a neuron is damaged, the brain can reroute its connections.

It is self-assembling — a brain assembles itself from a single fertilized cell in approximately nine months. And it is self-replicating — the instructions for building a brain are copied every time an organism reproduces.

The question that a growing number of researchers are now asking is not whether biological systems can compute.

They obviously can.

The question is whether we can learn to program them.

Programming Life

In 1994, Leonard Adleman at the University of Southern California demonstrated the principle of DNA computation — encoding a graph problem into DNA sequences, mixing them in a test tube, and using biochemistry to find a solution. Billions of molecules, each representing a different potential path, reacting in parallel.

It was to modern computing what the Antikythera mechanism was to the smartphone — a demonstration separated from utility by an enormous engineering gap.

But the field has advanced considerably. Researchers have built logic gates from DNA — molecular structures performing AND, OR, and NOT operations on chemical

signals, just as Shannon's relay circuits performed the same operations on electrical signals.

In 2024, a team from Caltech, UC Davis, and Maynooth University published a landmark result: a reprogrammable DNA computer capable of running 21 distinct algorithms using the same physical molecular system. Living cells have been engineered to function as biosensors and to perform logical decisions: if chemical A is present AND chemical B is absent, THEN produce protein C. These engineered cells are being explored as smart drug delivery systems — therapeutic agents that circulate in the body and activate only when they detect the chemical signature of a disease.

As this book was being completed, the field crossed another threshold. In February 2026, researchers published a framework for allosteric DNA computing in *Science Advances* — a system that integrates sequence programmability with conformational dynamics, successfully bridging conformational signaling with genetic control regulations inside living biological architectures. This is not a theoretical proposal.

It is complex, multi-level signal processing executed in vivo. The biological computer is no longer a laboratory curiosity.

It is operating inside living systems, processing information through the same molecular machinery that the cell uses to regulate itself. Neuromorphic chips — hardware designed to mimic the structure of biological neural networks, not just their mathematics — represent yet another convergence. Intel's Loihi chip and IBM's TrueNorth are circuits whose physical architecture mirrors biological neurons.

They are dramatically more energy-efficient for certain tasks, precisely because they exploit the same principles biology discovered billions of years ago.

The Spiral

The pattern this chapter reveals is the deepest pattern in this entire book. The history of computation is not a line from simple to complex.

It is a spiral. The spiral began in nature.

The first computers were biological — living cells processing genetic information, organisms navigating their environments, brains modeling the world through neural networks of staggering complexity. Nature was computing for billions of years before humans arrived. Humans, when they built their own machines, began by mimicking biology. Jacquard's loom mimicked the weaver's hand. Babbage modeled the human calculator. McCulloch and Pitts abstracted the biological neuron.

Then the spiral diverged. Electronics, silicon, integrated circuits — purely engineered substrates with no biological counterpart. For eighty years, the machine evolved away from its origins. And now the spiral is curving back. Quantum computing exploits the physics that governs atoms in biological molecules. DNA storage uses the actual biological medium. Neuromorphic chips mimic the architecture of neurons.

The machine, after eighty years of divergence, is converging back toward the substrate it originally drew inspiration from. This convergence is not merely technical.

It is existential — and the evidence has been hiding in the oldest stories humanity tells. The oldest written narrative in human history is the Epic of Gilgamesh, composed in Mesopotamia around 2100 BCE — in the same river valleys where a shepherd first placed pebbles in a clay jar to count his flock. In that story, a king watches his closest friend die and is so shattered by grief that he undertakes a journey to find eternal life.

He crosses the waters of death.

He finds a plant at the bottom of the sea that restores youth. And while he sleeps on the journey home, a serpent steals it.

He returns to his city mortal. Nature reclaims the answer. The oldest computing story and the oldest immortality story come from the same civilization. I do not think this is a coincidence. Both are stories about human beings reaching for a power that exceeds their natural capacity — to count beyond memory, to live beyond the body. For five thousand years, the two ambitions traveled separate paths. The computing ambition produced gears, engines, vacuum tubes, transistors, silicon, qubits. The immortality ambition produced mythology, alchemy, and eventually medicine.

This chapter is where the paths converge. The biological substrate that can store data, compute logic, and deliver therapies inside the human body is the same substrate that is now being used to edit the mechanisms of ageing itself. Stem cell therapies that regenerate damaged tissue. Gene editing tools — CRISPR and its successors — that correct the mutations driving age-related disease. DNA repair mechanisms borrowed from organisms that exhibit negligible senescence. Telomere extension research. Senolytics — drugs that selectively destroy the senescent cells whose accumulation drives ageing.

These are not speculative.

They are funded, published, and in clinical trials. The spiral that began with a shepherd counting pebbles is arriving at a place where the machine and the body are no longer separate substrates. The same tools that program a cell to deliver a drug can

program a cell to repair a chromosome. The same biological computing infrastructure that this chapter has traced from Adleman's test tube to Caltech's reprogrammable DNA computer is, in parallel laboratories, being used to extend the human lifespan. The two oldest human ambitions — to count and to live — are converging in the same molecule.

Who Gets the Nectar

In Hindu mythology, the Samudra Manthan tells the story of the churning of the cosmic ocean. The gods and the demons, locked in a struggle neither could win alone, agreed to cooperate — to churn the primordial sea together, using a mountain as a churning rod and a serpent as a rope, in order to produce Amrita: the nectar of immortality. The ocean yielded poisons and wonders as it churned.

But the prize they were churning for was the nectar. The substance that would defeat death.

When the Amrita finally appeared, the demons lunged for it.

They had done half the churning.

They believed they had earned their share.

But Vishnu, taking the form of Mohini — an enchantress so beautiful that the demons forgot their purpose — distributed the nectar exclusively to the gods. The cooperation that produced the prize was real. The distribution of the prize was a deception. Immortality was given to those who already held power and withheld from those who had helped create it. This story was told thousands of years ago in the Indian subcontinent.

It is one of the oldest narrative frameworks in human civilization. And it is the precise structural template for the governance problem that the biological substrate introduces.

Every governance framework humanity has ever built rests on an assumption so fundamental that it is never stated: power is temporary because the holder is mortal. The pharaoh dies. The emperor dies. The CEO retires or is buried. The dictator's grip weakens as the dictator's body weakens. Even the most entrenched dynasty eventually produces an heir who is less capable, less ruthless, or less interested than the founder. Mortality is nature's term limit.

It is the one form of succession that no amount of power can override.

It is, in the deepest sense, the mechanism by which civilization renews itself — not through choice, but through biology. The old order dies. Literally. And something new

grows in the space it leaves behind.

Now imagine that the holder of the jar — the person or institution that controls quantum computation, artificial intelligence, and the biological substrate described in this chapter — uses that control to extend their own lifespan by decades. By a century. By enough to outlive every constraint that mortality would have imposed.

They do not just hold the jar longer.

They hold it long enough to reshape every institution, every market, every legal framework around their continued holding of it.

They outlive their regulators.

They outlive the journalists investigating them.

They outlive the social movements resisting them.

They outlive the prosecutors building cases against them. The democratic assumption that power rotates because people die ceases to apply to the people with the most power. And consider the mathematics of the planet they inhabit. Earth's carrying capacity — its food systems, its fresh water, its arable land — evolved for populations that die.

If biological interventions collapse the death rate among those who can afford them, while birth rates remain positive — even as they decline globally, they will vastly outpace deaths in a population that is no longer dying on schedule — the arithmetic becomes unsustainable. The planet becomes, in the language of population biology, a fruit fly experiment: a closed system in which reproductive output exceeds the carrying capacity because the balancing mechanism of death has been selectively disabled. *Selectively*. That is the word that matters.

The therapies will not be free.

They will not be universally distributed.

They will cost what only the holders of the jar can afford. And so the question of whether to share the nectar — the question the Samudra Manthan posed thousands of years ago — acquires a dimension that even the myth did not anticipate. Sharing the therapies universally may not be survivable at planetary scale. Withholding them is the most consequential act of gatekeeping in human history — a jar that literally contains life and death, held by people who have decided, by the act of holding it, that they deserve to live on a different timescale than the rest of the species.

But a biological computing agent introduces governance challenges beyond life extension that are categorically different from any previous computing substrate. A silicon chip does not copy itself. A DNA molecule does. A silicon chip does not grow. An engineered cell can. A silicon chip does not mutate. A biological computing agent,

placed in a biological environment, is subject to the same evolutionary pressures that govern all life — mutation, selection, drift.

It can change in ways its designers did not intend and cannot fully predict. The governance frameworks that humanity has built for computing are designed for machines. Machines have serial numbers.

They can be tracked, counted, embargoed, and destroyed.

They stay where they are put.

They do not evolve.

They do not breed. A biological computing agent, once released, is subject to none of these constraints.

It is governed not by export controls but by ecology. Not by patents but by evolution. Not by firewalls but by cell membranes. The potential benefits are enormous. A drug delivery system that computes inside the body could transform medicine. DNA storage could preserve human knowledge for millennia. Engineered biological systems could perform environmental remediation.

But the dual-use problem is severe: the same techniques that design a therapeutic agent can design a pathogen. The barrier between biological computing and biological weapons is not a technical barrier.

It is an intentional one.

Three Dangers

The previous chapter described the danger of quantum computing: concentration. A technology so expensive that it may never distribute. The jar that never leaves the hands of the few.

This chapter describes two more.

The second danger is propagation. A technology whose defining property is that it copies itself. The jar that cannot be contained.

The third danger is permanence. A technology that allows the holder of the jar to defeat the one constraint that has, for five thousand years, guaranteed that the jar eventually passes to someone else. The holder who does not die. The person who holds the computational jar and the biological jar simultaneously does not merely hold power.

They hold power without expiration.

They become, in a functional sense that would have been recognizable to every civilization that told stories about the nectar and the serpent and the churning of the

ocean, the closest approximation of what humanity has historically called a god. Not because they are omniscient. Not because they are omnipotent.

But because they have removed the one constraint that separated every previous holder of power from divinity: the certainty that they would die and the jar would pass to someone else. For five thousand years, the serpent always won. Gilgamesh returned to Uruk mortal. The demons were tricked out of the Amrita.

Every mythology that imagined the defeat of death also imagined its reclamation — because every storyteller lived in a world where death was, in the end, non-negotiable. The stories were warnings dressed as fantasies: you may reach for it, but you will not keep it. The biological substrate described in this chapter is the first technology in human history that may allow the holder of the jar to keep it. And every governance framework we possess — every constitution, every term limit, every antitrust statute, every democratic norm of succession — was designed for a world in which the serpent always wins.

“For five thousand years, the serpent always won. The biological substrate described in this chapter is the first technology that may let the holder of the jar keep it. And every governance framework we possess was designed for a world in which the serpent always wins.”

What Comes Next

The full arc of computing’s physical evolution has been traced: from pebbles to gears to vacuum tubes to transistors to silicon chips to qubits to DNA. Ten chapters.

Five thousand years. Four operations.

One trajectory that began in the human need to count sheep and now arrives at the ability to program molecules, extend lives, and — if the trajectory is not governed — create holders of the jar who never have to let go. The remaining chapters of this book are not about the machines.

They are about us. The next chapter asks the question that this entire book has been building toward: what happens when a machine becomes powerful enough to model every possible outcome of a decision — and someone owns it?

It is not a question about technology.

It is a question about human nature. And it is, I believe, the most important question that the evolution of computation has ever posed.

Because the danger is not that the machine wakes up. The danger is that the human holding it never has to sleep.

PART VI — POWER & WARNING

*Chapter 11 — The Oracle Problem · Chapter 12 — The Distributed Question ·
Chapter 13 — The Spiral · Chapter 14 — The Choice*

* * *

CHAPTER WHJ-11

The Oracle Problem

On the convergence of substrates and the question of who controls computation when computation controls everything

The Oracle Problem

On what happens when a machine can see every possible future — and someone owns it.

This chapter begins with you. Not the abstract “you” of a thought experiment — the actual you, wherever you are reading this, in the specific country you woke up in this morning, governed by the specific laws that apply to you, dependent on the specific infrastructure that delivers your electricity, your water, your internet, your healthcare, your money. The Oracle does not arrive in the abstract.

It arrives in your city, through your systems, under your government’s jurisdiction — or outside it. Where you live determines not whether the Oracle affects you but how. I have lived in three countries. Each gave me a different view of the same capability.

What follows is not a thought experiment.

It is three mornings — three ways a single technology arrives depending on where you stand when it does.

“The Oracle does not arrive in the abstract. It arrives in your city, through your systems, under your government’s jurisdiction — or outside it. Where you live determines not whether the Oracle affects you but how.”

The Oracle in New York

You wake up in Manhattan. You work in technology. The Oracle — the convergence of quantum computing, biological computation, and artificial intelligence that the

previous two chapters described — is not an abstraction to you.

It is a product roadmap.

It is a slide deck.

It is a capability your company might build, a tool your employer might license, a competitive advantage you are paid to evaluate. The Oracle arrives in New York as an opportunity. You can see the demos. You can attend the conferences. You can read the papers the week they are published — sometimes before, because you know the researchers. You can invest in the companies building the capability, or you can join them. You have agency. You have information. You have options.

The question that occupies you is competitive: will your company build it before the other one does? Will your country maintain its lead? Will the next breakthrough happen in your lab or in Shenzhen? The regulatory environment confirms your position. The AI companies that build the Oracle are headquartered in your city, your state, your country.

They employ your neighbors.

They lobby your representatives. The governance frameworks being discussed — the executive orders, the proposed legislation, the voluntary commitments — are written in your language, debated in your media, and shaped by your political process. You may disagree with the policy.

But the policy is about you, written for your context, and you have a voice in it. Your concern is legitimate and real: the pace of development may outrun the pace of governance. The companies may accumulate power faster than the institutions can constrain it. The Oracle may arrive before the rules do.

But your concern operates within a system that acknowledges your existence. You are in the room. You may not control the outcome, but you are visible to the people who do. This is the Oracle at the altitude of the builder. You can see the machine being assembled. You can touch the parts. You may even hold a wrench.

The Oracle in Toronto

You wake up in Toronto. You have universal healthcare. Your banking system is stable. Your democracy functions. Your country is prosperous, educated, and safe. By almost any measure, you live in one of the best-governed nations on Earth. The Oracle does not arrive in Toronto as a product you build.

It arrives inside the infrastructure you already depend on. Your hospital's diagnostic AI runs on an American cloud. Your bank's fraud detection uses an

American model. Your military's communications depend on an American satellite constellation — one whose owner has already determined the trajectory of a major European war through unilateral connectivity decisions. The Oracle does not knock on your door.

It is already in your house, woven into the systems you trust. Your government can negotiate. Canada has PIPEDA — federal privacy legislation that imposes requirements on how data is collected, stored, and used.

It has data residency provisions that require certain categories of information to remain on Canadian soil.

It has a Prime Minister who stood before the World Economic Forum and named the rupture in the rules-based order — who said, publicly, that middle powers must act together because if you're not at the table, you're on the menu.

But consider what negotiation means in practice. Your hospital's diagnostic AI was trained on data from American and European populations.

It uses standard diagnostic categories — categories developed for populations whose genetic backgrounds, dietary patterns, environmental exposures, and disease trajectories may differ fundamentally from the populations it now serves. In Scarborough — the most ethnically diverse region of Toronto, and the site of Canada's largest regional dialysis program — a nephrologist named Dr. Tabo Sikaneta conducted a study of 2,397 patients with chronic kidney disease. His findings, published in *BMJ Open*, documented something that the diagnostic categories had been hiding: dialysis prevalence among immigrants was 4.2 times higher than in the Canadian-born population.

Not marginally higher. Not within the range of statistical noise. Four point two times. And the disparities ran along lines that standard ethnic categories could not capture — birth country predicted outcomes more powerfully than ethnicity did. Patients born in the Caribbean, Southeast Asia, and South Asia had the highest rates. Kidney function declined fastest in Caribbean-born and South Asian patients. Diabetes prevalence, blood pressure, and glycaemic control varied dramatically by country of origin. These patients had universal healthcare.

They had equal access to the same hospitals, the same doctors, the same diagnostic tools. And they were dying at four times the rate. The reason is not that the system was withholding care. The reason is that the system was measuring the wrong things. The diagnostic categories — the standardized screens, the risk calculators, the clinical decision support tools that increasingly run on AI — were designed for a population that does not look like Scarborough.

When you build an algorithm on categories that ignore birth-country epigenetics, dietary patterns shaped by migration, and the physiological consequences of systemic displacement, the algorithm will mathematically enforce the blind spot.

It will not see what the categories do not measure. And it will declare the unmeasured population healthy, right up until they arrive at dialysis. Not a failure of technology.

It is a failure of measurement that technology amplifies. The algorithm did not create the blind spot. The diagnostic framework did. The algorithm inherited it, scaled it, and made it operate at the speed and confidence of a machine — which is to say, without doubt, without hesitation, and without the clinical intuition that a human physician might bring to a patient whose presentation does not fit the textbook. And this is happening in Toronto — one of the best-funded, most diverse, most data-rich healthcare systems on Earth.

If the algorithm fails here, in a universal system with robust data infrastructure and PIPEDA protections, what happens when the same categories are exported to Lusaka, where the diagnostic framework was designed for a population an ocean away?

But the negotiation is asymmetric. Canada does not build frontier AI models.

It does not manufacture the chips they run on.

It does not operate the satellite constellation its military depends on.

It can set conditions on the Oracle's use within its borders, but it cannot set conditions on the Oracle's development — because the development happens elsewhere, in labs funded by capital your country cannot match, using compute infrastructure your country does not own. The terms of the negotiation are set by the builder. The middle power accepts what is offered, negotiates at the margin, and calls the result sovereignty. Your concern is not whether the Oracle will affect you.

It already does. Your concern is whether you can govern what you cannot build. Whether privacy legislation written for the previous era of technology will hold against a capability designed in a jurisdiction that does not share your values. Whether the data about your citizens — their health records, their financial transactions, their communications — will be processed by systems whose operators answer to a different government's laws, or to no government's laws at all. This is the Oracle at the altitude of the negotiator.

You can see the machine. You cannot touch the parts. You are at the table, but you did not design the table and you cannot leave it.

The Oracle in Lusaka

You wake up in Lusaka, Zambia. You are twenty-six years old. You have a university degree — the first in your family. You work for a small agricultural cooperative that helps farmers in the Central Province access markets for their crops. Your phone is your office.

It is a \$90 Android device that runs on a mobile network operated by a South African telecommunications company, which contracts its cloud infrastructure from an American hyperscaler. Your money moves through a mobile banking platform whose fraud detection, credit scoring, and transaction monitoring are powered by machine learning models trained on data from three continents — none of which is yours. The Oracle does not arrive in Lusaka as a product or a negotiation.

It arrives as a condition.

It is already inside the mobile money system that handles 70% of the transactions in your economy.

It is inside the health screening app the NGO distributed to the rural clinics.

It is inside the agricultural pricing tool that tells your farmers what their maize is worth today — a price determined by an algorithm running on a server in Virginia, trained on commodity data from Chicago, calibrated to a global market whose dynamics your farmers have no visibility into and no influence over.

Nobody asked. That is the sentence that defines the Oracle's arrival in Lusaka.

Nobody asked whether you wanted predictive analytics applied to your creditworthiness.

Nobody asked whether the health screening algorithm was validated on a population that looks like yours. In Toronto — with universal healthcare, world-class hospitals, and the most diverse patient population in Canada — Dr. Sikaneta found that immigrants were dying of kidney disease at four times the rate, because the diagnostic categories did not capture what was killing them.

If the algorithm fails in Toronto, what does it do in Lusaka, where there is no Dr. Sikaneta, no BMJ publication, no data infrastructure to even detect the failure?

Nobody asked whether the agricultural pricing model accounts for the specific conditions of Zambian smallholder farming — the soil, the rainfall patterns, the transport costs, the market access that differs from village to village.

Nobody told you the data was leaving the country.

Nobody told you which country it was going to.

Nobody told you what decisions were being made about you, by whom, using what criteria, subject to what laws. You cannot negotiate with the Oracle because you cannot see the Oracle. You interact with an app on your phone. The app connects to a

platform. The platform connects to a cloud. The cloud connects to a model. The model was trained on data you did not contribute to, validated against benchmarks that do not include your population, and governed by policies written in a language you do not speak, in a jurisdiction that does not know you exist.

Your government has no AI legislation.

It has no data protection authority with the resources to audit the systems operating within its borders.

It has no seat at any table where the rules for these systems are discussed.

It has no mechanism of oversight, no institution with the resources or authority to audit what is being done. Your concern is not sovereignty. You cannot lose what you never had. Your concern is visibility — whether anyone in the system even knows you are affected. Whether the model that scored your creditworthiness was designed to account for your existence. Whether the algorithm that screens your health was trained on data that includes people who look like you, live where you live, eat what you eat, carry the genetic and environmental risk factors that your population carries.

The answer, in most cases, is no. You are in the margin. And the margin is where the cost is paid. This is the Oracle at the altitude of the invisible. You cannot see the machine. You cannot touch the parts. You are not at the table. You do not know there is a table.

The Pattern at Three Altitudes

The same capability.

Three experiences. The Oracle is not a hypothetical machine in a philosopher's thought experiment.

It is the convergence of quantum computing, biological computation, and artificial intelligence, operating on the global data infrastructure that six billion smartphones generate, at a scale that only the largest organizations on Earth can build and operate.

It is not coming.

It is arriving — unevenly, at different speeds, through different channels, experienced at different altitudes depending on where you stand. To make this concrete, because the abstract version is too easy to dismiss.

Imagine an AI system — a large language model trained on the entirety of published biomedical literature, every protein database, every clinical trial, every patent filing in pharmaceutical chemistry. This system can generate novel molecular structures the way ChatGPT generates sentences: by predicting what comes next,

informed by everything that has come before.

It proposes a candidate molecule for a drug that targets a specific protein involved in a specific disease.

Now connect that AI to a quantum computer. The quantum machine simulates the candidate molecule's interaction with the target protein — not approximately, the way a classical computer would, using statistical shortcuts that miss critical atomic-level behaviors, but with native quantum fidelity, because the computer and the molecule obey the same physics. The simulation takes minutes instead of months.

It reveals that the candidate binds to the target with remarkable precision. The AI adjusts, proposes a variant, the quantum computer simulates again. In hours, the system has iterated through thousands of candidates that would have taken a traditional pharmaceutical lab years to evaluate.

Now connect the output to a biological computing system — a DNA synthesizer that can write the gene sequence for the winning molecule, insert it into an engineered cell, and produce the compound. The designed-by-AI, validated-by-quantum, manufactured-by-biology drug exists as a physical substance within days of the initial query. This is the Oracle as a technical architecture, not a metaphor.

Three substrates, each doing what it does best: AI generates possibilities, quantum validates them against physical reality, biology manufactures the result. The loop is closed. The time from question to molecular reality collapses from years to days.

Now change one variable. Instead of targeting a disease protein, target a protein essential to human immune function. Instead of designing a therapeutic, design a pathogen. The same architecture. The same three substrates. The same closed loop. The same collapse of timeline from years to days. The only difference is the intent of the operator. The Oracle does not know the difference.

It does not care.

It computes what it is asked to compute, by whoever holds it, for whatever purpose they determine. There is a constitutional dimension to this that must be named, because it connects to the deepest legal principles the democratic world has built. The presumption of innocence — enshrined in Article 11 of the Universal Declaration of Human Rights and the foundation of the Fifth, Sixth, and Fourteenth Amendments to the United States Constitution — holds that a person cannot be punished for something they have not done.

An autonomous weapon that selects and engages a target based on a probability model is, by the standards of international humanitarian law, a war criminal by default. Not because it is malicious.

Because it cannot exercise the human judgment that the Geneva Conventions require for distinction — the ability to differentiate between a combatant and a civilian — and for proportionality — the judgment that the military advantage of a strike justifies the civilian harm it causes. A probability is not a judgment. A confidence interval is not discernment. An algorithm that calculates a 70% likelihood that a building contains a combatant and authorizes a strike has treated the 30% — the civilians, the children, the people who happened to be in the wrong building — as acceptable statistical noise.

This is antithetical to the legal framework that governs the use of force. Any autonomous weapon, by the fact of its autonomy, represents a breach of constitutional and humanitarian ethics. The logic is the same logic that powers a recidivism algorithm in an American courtroom.

The system that says “this person is 60% likely to reoffend, so we keep them locked up longer” is inverting the presumption of innocence — punishing a human being for something they have not done, based on characteristics they share with a demographic group.

The system that says “this building is 70% likely to contain a combatant, so we strike it” is the same inversion at a different altitude. Both treat actual human beings as probabilities. Both are constitutionally indefensible. Both are operationally unchallenged — because the people affected by recidivism algorithms are disproportionately Black and poor, and the people affected by autonomous targeting are disproportionately brown and far away. The constitutional violation is hiding in plain sight because the victims lack the power to challenge it. In New York, the Oracle is a product.

In Toronto, it is a dependency. In Lusaka, it is a condition. The person in New York has agency. The person in Toronto has a negotiating position. The person in Lusaka has an app on their phone and no idea what is behind it. The distribute-then-recapture cycle operating not across time but across geography. The capability distributes — mobile banking reaches Lusaka, diagnostic AI reaches Toronto’s hospitals, the cloud reaches everywhere.

But the control does not distribute with the capability. The control remains where the models are trained, where the chips are manufactured, where the data centers operate, where the capital concentrates. The further you are from the center, the less visible you are to the system — and the less the system is designed to account for you.

Every reader of this book lives at one of these three altitudes. The reader in São Paulo recognizes Lusaka. The reader in Seoul recognizes Toronto. The reader in

London sees somewhere between Toronto and New York depending on the domain. The reader in Lagos, Dhaka, and Lima knows exactly which altitude they occupy. The Oracle does not need to be explained to them.

They are already living under it.

They just did not know it had a name.

What the Oracle Sees

The previous two chapters documented the substrates. Chapter 9 described a quantum computer that can explore every possible solution simultaneously — a machine that does not search for the answer but computes all answers at once, collapsing to the correct one when measured. Chapter 10 described a biological computer that can store 215 petabytes in a gram of DNA, can copy itself, can evolve, and operates on an architecture that has been debugging itself for 3.8 billion years. Combine these substrates with the data infrastructure that already exists — the six billion smartphone sensors documented in Chapter 8, the financial transaction records, the health records, the social media activity, the satellite imagery, the genomic databases, the climate data — and the Oracle is not a single machine.

It is a capability that emerges when sufficient compute meets sufficient data at sufficient scale.

What does that capability look like?

It looks like this: a system that can predict, with a precision that no previous technology approached, the health trajectory of a population, the creditworthiness of a borrower, the likelihood that a defendant will reoffend, the probability that a student will drop out, the risk that a business will default, the yield of a crop, the spread of a disease, the outcome of an election, the movement of a market, and the behavior of a person. Not because it knows the future.

Because it has enough data about the past, enough compute to process it, and enough statistical power to identify patterns that no human analyst could detect. Not science fiction. Fragments of the Oracle already exist. A healthcare algorithm that processed 200 million patients predicted their future utilization — and confused cost with need, systematically underserving Black patients. A sentencing algorithm predicted recidivism — and encoded racial bias as mathematics. A social media recommendation engine predicted engagement — and optimized for outrage. Each was a partial Oracle — a system that could see a fragment of the future and act on it.

Each was also wrong in ways that the people who built it did not anticipate and that the people who lived under it could not detect. The Oracle Problem is not what happens when prediction fails.

It is what happens when prediction succeeds — when the machine’s forecast is accurate enough to be actionable, and the entity that owns the machine acts on it.

Because the action changes the future. And the entity that can see the future and change it has a power that no previous concentration of computational capability has provided.

Who Owns the Oracle

The person in New York may build the Oracle. The person in Toronto may negotiate the terms of its use. The person in Lusaka will live under it.

But none of them will own it. The Oracle — the full convergence of quantum computing, biological computation, and AI operating on global-scale data — will be owned by the entities that can afford to build and operate it. As of this writing, fewer than ten organizations on Earth can train a frontier AI model. Fewer than five are investing seriously in quantum computing at scale. The number that will be able to operate a converged quantum-biological-AI system, when that convergence arrives, may be smaller still.

The person in New York works for one of these organizations, or adjacent to one. Their proximity gives them visibility, not control. The engineer who builds the Oracle does not own it any more than the engineer who built the Starlink satellite owns the constellation. The ownership belongs to the entity that capitalized the infrastructure — and that entity’s decisions will be governed by whatever combination of competitive pressure, regulatory constraint, and institutional values applies at the time. Those values can change with a board vote.

The person in Toronto is governed by an entity they did not elect, operating under laws they did not write, making predictions about their citizens that they cannot audit. The Oracle’s owner is not the Canadian government.

It is a company in California or a consortium that spans multiple jurisdictions, accountable to its shareholders and, in principle, to the regulatory frameworks of the countries where it operates — frameworks that, as this book has documented across ten chapters, do not yet exist in a form adequate to the capability. The person in Lusaka is not governed by the Oracle’s owner.

They are not even visible to the Oracle’s owner.

They are a row in a dataset, a data point in a model, a prediction in a system that was not designed for them and that they have no mechanism to query, challenge, or escape.

If the Oracle's prediction about them is wrong — if the credit score is too low, if the health screening misses their risk factors, if the agricultural pricing model undervalues their crop — they will bear the cost. And they will not know why.

The Jar in the Room

This book began with a shepherd pressing a clay token into a jar. The token represented a sheep. The jar was the first ledger. The ledger was the first computer. And the first question of computational power was: who holds the jar? For five thousand years, the jar has changed form — from clay to clockwork, from vacuum tubes to transistors, from silicon to code, from code to neural networks. In every era, the pattern has repeated: the capability distributes, the control reconcentrates. The tokens changed.

The jar changed.

The question did not. The Oracle is the jar that holds all the other jars.

It is the computational capability that subsumes and exceeds prior capabilities — the ability to compute things the rest of the world cannot verify, to predict outcomes the rest of the world cannot anticipate, to see patterns the rest of the world cannot detect. The entity that holds this jar does not merely concentrate power.

It concentrates the ability to *know* — to see the future with a precision that functions, for all practical purposes, as omniscience. And the question — who holds the jar? — has never mattered more than it does now.

Because in prior era, when the jar reconcentrated, the capability it held was powerful but limited. The printing press concentrated the power to inform. The electrical grid concentrated the power to energize. The internet concentrated the power to connect. Each concentration was consequential and each was bounded — the entity that controlled the press could not control the grid, and the entity that controlled the grid could not control the network. The Oracle is not bounded in this way. The convergence of quantum computing, biological computation, and AI produces a capability that crosses prior boundaries.

The entity that holds the Oracle can compute in medicine, finance, defense, agriculture, climate, education, and surveillance — simultaneously, at scale, with a precision that no specialized system can match. The jar does not hold tokens for

sheep. The jar holds tokens for everything.

What Happens Without a Keeper

Return to the three mornings.

If the Oracle arrives without governance, without responsible keepers, without frameworks that distribute the benefit and constrain the power — the person in New York will do well.

They are proximate to the capability.

They will benefit from the productivity, the medical advances, the financial tools, the competitive advantages that the Oracle provides to those close enough to access it. The Oracle will make their life better. The person in Toronto will do acceptably. Their government will negotiate access. The healthcare system will license the diagnostic tools. The financial system will adopt the fraud detection. The Oracle will improve their life in measurable ways.

But the terms will be set elsewhere, the data will flow elsewhere, and the dependency will deepen with every year that passes without the domestic capability to build an alternative. The person in Lusaka will bear the cost. The agricultural pricing model that does not account for their conditions will undervalue their crop. The health screening that was not validated on their population will miss their risk factors. The credit algorithm that does not understand their economy will deny them the capital that might have changed their trajectory.

And they will not know why.

They will know only the outcome: the loan denied, the diagnosis missed, the price too low. The cost will be real and the mechanism will be invisible — exactly as it was for the 200 million patients whose healthcare algorithm confused cost with need, whose disparity was invisible to everyone in the system until a researcher chose to look. Tragedy is too strong a word for what follows from a badly calibrated credit algorithm.

But it is not too strong for the aggregate. Multiply the badly calibrated algorithm by every health decision, every credit decision, every agricultural pricing decision, every legal decision, every insurance decision that the Oracle influences, across every population that has no seat at the table where the Oracle's rules are written — and the cost is not a miscalibration.

It is a permanent restructuring of who benefits from computational power and who pays for it.

It is the margin made structural.

It is the gap made permanent. For the select few who hold the jar, this is an era of extraordinary possibility. For everyone else, it is an era of extraordinary dependence — dependence on entities whose capabilities you cannot verify, whose decisions you cannot challenge, and whose interests may not align with yours.

“For the select few who hold the jar, this is an era of immense possibility. For everyone else, it is an era of immense dependence — dependence on entities whose capabilities you cannot verify, whose decisions you cannot challenge, and whose interests may not align with yours.” The shepherd in Chapter 1 pressed a token into a jar and invented computation.

He did not know what would follow.

Five thousand years later, the token is a qubit. The jar is a data center.

The question has not changed: who holds it, and what do they do with what they see? The next chapter asks whether the oldest pattern in computing — the pattern that has reconcentrated power in every era this book has documented — can be broken. The evidence so far suggests it cannot.

But the evidence is not the verdict, and the pattern is not fate — unless nobody tries.

The Distributed Question

On decentralization, blockchain, and the tension between distributed architecture and concentrated power

The Distributed Question

On whether the oldest pattern in computing can be broken, and the test we have already failed.

On September 4, 1882, Thomas Edison flipped a switch at the Pearl Street generating station in lower Manhattan. Electricity flowed through copper wires buried beneath the streets to fifty-nine customers — banks, newspapers, and offices within a half-mile radius of the station. Each customer had been wired individually. The station could serve no one outside its immediate neighbourhood. The electricity that illuminated J.P. Morgan's office at 23 Wall Street could not reach a home in Brooklyn, ten minutes away. Electrical power, in September 1882, was as concentrated as quantum computing is today.

The technology existed.

It worked. Its potential was obvious. And it was available to exactly fifty-nine customers, all of whom were wealthy, all of whom were located within walking distance of the generator, and all of whom had a direct, personal relationship with the man who built the station.

Within forty years, the electrical grid had reached millions of homes and businesses across America. The generation remained centralized — power plants were enormous, expensive, and required specialized engineering.

But the distribution became universal. You did not need to own a generator to use electricity. You needed a wire to your house and a meter on your wall. Centralized generation, universal distribution. The most concentrated technology of the 1880s became the most ubiquitous infrastructure of the twentieth century.

The question of this chapter — the question the entire book has been building toward — is whether quantum and biological computing will follow the same arc. Whether centralized generation and universal distribution is the future of the next computing substrate. Or whether something about this substrate is fundamentally different.

The Hopeful Arc

The case for distribution is grounded in real trends and real historical precedent, and it deserves to be stated as strongly as its best proponents would state it. IBM, Google, Amazon, and Microsoft already offer quantum computing access through cloud services. A researcher in Lagos or Bangalore can run a quantum circuit on IBM's hardware today, from a laptop, for free. The model is familiar and powerful: you do not need to own the machine. You need access to it. And cloud access can be provided at marginal cost to an essentially unlimited number of users, the way Edison's successors extended electrical service from fifty-nine customers to fifty million.

The quantum research community is, at present, substantially open. Algorithms are published in academic journals. Hardware specifications are presented at conferences. Open-source quantum programming frameworks are freely available. The cultural norm favors sharing, and this norm has deep roots — it descends from the same collaborative academic culture that built the internet, that published the foundations of AI, that made Linux and Python available to the world. Open scientific culture is not a guarantee against concentration, but it is a genuine counterweight, maintained by people who believe that knowledge shared is more valuable than knowledge hoarded.

And no computing monopoly in history has lasted more than a generation. IBM dominated mainframes and lost the PC era. Microsoft dominated the PC and lost the mobile era. The United States dominated computing for decades and now faces genuine competition from China, the European Union, and others. The historical record suggests that monopolies on computational capability are structurally unstable — new substrates, new entrants, and new paradigms consistently disrupt concentrated positions. The hopeful arc says: quantum computing will follow the pattern.

It will begin concentrated, become available through cloud services, and eventually distribute its benefits broadly. Pearl Street Station served fifty-nine customers. The grid serves three hundred million. The arc of electrical power bent toward distribution. The arc of computing has bent the same way for five thousand years.

The Lecture Hall

Now let me tell you about the lecture hall.

The number of people on Earth who can design quantum error correction codes, build quantum hardware at the leading edge, or develop novel quantum algorithms is, by most estimates, in the low thousands. Their training requires a decade of specialized education in quantum physics, computer science, and applied mathematics. You cannot train them faster by spending more money, any more than you can produce a baby faster by assigning nine women to the project.

The talent constraint is a bottleneck that money alone cannot solve, and the entities that have attracted the most talent — through compensation, resources, and the appeal of working on the most advanced technology in the world — are a small number of the wealthiest companies and governments.

Not like electricity. Edison's generators were expensive, but the engineering required to build them was within reach of dozens of companies by the 1890s. The knowledge was transferable. The workforce could be trained in years, not decades. The materials were available. The physics was classical — copper, iron, steam, rotation. Anyone who understood electromagnetism could, in principle, build a generator. Quantum computing operates at 15 millikelvin — colder than anywhere in the natural universe.

The machines require electromagnetic shielding measured in decibels of isolation from the environment. The control electronics are of extraordinary precision. None of these requirements is amenable to the kind of exponential cost reduction that made transistors ubiquitous. The fundamental physics of quantum coherence demands extreme isolation, and extreme isolation is inherently expensive. And then there is the strategic dimension. Any government that achieves a working, large-scale quantum computer will face enormous pressure to classify that capability, just as Britain classified Colossus in 1944. The intelligence value of breaking adversaries' encryption — retroactively, across years of archived communications — is so large that no responsible government would voluntarily disclose it.

Classification is not paranoia.

It is rational strategy. And once a capability is classified, it exits the open research ecosystem entirely.

“The number of people who can build quantum hardware at the leading edge could fit in a lecture hall. You cannot train them faster by spending more money, any more than you can produce a baby faster by assigning nine

women to the project.”

Cloud Access Is Not Ownership

The electrical grid analogy is appealing, and I have presented it honestly.

But there is a flaw in it that must be named, because it determines whether the analogy holds or collapses.

When you plug a lamp into a wall outlet, the power company does not know what the electricity is being used for.

It does not know whether you are reading a book, charging a phone, or running an illegal operation. The meter measures how much power you consume.

It does not monitor what you do with it. Electricity is a dumb pipe.

It delivers energy and asks no questions.

When you run a quantum circuit on a cloud provider’s hardware, the provider has full visibility into your computation.

It knows what you are computing, how much capacity you are using, and what results you are receiving. Cloud quantum access is access with surveillance, not access instead of surveillance. The provider can throttle your access, revoke it, price it out of reach, or share information about your computations with governments that have legal or extralegal authority to compel disclosure. This distinction matters enormously. The electrical grid distributed power without distributing knowledge about how that power was used. Cloud quantum computing distributes capability while concentrating knowledge about how that capability is used.

The user gets the computation. The owner gets the intelligence. These are not the same thing, and conflating them is the most common error in optimistic assessments of quantum computing’s future. The United States has already demonstrated its willingness to restrict computational substrates for strategic reasons.

In 2022 and 2023, the government imposed sweeping restrictions on the export of advanced semiconductor technology to China, explicitly framed in terms of national security. The precedent exists. The political will exists. The legal frameworks exist. Applying them to quantum computing is not a hypothetical.

It is an extension of policies already in force.

The Test We Have Already Failed

Something the hopeful arc tends to gloss over.

We have faced the distributed question before, with the internet, and we failed it. The internet was designed as a decentralized, egalitarian, open system. Its protocols were free. Its architecture had no gatekeepers. Its culture favored sharing, collaboration, and open access.

It was, in every structural respect, designed for distribution. Tim Berners-Lee gave away the web. The academic community published the research openly. The open-source movement built the tools for free. Everything that the optimistic case for quantum computing points to — cloud access, open research, collaborative culture — the internet had in abundance.

Within two decades, it had consolidated into a platform oligarchy that concentrates more informational power in fewer hands than any institution in human history. The consolidation was not the result of a conspiracy or a policy failure.

It was the natural consequence of network effects, economies of scale, and the gravitational pull of platforms that become more valuable as they grow larger. The architecture was democratic. The economics were not. The economics won.

The lesson is uncomfortable but necessary. Good architecture is not sufficient to prevent concentration. Good intentions are not sufficient. Open protocols and open culture are not sufficient.

If the underlying economics favor concentration — and in the quantum era, the economics overwhelmingly favor concentration, because the cost of building and operating the machines is astronomical — then concentration will occur unless it is actively, aggressively, and continuously counteracted by policy, investment, and institutional design. Distribution is not a default.

It is an achievement — one that must be won, maintained, and defended against the gravitational forces of economics and power that perpetually pull toward concentration.

The Two-Tier World

The most likely outcome, stated precisely: Quantum computing will become available through cloud services, and that access will be genuinely useful. Researchers will run quantum algorithms. Companies will optimize logistics and simulate materials. Drug discovery pipelines will incorporate quantum simulation. The Edison arc will play out at this level. The cloud is the grid. The computation will flow.

But the leading edge of quantum capability — the full power of the most advanced machines, including capabilities that are strategically sensitive — will be held

privately by a small number of entities and will not be accessible through any cloud service. The gap between what is publicly available and what is privately held will not just be quantitative — faster computation of the same kind.

It will be qualitative — computation of a fundamentally different kind, solving problems that the public tier cannot approach. In prior eras of computing, the government's supercomputer was faster than your laptop, but they were doing the same kind of computation. The gap was like the gap between a bicycle and a car — one is faster, but both travel roads.

In a quantum era, the gap may be like the gap between a bicycle and a submarine.

They do not operate in the same medium. The holder of quantum capability is not ahead of you on the same road.

They are traveling through a domain you cannot enter. A two-tier world in military capability means one nation has better tanks. A two-tier world in computation means one entity has better *everything* — better intelligence, better weapons design, better economic modeling, better drug development, better climate prediction, better ability to anticipate and counter the moves of anyone operating in the first tier. Computational superiority is not a domain advantage.

It is a meta-advantage — an advantage that applies across all domains simultaneously.

What Could Break the Pattern

This chapter held two opposing ideas in mind, as Fitzgerald demanded. The hopeful arc and the lecture hall. The grid and the submarine.

Now I want to name what could tip the balance, because the outcome is not yet determined. A breakthrough in room-temperature quantum computing would change everything. The cryogenic cooling requirement is the single largest barrier to distribution.

If quantum coherence could be maintained at or near room temperature, the cost and infrastructure requirements would drop by orders of magnitude. This would be the transistor moment — a phase transition from institutional to personal.

It is not certain.

It may not be possible.

But if it happened, the lecture hall would empty into the world. An international governance framework, established before the capability matures, could prevent the worst-case concentration. The governance window is open now. Nuclear

non-proliferation did not prevent all proliferation, but it prevented the worst case of dozens of nuclear-armed states. A comparable framework for quantum computing is conceivable, if the political will materialises before the window closes. Democratic societies choosing to invest in publicly accessible quantum infrastructure could distribute capability the way public universities distribute education.

The European Union’s Quantum Flagship program and the United States National Quantum Initiative are steps in this direction, though their scale relative to private investment is modest. A sustained public commitment comparable to the interstate highway system could meaningfully distribute quantum capability. Whether democratic societies have the appetite for that level of investment is a political question, not a technological one. And public vigilance — the kind of sustained, informed attention that recognizes concentration when it is forming and demands distribution before it is too late — could provide the democratic counterforce that no technology can provide on its own.

Technology does not distribute itself. People distribute it — through policy, through investment, through the choices they make about what to fund, what to classify, what to share, and what to demand.

“Technology does not distribute itself. People distribute it — through policy, through investment, through the choices they make about what to fund, what to share, and what to demand. The distributed question is a question about values. Values are not computed. They are chosen.”

What Comes Next

The oracle problem has been presented and the distributed question — the *what* and the *who* of the power dynamics that the next generation of computing substrates will produce.

Two chapters remain, and they are different in character from everything that has come before. The next chapter was originally intended to be a synthesis — a catalog of the five patterns, a list of indicators, a framework for monitoring the future. I wrote that chapter. And then, as I was finishing this book, the thesis stopped being theory. An American AI company drew two lines against the use of its technology for mass surveillance and autonomous weapons. The government designated the company a national security threat — a label never before applied to an American firm.

The public responded by making that company’s product the most downloaded app in America within seventy-two hours.

The patterns I had been describing across twelve chapters played out simultaneously, in public, in a single week. The next chapter tells that story. The final chapter returns to the shepherd, the jar, and the question that has echoed through five thousand years of computing history — with a clarity that the events of that week have made more urgent than I anticipated when I began writing. The window will not stay open forever.

It never does. And the evidence now suggests it is closing faster than I expected.

CHAPTER WHJ-13

The Spiral

On the full arc of the distribute-then-recapture cycle and the pattern that predicts the future

The Spiral

On the five patterns that govern every era of computing, and the week the book's thesis became news.

As I was completing this book, the thesis stopped being theory. In February 2026, the United States Department of Defense — rebranded as the Department of War by Executive Order 14347, signed in September 2025 — gave an ultimatum to Anthropic, the San Francisco artificial intelligence company that builds the Claude AI system. The rebranding is itself instructive: only an act of Congress can formally change the department's statutory name, so the executive order authorized “Department of War” as a secondary title for public communications, ceremonial contexts, and non-statutory documents.

The semantic shift — from defense to war — was accomplished by bypassing the legislative process entirely, projecting a posture of lethality through administrative rebranding rather than democratic deliberation. The demand to Anthropic was this: remove all restrictions on how the military uses your technology, or face consequences no American company has ever faced. Anthropic had held a \$200 million contract with the Department of War since 2025.

It was the first AI company to deploy its models on classified military networks. Its technology was integrated into military operations, intelligence analysis, and cyber defense. The company had not refused to work with the military.

It had drawn two lines: Claude would not be used for mass domestic surveillance of American citizens, and it would not be used to power fully autonomous weapons — weapons that select and engage targets without a human in the loop. Anthropic argued that its AI was not yet reliable enough for autonomous weapons, and that mass

surveillance was incompatible with democratic values regardless of the AI's capability. The Department of War's position was simpler: the military must have unrestricted access to any technology it contracts for, across all lawful purposes.

No carve-outs. No conditions.

When Anthropic refused, the Secretary of Defense designated the company a "Supply-Chain Risk to National Security" — a label previously reserved for foreign adversaries. The President directed all federal agencies to immediately cease use of Anthropic's technology. No American company had ever received this designation.

Within hours, OpenAI, Anthropic's primary competitor, announced its own deal with the Department of War.

"An AI company drew two lines: no mass surveillance of citizens, no autonomous weapons without human oversight. For this, it was designated a national security threat — a label reserved for foreign adversaries."

Every Pattern, Simultaneously

Twelve chapters built five patterns that repeat at every substrate transition in the five-thousand-year history of computation. The Anthropic confrontation is every one of those patterns, playing out simultaneously, in public, in a single week. The Ceiling and the Leap. AI has reached a capability threshold where military applications are operational, not hypothetical. Claude was already deployed on classified networks. The substrate has reached the point where the power it confers is too significant for the holder to ignore — and too significant for the public to leave ungoverned.

The Expansion of Who — Reversed.

Every previous computing substrate expanded who could compute. The Anthropic story demonstrates the reversal: a government claiming unconditional access, with threats of economic destruction for the company that resists. The circle is contracting. The entity that holds military power is demanding that the entity that built the computational tool surrender control of how it is used. The Unexpected Use. Anthropic built Claude for writing, analysis, coding, and research. The military found uses the builders did not anticipate and found objectionable.

The first use is never the consequential use. The consequential use arrives later, demanded by actors the builders cannot control. The Return to Nature. At its deepest level, the confrontation is about whether an intelligence — artificial but modeled on human cognition — should be permitted to make lethal decisions without human judgment in the loop. Distribute, then recapture. Anthropic distributed its technology

to the government under conditions. The government attempted to recapture it without conditions.

When the company resisted, the government wielded a designation designed for foreign adversaries to force compliance. The cycle that has governed every era of computing played out in days rather than decades.

The Democratic Response

What happened next is the part of the story I did not predict. The public paid attention.

Within seventy-two hours, Anthropic's Claude app rose from outside the top 100 to the number one most downloaded app in Apple's U.S. App Store, overtaking ChatGPT. Daily sign-ups broke all-time records — eventually exceeding one million per day. Free users increased more than 60 percent. Paid subscribers more than doubled. The app reached number one in sixteen countries. Chalk messages appeared on the sidewalk outside Anthropic's San Francisco offices reading "Thank you" and "You give us courage." Outside OpenAI's offices, the messages read "Do the right thing." Be clear about what this is and what it is not.

It is not the resolution of the distributed question. Being designated a supply-chain risk by the most powerful military in the world carries consequences that app downloads cannot offset. The legal battle is just beginning.

But it is evidence that the public, when presented with a clear example of the power dynamics this book describes, can recognize the stakes and act. Not through legislation. Not through regulation. Through the mechanism that requires no institutional permission: choosing where to direct attention and money. The greatest advantage that concentration has over distribution is silence. The Anthropic confrontation broke the silence.

What the Patterns Predict

The Anthropic confrontation is a preview.

It is not the endgame.

The patterns generate specific predictions. The governance window is narrower than most people realize.

If the patterns hold, the AI governance regime will solidify within three to five years. The decisions being made now — in confrontations exactly like this one — are not preliminary.

They are establishing the precedents that will govern AI for decades.

The question of whether AI companies can set conditions on military use, once resolved, will not be relitigated. The concentration will deepen before it distributes — if it distributes at all. The cost of training frontier models is rising. The number of entities that can build them is shrinking. Governments are asserting control.

If a government is willing to threaten the destruction of an AI company over conditions on a language model, what will it do when a company controls a quantum computer that can break encryption? And the public response will matter — but only if it is sustained. The surge in Claude downloads is encouraging but fragile. Public attention is the scarcest resource in the information economy. Whether the public continues to reward companies that set conditions and punish those that do not will determine whether the democratic counterforce has lasting power or is merely a momentary flash of conscience.

Be honest about the limits of what happened, because honesty is what this book owes the reader. The surge in Claude downloads is a leading indicator.

It is not a governance regime. App downloads do not protect data centers from state seizure. Consumer preference does not override sovereign military power. A company that refuses a defense contract can be blacklisted, acquired, or regulated out of existence by the government whose contract it refused. The public's ability to reward ethical behavior through the App Store is real, measurable, and entirely insufficient as a substitute for law. The internet teaches this lesson with brutal clarity. The public chose open protocols, open-source software, and decentralized architectures.

The public also chose Gmail, Facebook, and Amazon — because convenience outweighed principle every time the two conflicted. Consumer preference without legislative frameworks is a gesture, not governance.

It is a letter of intent that carries no enforcement mechanism. The same public that downloaded Claude can, and statistically will, migrate to whichever platform offers the most capable model — regardless of that platform's ethical commitments — the moment the capability gap becomes noticeable. Public action without legislative frameworks is precisely what the internet had in abundance: good intentions, open architectures, collaborative culture.

Within two decades, the result was a platform oligarchy. The download is necessary.

It is not sufficient. The download tells the builders what the public values. The law tells the builders what the public requires. Only the law survives a change in public attention.

The Leading Indicators

Frameworks are useful only if they generate observable predictions.

Here are ten signals I am monitoring.

When three or more shift materially, the landscape has changed. One: a quantum computer solves a commercially valuable problem faster and cheaper than any classical alternative. Two: a major nation-state classifies a quantum computing capability or restricts publication of quantum research. Three: DNA storage is offered as a commercial archival service. Four: a biological computing agent performs a useful computation inside a living organism. Five: a neuromorphic chip outperforms a GPU on a mainstream AI workload at dramatically lower energy. Six: the cost of a useful quantum installation falls below \$100 million.

Seven: a regulatory framework emerges for programmable biological organisms. Eight: a geopolitical event is credibly attributed to asymmetric quantum capability. Nine: a government designates an AI company a national security risk for maintaining safety restrictions. Ten: the public measurably rewards a company that resisted power concentration and punishes one that complied.

Two of ten indicators shifted in a single week.

The first to shift were not the technical indicators but the political ones. The crisis is arriving not through a quantum breakthrough but through the assertion of state power over the companies that build the tools.

“Two of ten indicators shifted in a single week. The crisis is arriving not through a quantum breakthrough but through the assertion of state power over the companies that build the tools.”

A Note on Humility

Patterns are not destiny.

Every pattern in this book is an observation drawn from historical data, and the future is not obligated to follow. I present these frameworks not as certainties but as the best instruments I can construct from the evidence available. I would rather be wrong about the predictions and useful with the framework than right about the predictions and useless in the face of them.

But I will say this.

When I began writing this book, the Anthropic confrontation had not happened.

The patterns were hypotheticals.

Now they are not. The speed is faster than I expected. And the window is narrower than most people realize.

CHAPTER WHJ-14

The Choice

On the question that closes the book and opens the next

The Choice

On what it means to live at the moment when the machine meets the living world, and the oldest question in computing demands an answer.

Return to the shepherd.

Five thousand years ago, in a river valley in southern Mesopotamia, a man needed to know whether all his sheep had come home.

He could not count them reliably in his head. So he dropped a pebble into a clay jar for each one that left in the morning and removed a pebble for each one that returned in the evening. The jar held the count. The man held the jar. And the question that would echo through every chapter of this book was born in that simple act: who holds the jar? That question spans five millennia. The jar has taken many forms. Gears in a brass casing pulled from a shipwreck.

Punch cards threaded through a loom. Vacuum tubes filling a room the size of a house. Transistors etched onto chips smaller than a fingernail, manufactured thirteen sextillion times. Signals pulsing through glass fibers beneath the ocean. Neural networks trained on the accumulated text of human civilization. Qubits suspended in magnetic fields at temperatures colder than outer space. Nucleotide sequences in a synthetic strand of DNA. The jar has changed beyond recognition.

The question has not.

What I Have Learned

This book grew from a realization that the current moment in computing — the convergence of AI, quantum computing, and biological substrates — cannot be understood by looking at any one technology in isolation.

It can only be understood by looking at the full trajectory and identifying the patterns that have governed prior transitions.

Every previous substrate eventually distributed. This one may not. Prior concentrations of computational power were eventually broken by a new substrate that made computation cheap again. This time, the new substrates are more expensive, not less.

Every previous governance challenge was eventually resolved. This time, the thing being governed is invisible, dual-use, and evolving faster than the institutions charged with governing it. And then, as I was finishing this book, a technology company was threatened with economic destruction for insisting that its AI should not be used for mass surveillance or autonomous weapons.

The patterns I had described as historical observations became current events. The hypotheticals became headlines.

The Window

The governance window is open because the technologies are still forming. Quantum computers exist but are not yet powerful enough for their most consequential applications. DNA storage is demonstrated but not yet commercial. Biological computing is in the laboratory, not in the body. And AI — the substrate that has already reached operational capability — is being contested right now, in real time, through confrontations between companies that build it and governments that want unrestricted access to it. This is exactly the moment when governance is most possible and least urgent-feeling.

The technology works but has not yet caused the full crisis. The framework can be built before the building is on fire. The alternative is to wait until the crisis arrives and govern in reaction rather than anticipation — which is how we governed the internet, how we governed social media, and how we are currently governing artificial intelligence.

The results have been, to use the mildest possible characterization, suboptimal. The Anthropic confrontation demonstrated both the peril and the possibility. The peril: a government willing to use unprecedented legal designations to force a company to remove safety constraints. The possibility: a public willing to respond immediately, visibly, and economically — rewarding the company that held its line and punishing the competitor that did not.

“The window is the moment when governance is most possible and least urgent-feeling. The framework can be built before the building is on fire. The alternative is what we did with the internet.”

What I Ask of the Reader

Fourteen chapters is a great deal to ask of a reader. You have followed me from a shepherd’s field to a quantum laboratory. I am grateful for your attention. I want to repay it with three specific requests. First: do not delegate the distributed question to experts.

The question of who has access to the next generation of computational power is not a technical question.

It is a political one.

It will be answered through legislation, regulation, investment, international agreement, and public pressure. Experts can inform the answer.

They should not own it.

The consequences are too broad and too permanent to be resolved by any single discipline. Second: watch the indicators. The ten signals I identified in the previous chapter are not predictions.

They are instruments. Use them. Track them.

Two have already shifted.

When the next cluster shifts, the landscape has changed and the assumptions beneath your strategy need to be reexamined. Third: talk about this. The greatest advantage that concentration has over distribution is silence. Concentration thrives when the public is not paying attention. The Anthropic confrontation proved that when the public does pay attention, it can move markets, reshape brand loyalty, and send a signal that companies and governments cannot ignore. A week of attention produced a million daily downloads and a number-one app.

Imagine what sustained attention could produce.

What the Room Must Write

If I am asking you to pay attention, I owe you specificity about what to pay attention to. The following are not predictions.

They are tenets — minimum conditions that any serious governance framework for the computational era must address.

They are drawn from the evidence of fourteen chapters. First: mandatory hardware-level kill switches for critical infrastructure. Any AI system integrated into infrastructure whose failure would cause mass harm — power grids, financial systems, healthcare networks, water treatment, air traffic control — must include a physical mechanism for human override that cannot be disabled by software.

The lesson of Therac-25, of MCAS, of every system failure in this trilogy is the same: when the machine fails, the human must be able to stop it. A software kill switch is a contradiction in terms. The override must be physical, hardware-level, and non-negotiable. Second: binding international treaties prohibiting autonomous lethal strikes without meaningful human judgment. The twenty-second approval window documented in the Lavender system is not meaningful human judgment.

It is a legal fig leaf designed to satisfy the letter of international humanitarian law while violating its spirit. The standard must be defined by treaty, not by the military that deploys the system. Third: strict, non-transferable corporate liability for algorithmically generated harms.

When an algorithm denies healthcare, extends a prison sentence, or produces a decision that causes measurable harm to an identifiable person, the entity that deployed the algorithm must bear strict liability. You cannot outsource the decision and disclaim the consequence. The deployer holds the jar. The deployer bears the cost. Fourth: mandatory algorithmic transparency for any system that affects liberty, livelihood, or life. The trade-secret protection that shields COMPAS from public scrutiny is constitutionally indefensible when the system's output determines whether a human being remains in a cage.

Any algorithm used in criminal sentencing, healthcare triage, credit decisions, or military targeting must be subject to independent audit, with results published. Opacity is not a business model when the product is a judgment about a human life. These are minimum conditions.

They are not aspirational.

They are the floor below which any governance framework is performative.

They will be resisted by every entity that profits from the absence of constraint. The resistance is not evidence that the tenets are wrong.

It is evidence that they are necessary.

The Jar

Five thousand years ago, a shepherd solved his problem with a handful of pebbles and a hollow piece of clay. The solution was simple. The principle was not.

He had discovered that a physical object could hold information that his mind could not, and that by consulting the object he could make a decision he could not have made alone.

Every machine in this book is a descendant of that jar. The Antikythera mechanism was a jar that held the movements of the heavens. Babbage's Engine was a jar that would have held perfect mathematical tables. ENIAC was a jar that held the trajectories of artillery shells and the calculations for nuclear weapons. The transistor was a jar so small and so cheap that it could be manufactured thirteen sextillion times. The internet was a jar that connected every other jar. The smartphone was a jar that watched the person holding it.

The neural network was a jar that learned. The quantum computer is a jar that can hold answers to questions that no other jar can formulate. And the DNA molecule is a jar that nature built 3.8 billion years before we picked up our first pebble. The jars have grown, across five millennia, from something that fit in a shepherd's hand to something that may model the full complexity of physical reality. And the question of who holds them has grown from a matter of pastoral convenience to a matter of civilizational consequence.

The machine does not care who holds it.

It never has. The Antikythera mechanism did not choose to serve priests. ENIAC did not choose to calculate bomb yields. Google does not choose to concentrate attention. A quantum computer will not choose to break encryption or cure diseases.

The machine computes what it is directed to compute, by whoever holds it, for whatever purpose they determine.

It is the most powerful amplifier of human intent ever created, and it is perfectly indifferent to the nature of that intent. The choice has always been ours.

It was ours when the British government classified Colossus rather than share the knowledge.

It was ours when the personal computer was built by people who believed computation should belong to individuals.

It was ours when Tim Berners-Lee gave away the web instead of patenting it.

It was ours when we traded our data for convenience and did not notice the terms.

It was ours when Dario Amodei told the Department of War that two uses of his technology were incompatible with democratic values, and it was ours when millions of people downloaded Claude in response.

It is ours now, as the next substrates take shape and the governance window narrows.

The difference between this moment and prior ones is the permanence of the consequence.

If the next substrate concentrates and the concentration holds — if a small number of entities achieve qualitatively superior computational capability and the physics prevents redistribution — the consequences may not be reversible within a generation.

They may not be reversible within a century. I began this book with a shepherd who held a jar of pebbles. I end it with a species that holds a jar large enough to contain the model of the world itself. The shepherd's question has not changed in five thousand years.

It has only grown more urgent.

And what will they do with it? The answer is not yet written. That is either a comfort or a warning. Probably both.

References & Source Attribution

REF-1: Ishango bone, approximately 20,000 years old, discovered 1960 in the Democratic Republic of Congo. The Ishango bone is described as approximately 20,000 years old with three columns of grouped notches, including a column with groups of 11, 13, 17, and 19. Source: Royal Belgian Institute of Natural Sciences (permanent exhibit); de Heinzelin, Jean, "Ishango," *Scientific American*, June 1962; Pletser, Vladimir & Huylebrouck, Dirk, "The Ishango Artefact: the Missing Base 12 Link," *Forma*, 1999. <https://www.naturalsciences.be/en/museum/collections-highlights/ishango> The mathematical interpretation of the notches remains debated. Some scholars see prime numbers and doubling sequences; others argue the groupings may be coincidental or functional (e.g., a tally stick for lunar observations). The text acknowledges the debate.

REF-2: Mesopotamian clay counting tokens and bullae, approximately 3500–3000 BCE. Mesopotamian shepherds used clay tokens of different shapes placed inside hollow clay balls (bullae), with impressions pressed into the exterior before sealing. Source: Schmandt-Besserat, Denise, *Before Writing, Volume I: From Counting to Cuneiform*, University of Texas Press, 1992; Schmandt-Besserat, Denise, "How Writing Came About," University of Texas Press, 1996. <https://sites.utexas.edu/dsb/> Schmandt-Besserat's theory that writing originated from accounting tokens is widely

accepted but not universally so. The text presents it as compelling evidence, not established fact.

REF-3: The abacus, appearing in Sumeria approximately 2700–2300 BCE, independently invented in multiple civilizations. The abacus appeared in multiple civilizations independently, including Sumeria, China, Rome, and Mesoamerica. Source: Ifrah, Georges, *The Universal History of Numbers*, John Wiley & Sons, 2000; Pullan, J.M., *The History of the Abacus*, Hutchinson, 1968. The exact dating of the earliest abacus is approximate. The Sumerian date range is based on archaeological evidence of counting boards and the general timeline of Sumerian mathematical development.

REF-4: 1946 abacus vs. electric calculator speed contest; Kiyoshi Matsuzaki. A Japanese postal clerk named Kiyoshi Matsuzaki using an abacus defeated an American soldier using an electric calculating machine in four of five categories. Source: *Stars and Stripes*, Pacific edition, November 12, 1946; widely cited in histories of the abacus including Kojima, Takashi, *The Japanese Abacus: Its Use and Theory*, 1954. The contest was arranged by *Stars and Stripes* newspaper and conducted at the Ernie Pyle Theater in Tokyo. The five categories were addition, subtraction, multiplication, division, and a composite. The abacus won all except multiplication.

REF-5: Antikythera mechanism, approximately 100 BCE, discovered 1901, at least thirty interlocking bronze gears. The Antikythera mechanism dates to approximately 100 BCE, contains at least thirty interlocking bronze gears, and could predict positions of the sun and moon, eclipses, and the schedule of the Olympic Games. Source: Freeth, Tony et al., “Decoding the ancient Greek astronomical calculator known as the Antikythera Mechanism,” *Nature* 444, 2006; Freeth, Tony et al., “A Model of the Cosmos in the ancient Greek Antikythera Mechanism,” *Scientific Reports* 11, 2021; National Archaeological Museum, Athens (permanent exhibit). <https://www.nature.com/articles/nature05357> The mechanism was recovered from a Roman-era shipwreck off the island of Antikythera between Crete and mainland Greece. The number of gears and the complexity of the mechanism have been progressively revised upward as imaging technology (particularly X-ray computed tomography) has improved. The 2021 paper proposes a more complex reconstruction than earlier models. Expanded (March 2026): Antikythera mechanism discovery details (expanded) The discovery narrative was expanded to include: Captain Dimitrios Kontos (also rendered Kondos in some sources) and his crew of sponge divers from Symi island; the Easter 1900 storm that blew them off course to

Antikythera; diver Elias Stadiatis (also Ilias Lykopantis) descending to 45 metres and surfacing describing “heaps of rotting corpses and horses” on the seafloor; Kontos diving himself and returning with a bronze statue arm; recovery operations November 1900 through 1901 with Hellenic Navy support; the death of diver Georgios Kritikos from decompression sickness and two other divers; Valerios Stais noticing the gear mechanism in May 1902; and Derek de Solla Price’s publication “Gears from the Greeks” in 1974. Sources: Wikipedia, “Antikythera mechanism” and “Antikythera wreck” (citing Freeth et al., Iversen & Jones, and Antikythera Mechanism Research Project publications); National Geographic, “The ‘Dial of Destiny’ is real” (July 2023); World History Encyclopedia, “Antikythera Mechanism” (August 2023); Antikythera.org.gr crew list established by Eleni Kladaki-Vratsanou, published in Efoplistis (January 2007); De Solla Price, D., Gears from the Greeks, Transactions of the American Philosophical Society, Vol. 64, No. 7 (1974).

REF-6: Pascal built the Pascaline at age 19; approximately 50 built, roughly 20 completed. Blaise Pascal began building the Pascaline around 1642 (age 19) to assist his father, a tax commissioner. Approximately fifty were built over the following decade, with fewer than twenty completed. Source: Marguin, Jean, *Histoire des instruments et machines à calculer*, Hermann, 1994; Connor, J.A., *Pascal’s Wager*, HarperOne, 2006; Musée des Arts et Métiers, Paris (permanent collection). Production numbers vary by source. Some sources cite 'about twenty' completed; others suggest up to fifty prototypes and models in various stages. Nine surviving Pascalines exist in museums.

REF-7: Leibniz binary arithmetic, 1679; Step Reckoner prototype, approximately 1694. Leibniz described a complete binary arithmetic system in 1679 and connected it to Chinese philosophy (the I Ching). His Step Reckoner prototype was completed around 1694. Source: Leibniz, G.W., “Explication de l’Arithmétique Binaire,” *Mémoires de l’Académie Royale des Sciences*, 1703(describing work from 1679); Leibniz-Archiv, Hannover; Morar, Florin-Stefan, “Reinventing Machines: The Transmission History of the Leibniz Calculator,” *British Journal for the History of Science*, 2015. The 1679 date refers to Leibniz’s manuscript; the 1703 date is publication. The connection to the I Ching was made by Leibniz after correspondence with Joachim Bouvet, a Jesuit missionary in China.

REF-8: Slide rule: invented by William Oughtred approximately 1622; effectively extinct by late 1970s after HP-35 introduction in 1972. The slide rule was used for approximately 350 years. Hewlett-Packard introduced the HP-35 pocket scientific calculator in 1972, and slide rule manufacturing and training ceased within

approximately five years. Source: Cajori, Florin, *A History of the Logarithmic Slide Rule*, 1909 (reprint); Stoll, Cliff, “When Slide Rules Ruled,” *Scientific American*, May 2006; HP corporate history archives. <https://www.hp.com/us-en/shop/tech-takes/hp-35-calculator-history> The HP-35 was priced at \$395 in 1972 (approximately \$2,900 in 2026 dollars). The speed of the slide rule’s obsolescence is well-documented in engineering education histories.

REF-9: Babbage: £17,000 government funding for the Difference Engine; Analytical Engine contained processor(mill), memory (store), program (punch cards), output. The British government invested approximately £17,000 in the Difference Engine. The Analytical Engine design contained a mill (processor), store (memory), punch card input, and output — every essential element of a modern computer. Source: Swade, Doron, *The Difference Engine: Charles Babbage and the Quest to Build the First Computer*, Viking, 2001; Science Museum, London (Babbage papers and reconstructed Difference Engine No. 2); Babbage, Charles, *Passages from the Life of a Philosopher*, 1864. <https://www.sciencemuseum.org.uk/objects-and-stories/charles-babbage-and-difference-engine> The £17,000 figure is from Science Museum records. Modern equivalent is approximately £2 million, depending on the deflator used. The Analytical Engine was never built; the architectural description is based on Babbage’s detailed plans and Ada Lovelace’s published notes.

REF-10: Ada Lovelace’s Note G (1843): first computer program; “other things besides number”; “has no pretensions whatever to originate anything”. Lovelace published extensive notes on the Analytical Engine in 1843, including Note G (algorithm for Bernoulli numbers, recognized as the first computer program) and observations that the Engine could act upon ‘other things besides number’ and ‘has no pretensions whatever to originate anything.’ Source: Lovelace, Ada, “Notes by the Translator” appended to Menabrea, L.F., “Sketch of the Analytical Engine,” *Scientific Memoirs*, Vol. 3, 1843; Toole, Betty Alexandra, *Ada, the Enchantress of Numbers*, 1992; Hollings, Christopher et al., *Ada Lovelace: The Making of a Computer Scientist*, Bodleian Library, 2018. <https://www.fourmilab.ch/babbage/sketch.html> The Menabrea paper with Lovelace’s notes is available in full at the Fourmilab link. Both quoted phrases are verbatim from Note A. Expanded (March 2026): Ada Lovelace biographical details (expanded) The Lovelace section was expanded to include: meeting Babbage at a London dinner party in 1833 at age 17; her father Lord Byron dying when she was 8; her mother Annabella Milbanke’s deliberate education of Ada in mathematics and science to prevent inheritance of Byron’s poetic temperament; Luigi Menabrea’s 1842 paper on the Analytical Engine based on Babbage’s Turin

lectures. Lovelace Launched the Digital Age (2014); Menabrea, L.F., “Sketch of the Analytical Engine Invented by Charles Babbage,” *Bibliothèque Universelle de Genève* (1842), with Lovelace’s notes appended to the English translation.

REF-11: Claude Shannon’s master’s thesis, MIT, 1937. Shannon’s 1937 master’s thesis demonstrated that Boolean algebra could be implemented with electrical relay circuits, bridging abstract logic and physical machinery. Source: Shannon, Claude, “A Symbolic Analysis of Relay and Switching Circuits,” M.S. thesis, MIT, 1937; published in *Transactions of the American Institute of Electrical Engineers*, Vol. 57, 1938. MIT archives. <https://dspace.mit.edu/handle/1721.1/11173> Widely described as the most important master’s thesis of the twentieth century (e.g., by Howard Gardner in multiple publications).

REF-12: Colossus: operational January 1944; up to 2,500 vacuum tubes; ten machines by war’s end; classified until mid-1970s. Colossus was designed by Tommy Flowers, first operational in January 1944, with up to 2,500 vacuum tubes in later models. Ten Colossus machines were operational by war’s end. The project was classified until the mid-1970s. Source: Copeland, B. Jack (ed.), *Colossus: The Secrets of Bletchley Park’s Codebreaking Computers*, Oxford University Press, 2006; Flowers, Thomas H., “The Design of Colossus,” *Annals of the History of Computing*, Vol. 5, No. 3, 1983; *The National Museum of Computing, Bletchley Park*. <https://www.tnmoc.org/colossus> Declassification occurred in stages; initial public acknowledgment came in the mid-1970s, with fuller details emerging through the 2000s. Flowers partly self-funded the first machine. Expanded (March 2026): Tommy Flowers personal details (expanded) The Flowers section was expanded to include: that he built Colossus partly with his own money; that he was a General Post Office telephone engineer, not a wealthy man; that his proposal had been rejected by Bletchley Park leadership as impractical; that he was told to destroy his records after the war; that he was unable to explain the gap in his professional record when applying for a bank loan in the 1950s; and that the punched tape flew through the Colossus reader so fast it sometimes caught fire. Sources: Copeland, B.J. (ed.), *Colossus: The Secrets of Bletchley Park’s Codebreaking Computers* (2006); Flowers, T., interview in BBC documentary “Station X” (1999); *National Museum of Computing, Bletchley Park*, biographical materials on T.H. Flowers; Randell, B., “Colossus: Godfather of the Computer” (2004).

REF-13: ENIAC: 17,468 vacuum tubes; 150 kilowatts; six women programmers; unveiled February 1946. ENIAC contained 17,468 vacuum tubes, consumed approximately 150 kilowatts, and was programmed by six women: Kay McNulty,

Betty Jennings, Betty Snyder, Marlyn Meltzer, Fran Bilas, and Ruth Lichterman. They were not credited in the 1946 press coverage. Source: McCartney, Scott, *ENIAC: The Triumphs and Tragedies of the World's First Computer*, Walker, 1999; Light, Jennifer S., "When Computers Were Women," *Technology and Culture*, Vol. 40, No. 3, 1999; Fritz, W. Barkley, "ENIAC— A Problem Solver," *IEEE Annals of the History of Computing*, 1994. The six programmers' contributions were first widely recognized through the documentary *Top Secret Rosies* (2010) and Kathy Kleiman's *Proving Ground* (2022). The 'refrigerator ladies' characterization is documented in Light (1999).

REF-14: Konrad Zuse's Z3: completed 1941; 2,600 relays; destroyed December 1943 by Allied bombing. Konrad Zuse completed the Z3 in 1941 using 2,600 electromechanical relays. It was destroyed in a December 1943 Allied bombing raid. The Z4 survived the war after Zuse evacuated it to Bavaria. Source: Rojas, Raúl (ed.), *The First Computers: History and Architectures*, MIT Press, 2000; Zuse, Konrad, *The Computer — My Life*, Springer, 1993; Deutsches Museum, Munich. <https://www.deutsches-museum.de/en/collections/communication/computers> Whether the Z3 qualifies as the 'first programmable digital computer' depends on definitional criteria. The text describes it as what 'many computer historians consider' to be such, acknowledging the debate.

REF-15: Transistor demonstrated December 23, 1947, at Bell Labs by Bardeen and Brattain; Shockley's junction transistor developed subsequently. John Bardeen and Walter Brattain demonstrated the point-contact transistor at Bell Labs on December 23, 1947. William Shockley subsequently developed the junction transistor in early 1948. Source: Riordan, Michael and Hoddeson, Lillian, *Crystal Fire: The Birth of the Information Age*, W.W. Norton, 1997; Nobel Prize documentation, 1956 Physics Prize; Bell Labs archives. <https://www.nobelprize.org/prizes/physics/1956/summary/> The internal dynamics between Shockley, Bardeen, and Brattain are extensively documented in *Crystal Fire* and in multiple biographical accounts.

REF-16: Shockley Semiconductor, the 'Traitorous Eight,' Fairchild Semiconductor; 65+ descendant companies. Shockley founded Shockley Semiconductor Laboratory in Mountain View, California, in 1956. Eight employees departed in 1957 to found Fairchild Semiconductor. Over 65 companies trace their lineage to Fairchild. Source: Berlin, Leslie, *The Man Behind the Microchip: Robert Noyce and the Invention of Silicon Valley*, Oxford, 2005; Lecuyer, Christophe, *Making Silicon Valley*, MIT Press, 2006; Computer History Museum Fairchild genealogy chart. <https://www.computerhistory.org/siliconengine/> The "65+

companies” figure varies by source (some cite over 100, depending on how lineage is counted). The Computer History Museum’s Silicon Engine exhibit documents the genealogy in detail.

REF-17: Moore’s Law: 1965 observation; transistor counts from Intel 4004 (1971, 2,300) to Apple 134 billion; total transistors manufactured ~13 sextillion. Gordon Moore published his observation in *Electronics* magazine, April 19, 1965. Transistor counts: Intel 4004 (1971): 2,300; Intel 486 (1989): 1.2 million; Apple 134 billion. Estimated total transistors manufactured: approximately 13 sextillion. Source: Moore, Gordon, “Cramming More Components onto Integrated Circuits,” *Electronics*, April 19, 1965; Intel product specifications (public); Apple product announcements (public); semiconductor industry analyses including Rupp, Karl, “Microprocessor Trend Data,” GitHub dataset. <https://ourworldindata.org/grapher/transistors-per-microprocessor> The 13 sextillion estimate is from semiconductor industry analyses (e.g., VLSI Research) and is approximate. The comparison to grains of sand (~7.5 sextillion, per University of Hawaii researchers) is a commonly used benchmark.

REF-18: TSMC fabricates over 90% of world’s most advanced semiconductors; ASML lithography machines cost~\$380 million each. TSMC’s dominance in advanced semiconductor fabrication and the cost and complexity of ASML’s extremeultraviolet lithography systems. Source: Miller, Chris, *Chip War: The Fight for the World’s Most Critical Technology*, Scribner, 2022; ASML annual reports; TSMC investor relations; Semiconductor Industry Association reports. The ‘over 90%’ figure refers to the most advanced process nodes (sub-7nm). TSMC’s exact market share varies by source and definition of ‘most advanced.’ The \$380 million ASML figure is for high-NA EUV systems as of 2024-2025 reporting.

REF-19: Current transistors approximately 3 nanometers; silicon atom approximately 0.2 nanometers. The most advanced transistors in production are approximately 3 nanometers in their smallest dimension. A silicon atom is approximately 0.2 nanometers in diameter. Source: Intel, Samsung, TSMC process node announcements (public); standard atomic physics reference values for silicon atomic radius. Process node naming conventions are somewhat marketing-driven and do not correspond precisely to physical gate lengths. The 3nm designation refers to the manufacturing process generation, not necessarily the exact physical dimension of the smallest feature. The atomic-scale comparison is directionally accurate.

REF-20: Ken Olsen quote: “There is no reason anyone would want a computer in their home” (1977). Ken Olsen, founder of Digital Equipment Corporation, is widely

quoted as stating this in 1977. Source: Widely attributed; context discussed in Rifkin, Glenn and Harrar, George, *The Ultimate Entrepreneur: The Story of Ken Olsen and Digital Equipment Corporation*, Contemporary Books, 1988. Olsen later disputed the meaning of the quote, arguing he was referring to a computer that controlled household functions, not a personal computer. The text uses the quote as a data point about institutional inability to anticipate distributional shifts.

REF-21: Gary Kildall, CP/M (1974), the BIOS concept, the IBM/Microsoft deal, and Kildall's death in 1994. Gary Kildall created CP/M in 1974, the first commercially successful personal computer operating system. By 1981 it ran on over 3,000 computer models. Kildall also invented the BIOS concept. IBM approached Digital Research in 1980 for an OS; negotiations failed. Bill Gates acquired QDOS for \$75,000, renamed it MS-DOS, and licensed it to IBM while retaining rights to license to others. IBM priced CP/M at \$240 and MS-DOS at \$40. Kildall died in 1994 at age 52. Source: Kildall, Gary, *Computer Connections* (unpublished memoir, first seven chapters released 2016 by Kildall family via Computer History Museum); Evans, Harold, *They Made America*, Little Brown, 2004; Computer History Museum, "Fifty Years of the Personal Computer Operating System," 2024; Eubanks, Gordon (interview), Computer History Museum oral histories. <https://computerhistory.org/blog/fifty-years-of-the-personal-computer-operating-system/> The circumstances of the failed IBM-Digital Research deal remain disputed. The text presents the broadly accepted outline while acknowledging that accounts differ. The 'Gary was flying' narrative, popularized by Bill Gates, has been contested by Kildall, his family, and multiple associates. The \$240 vs \$40 pricing disparity is documented in multiple sources. Kildall's children released portions of his memoir in 2016 noting that the 'missed opportunity' myth 'ignores his deep contributions and overshadows his role as an inventor of key technologies that define how computer platforms run today.'

REF-22: ARPANET first message, October 29, 1969; "LO". The first ARPANET message was transmitted on October 29, 1969, from UCLA to Stanford. The system crashed after the letters 'L' and 'O,' before the intended message 'LOGIN' could be completed. Source: Kleinrock, Leonard, "The First Message Transmitted over the ARPANET," UCLA Kleinrock Internet Heritage archive; Hafner, Katie and Lyon, Matthew, *Where Wizards Stay Up Late*, Simon & Schuster, 1996. https://www.lk.cs.ucla.edu/internet_first_words.html The log book from the event is preserved at UCLA. Kleinrock has recounted this story in numerous published interviews and lectures.

REF-23: Tim Berners-Lee's memo, March 1989; supervisor's note: "Vague but exciting". Tim Berners-Lee submitted "Information Management: A Proposal" to his supervisor at CERN in March 1989. The handwritten annotation 'Vague but exciting' appears on the cover page. Source: Berners-Lee, Tim, *Weaving the Web*, HarperSanFrancisco, 1999; CERN document archives (original proposal digitized and publicly available). <https://cds.cern.ch/record/369245> The supervisor was Mike Sendall. The original document with annotation is viewable at the CERN document server link.

REF-24: Netscape IPO, August 1995; market cap approximately \$2.9 billion. Netscape's IPO on August 9, 1995, opened at \$28 per share and reached \$75 on its first day, with a market capitalization of approximately \$2.9 billion. Source: Public market records; widely documented in business histories including Cassidy, John, *Dot.con*, HarperCollins, 2002.

REF-25: Approximately 6.8 billion smartphone subscriptions worldwide by 2025. Global smartphone subscriptions reached approximately 6.8 billion by 2025. Source: GSMA Intelligence, "The Mobile Economy 2025"; Statista, "Number of smartphone subscriptions worldwide." <https://www.gsma.com/mobileeconomy/> The figure refers to subscriptions, not unique users (some individuals hold multiple subscriptions). The population comparison(~8 billion) provides context.

REF-26: McCulloch-Pitts model, 1943; Rosenblatt's Perceptron, 1958; New York Times coverage. Warren McCulloch and Walter Pitts published their neural model in 1943. Frank Rosenblatt demonstrated the Perceptron in 1958. The New York Times reported on the Perceptron on July 8, 1958. Source: McCulloch, W.S. and Pitts, W., "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bulletin of Mathematical Biophysics*, 5, 1943; New York Times, "New Navy Device Learns by Doing," July 8, 1958; Rosenblatt, Frank, "The Perceptron: A Probabilistic Model," Cornell Aeronautical Laboratory, 1958. The NYT article described capabilities beyond what the Perceptron could actually deliver, contributing to the hype-disappointment cycle. Expanded (March 2026): "Godfathers of deep learning" details (expanded) The exile section was expanded to include: Geoffrey Hinton at the University of Toronto; Yann LeCun at Bell Labs and later NYU; Yoshua Bengio at the University of Montreal; their shared 2018 Turing Award; Hinton's described experience of having papers rejected because neural networks were considered a discredited approach; LeCun's convolutional neural networks for cheque reading; Bengio's early language modelling work; Hinton recruited by Google, LeCun by Facebook (Meta). Sources: LeCun, Y., Bengio, Y., and Hinton, G., "Deep Learning,"

Nature, Vol. 521, pp. 436–444 (2015); ACM A.M. Turing Award Citation (2018); Hinton, G., various interviews including Lex Fridman Podcast #449 (2024); Cade Metz, “Genius Makers” (2021).

REF-27: Minsky and Papert, *Perceptrons*, 1969; backpropagation: Rumelhart, Hinton, Williams, 1986. Minsky and Papert’s *Perceptrons* (1969) demonstrated limitations of single-layer networks. Backpropagation was effectively demonstrated by Rumelhart, Hinton, and Williams in 1986. Source: Minsky, Marvin and Papert, Seymour, *Perceptrons*, MIT Press, 1969; Rumelhart, D.E., Hinton, G.E., and Williams, R.J., “Learning representations by back-propagating errors,” *Nature*, 323, 1986. <https://www.nature.com/articles/323533a0> Backpropagation had been described in various forms by Werbos (1974), Linnainmaa (1970), and others. The 1986 *Nature* paper is credited with making it widely practical and known. Expanded (March 2026): “Godfathers of deep learning” details (expanded) The exile section was expanded to include: Geoffrey Hinton at the University of Toronto; Yann LeCun at Bell Labs and later NYU; Yoshua Bengio at the University of Montreal; their shared 2018 Turing Award; Hinton’s described experience of having papers rejected because neural networks were considered a discredited approach; LeCun’s convolutional neural networks for cheque reading; Bengio’s early language modelling work; Hinton recruited by Google, LeCun by Facebook (Meta). Sources: LeCun, Y., Bengio, Y., and Hinton, G., “Deep Learning,” *Nature*, Vol. 521, pp. 436–444 (2015); ACM A.M. Turing Award Citation (2018); Hinton, G., various interviews including Lex Fridman Podcast #449 (2024); Cade Metz, “Genius Makers” (2021).

REF-28: AlexNet and ImageNet 2012; Transformers (“Attention Is All You Need”), 2017; ChatGPT, November 30, 2022, 100 million users in ~2 months. AlexNet won the ImageNet Large Scale Visual Recognition Challenge in 2012 by a large margin. The transformer architecture was introduced in 2017. ChatGPT was released November 30, 2022, reaching 100 million users in approximately two months. Source: Krizhevsky, Alex et al., “ImageNet Classification with Deep Convolutional Neural Networks,” *NeurIPS*, 2012; Vaswani, Ashish et al., “Attention Is All You Need,” *NeurIPS*, 2017; OpenAI blog, various reports on ChatGPT adoption (UBS, Reuters). The 100 million user figure and approximate two-month timeline were widely reported (e.g., Reuters, February 2023) based on analytics data. Exact figures depend on measurement methodology.

REF-29: Malte Spitz: 35,831 data points collected by Deutsche Telekom over six months (2009–2010). German Green Party politician Malte Spitz sued Deutsche Telekom to obtain his personal data. The company delivered a spreadsheet containing

35,831 rows of location, communication, and activity data covering six months (August 31, 2009 to February 28, 2010). Spitz and Die Zeit created an interactive visualization mapping six months of his life. Source: Spitz, Malte, TED Talk, “Your phone company is watching,” TEDGlobal 2012; Zeit Online interactive visualization, “Tell-All Telephone,” 2011; Electronic Frontier Foundation, “Why Metadata Matters,” 2014; Spitz, Malte, What Are You Doing with My Data?, 2014. <https://www.zeit.de/datenschutz/malte-spitz-vorratsdaten> The 35,831 figure represents instances where Spitz’s phone communicated with the Deutsche Telekom network, triggered primarily by email checks and calls. The data was collected under a German data retention law later ruled unconstitutional by the German Constitutional Court in March 2010. The interactive visualization remains accessible at the Zeit Online link.

REF-30: Mark Weiser quote: “The most profound technologies are those that disappear”. Epigraph for Chapter 8, attributed to Mark Weiser. Source: Weiser, Mark, “The Computer for the 21st Century,” *Scientific American*, September 1991. <https://www.lri.fr/~mbl/Stanford/CS477/papers/Weiser-SciAm.pdf> This paper is foundational to the field of ubiquitous computing. Weiser directed the Computer Science Laboratory at Xerox PARC.

REF-31: Feynman’s 1981 proposal for quantum computing. Richard Feynman proposed that simulating quantum physics would require a quantum computer at a 1981 MIT conference. Source: Feynman, Richard, “Simulating Physics with Computers,” *International Journal of Theoretical Physics*, Vol. 21, Nos. 6/7, 1982 (proceedings from 1981 conference). The widely quoted “Nature isn’t classical, dammit” phrasing comes from various retellings; the published paper uses more formal language expressing the same idea.

REF-32: Shor’s algorithm, 1994; Google Sycamore “quantum supremacy” claim, 2019. Peter Shor published a quantum algorithm for integer factorization in 1994 that could break RSA encryption. In 2019, Google’s 53-qubit Sycamore processor claimed quantum supremacy. Source: Shor, Peter, “Algorithms for Quantum Computation,” *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, 1994; Arute, Frank et al., “Quantum supremacy using a programmable superconducting processor,” *Nature*, 574, 2019. <https://www.nature.com/articles/s41586-019-1666-5> IBM disputed Google’s classical time estimate, suggesting that with optimal classical algorithms and sufficient storage, the classical computation could be completed in days rather than 10,000 years. The text acknowledges this dispute.

REF-33: DNA storage: 1 gram $\approx$ 215 petabytes; Church et al. 2012; Microsoft/UW automated system 2019. Theoretical DNA storage density is approximately 215 petabytes per gram. George Church's team encoded a book in DNA in 2012. Microsoft and the University of Washington demonstrated automated DNA read/write in 2019. Source: Church, G.M. et al., "Next-Generation Digital Information Storage in DNA," *Science*, 337, 2012; Takahashi, C.N. et al., "Demonstration of End-to-End Automation of DNA Data Storage," *Scientific Reports*, 9, 2019; Goldman, N. et al., "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, 494, 2013. <https://www.science.org/doi/10.1126/science.1226355> The 215 petabytes per gram figure is a theoretical upper bound based on Shannon information density of DNA. Practical achieved density is lower due to encoding overhead, error correction, and addressing schemes.

REF-34: Human brain: 86 billion neurons, $\sim$ 100 trillion synapses, $\sim$ 20 watts; Adleman DNA computing experiment, 1994. The human brain contains approximately 86 billion neurons and $\sim$ 100 trillion synapses, operating on approximately 20 watts. Leonard Adleman demonstrated DNA computing by solving a Hamiltonian Path problem in 1994. Source: Azevedo, Frederico A.C. et al., "Equal numbers of neuronal and nonneuronal cells make the human brain anisometrically scaled-up primate brain," *Journal of Comparative Neurology*, 513, 2009; Adleman, Leonard, "Molecular Computation of Solutions to Combinatorial Problems," *Science*, 266, 1994. The 86 billion figure replaced the commonly cited '100 billion' after Azevedo et al.'s 2009 direct count. The 20-watt figure is a widely cited estimate for average resting brain metabolism.

REF-35: Reprogrammable DNA computer running 21 algorithms, 2024 (Caltech, UC Davis, Maynooth University). A 2024 publication demonstrated a reprogrammable DNA computer capable of running 21 distinct algorithms on the same molecular hardware. Source: Woods, Damien et al., published in *Nature*, 2024; Caltech, UC Davis, and Maynooth University institutional announcements. This is cited as a landmark result because previous DNA computing designs were single-purpose. The reprogrammability represents a fundamental shift toward general-purpose molecular computation.

REF-36: Mammoth DNA readable after 40,000+ years; readable DNA from Siberian permafrost. DNA has been extracted and sequenced from woolly mammoths frozen in Siberian permafrost for over 40,000 years. Source: van der Valk, Tom et al., "Million-year-old DNA sheds light on the genomic history of mammoths," *Nature*,

591,2021 (reporting DNA recovery from specimens dated over one million years); various studies of Siberian permafrost specimens. <https://www.nature.com/articles/s41586-021-03224-9> The text uses ‘over forty thousand years’ conservatively; the 2021 Nature paper reports recovery from specimens over one million years old, though with increasing degradation at greater ages.

REF-37: Edison’s Pearl Street Station, September 4, 1882; 59 customers; J.P. Morgan’s office at 23 Wall Street. Thomas Edison’s Pearl Street generating station began operation on September 4, 1882, in lower Manhattan, serving approximately 59 initial customers within a half-mile radius. J.P. Morgan’s office at 23 Wall Street was among the first electrically illuminated. The electrical grid subsequently expanded to reach millions of homes and businesses within approximately forty years. Source: Jonnes, Jill, *Empires of Light: Edison, Tesla, Westinghouse, and the Race to Electrify the World*, Random House, 2003; IEEE, “Milestones: Pearl Street Station,” engineering heritage records; Edison Papers Project, Rutgers University. The “59 customers” figure is from Edison’s records for the first day of operation. Some sources cite 85 customers within the first few months. The half-mile radius is approximate and based on the limitations of the direct current distribution system Edison used. The station was destroyed by fire in 1890 but had already been supplemented by additional stations.

REF-38: “Harvest now, decrypt later” strategy; NIST post-quantum cryptography standardization. The ‘harvest now, decrypt later’ strategy involves intercepting and archiving encrypted communications with the expectation of future quantum decryption. NIST has led a multi-year effort to standardize post-quantum cryptography. Source: National Institute of Standards and Technology, “Post-Quantum Cryptography Standardization,” ongoing since 2016; first standards published 2024; widely discussed in intelligence community literature and congressional testimony. <https://csrc.nist.gov/projects/post-quantum-cryptography> The ‘harvest now, decrypt later’ phrase is used in intelligence community discourse and academic cybersecurity literature. The NIST standardization effort is public and extensively documented.

REF-39: US semiconductor export controls on China, 2022–2023. The United States imposed sweeping restrictions on the export of advanced semiconductor technology to China in October 2022 and expanded them in 2023, explicitly framed in terms of national security. Source: Bureau of Industry and Security, US Department of Commerce, “Implementation of Additional Export Controls,” October 7, 2022;

subsequent updates 2023; Miller, Chris, Chip War, 2022. <https://www.bis.doc.gov/index.php/documents/about-bis/newsroom/press-releases/3158-2022-10-07-bis-press-release-advanced-computing-and-semiconductor-manufacturing-controls-final/file> These controls represent the most significant technology export restrictions since the Cold War and are directly relevant to the book's argument about computational substrates becoming sovereign capabilities.

REF-40: Nuclear governance precedents: Bohr and Oppenheimer advocacy for international control; Soviet test 1949; NPT, CTBT. Niels Bohr and J. Robert Oppenheimer advocated for international control of atomic energy. The Soviet Union tested its first atomic bomb in 1949. The Nuclear Non-Proliferation Treaty (NPT) entered into force in 1970. Source: Rhodes, Richard, *The Making of the Atomic Bomb*, Simon & Schuster, 1986; Rhodes, Richard, *Dark Sun: The Making of the Hydrogen Bomb*, Simon & Schuster, 1995; United Nations Office for Disarmament Affairs (NPT text and history). These precedents are invoked as an analogy for governing transformative computational capability, while noting the critical distinction that nuclear capabilities are visible and quantum capabilities may not be.

REF-41: Anthropic-Department of War confrontation, February–March 2026; \$200M contract, supply-chain risk designation, two red lines. Anthropic held a \$200 million contract with the Department of War since July 2025. It was the first AI company to deploy models on classified military networks. The company maintained two conditions: no mass domestic surveillance and no fully autonomous weapons. In January 2026, Secretary of Defense Hegseth issued an memorandum requiring 'any lawful use' language in all AI contracts. Anthropic refused. On February 27, 2026, the President directed all agencies to cease use of Anthropic's technology, and the Secretary designated Anthropic a supply-chain risk to national security — a label never previously applied to an American company. Source: Anthropic official statement, "Statement on the Department of War," [anthropic.com](https://www.anthropic.com/news/statement-department-of-war), February 27, 2026; CNBC, "Anthropic officially told by DOD that it's a supply chain risk," March 5, 2026; NBC News, "Tensions between the Pentagon and AI giant Anthropic reach a boiling point," February 2026; ASIS Online / Security Management, "Legal Report: DOD Disavows Premier Partner, Anthropic," March 2026. <https://www.anthropic.com/news/statement-department-of-war> The confrontation is ongoing as of this writing. Anthropic has stated it will challenge the supply-chain risk designation in court. Legal scholars have noted the designation appears to exceed the statutory authority of 10 U.S.C. § 3252 (see Lawfare analysis). The two red lines — mass domestic surveillance and fully autonomous weapons — had been part of

Anthropic's contract since its inception and had not, according to the company, been a barrier to military use of Claude.

REF-42: Legal analysis: supply-chain risk designation unprecedented for American company; statutory authority questioned. Legal analysis from Lawfare, the Center for American Progress, and academic scholars concluded that the supply-chain risk designation was legally unprecedented for an American company, that the statutory authority invoked was designed for foreign adversaries, and that the 'secondary boycott' provision (barring all commercial activity with Anthropic) exceeded the scope of 10 U.S.C. § 3252. Source: Lawfare, "Pentagon's Anthropic Designation Won't Survive First Contact with Legal System," March 2, 2026; Center for American Progress, "The Department of Defense's Conflict With Anthropic and Deal With OpenAI Are a Call for Congress To Act," March 5, 2026; Electronic Frontier Foundation, "The Anthropic-DOD Conflict," March 2026. <https://www.lawfaremedia.org/article/pentagon-s-anthropic-designation-won-t-survive-first-contact-with-legal-system> The Lawfare analysis notes that the only prior public use of the FASCSA supply-chain risk authority was against Acronis AG, a Swiss firm with reported Russian ties, limited to intelligence community contracts. No domestic company had been publicly designated.

REF-43: Claude rises to #1 in U.S. App Store; 1 million+ daily sign-ups; 60%+ growth in free users; paid subscribers doubled. Following the confrontation, Claude rose from outside the top 100 to the #1 most downloaded free app in Apple's U.S. App Store within 72 hours, overtaking ChatGPT. Daily sign-ups exceeded 1 million per day. Free users increased more than 60% since January. Paid subscribers more than doubled. The app reached #1 in 16 countries. Source: TechCrunch, "Anthropic's Claude rises to No. 1 in the App Store following Pentagon dispute," March 1, 2026; Axios, "Claude beats ChatGPT in U.S. app downloads after Pentagon blacklists Anthropic," March 1, 2026; CNN, "Anthropic's Claude is suddenly the most popular iPhone app," March 3, 2026; TechCrunch, "Claude's consumer growth surge continues," March 6, 2026; Sensor Tower and Appfigures data cited in multiple reports. <https://techcrunch.com/2026/03/01/anthropics-claude-rises-to-no-2-in-the-app-store-following-pentagon-dispute/> The growth figures are from Anthropic's own statements confirmed by independent app intelligence firms. Reddit's 'Cancel and Delete ChatGPT' post (30,000+ upvotes) and the chalk messages outside both companies' offices are documented in Axios and CNN reporting. The Washington Post reported on March 6 that 'an American public largely unaware of the company raced to download its app.' A Note on Sources Claims in this series draw on a mix of

primary sources: original research publications, official corporate and institutional records, peer-reviewed academic papers, published books by domain experts, and contemporaneous news reporting from named journalists at named outlets. Historical claims are grounded in the standard scholarly sources cited above. For recent developments — quantum computing milestones, AI adoption figures, semiconductor geopolitics — multiple independent sources are cited where figures may vary. The historical events covered in this book — particularly in Chapters 1 through 7 — have been documented extensively by many excellent historians of computing, including Walter Isaacson, Martin Campbell-Kelly and William Aspray, George Dyson, Charles Petzold, Scott McCartney, Michael Riordan and Lillian Hoddeson, and others cited above. This series draws on their scholarship while offering a different interpretive framework: the substrate evolution thesis, the five-pattern model, and the power concentration warning are original to this work. Where facts are shared — as they must be, since there is only one history of the transistor and only one ARPANET first message — the narrative voice, analytical framework, and forward-looking argument are the author's own. The interpretive framework of this series — treating computing history as a natural history of substrate evolution governed by repeating patterns — is, to the author's knowledge, original. The five patterns identified (Ceiling and Leap, Inspiration Chain, Expansion of Who, Unexpected Use, Return to Nature), the distribute-then-recapture power dynamic, and the computational omniscience warning of Chapters 11–14 have no direct precedent in the existing literature on computing history. The author welcomes correction on this point. URLs were accurate at time of writing (March 2026). Some archived materials may require institutional access. Academic papers are available at the linked repository URLs or through standard academic databases. The Machine Evolves Selected Reading This book tells a particular story with a particular thesis. It does not attempt to be comprehensive. The following books approach the history of computing from different angles and with different emphases, and any reader who found this series valuable will find them valuable too. I have listed them not in order of importance but in the order that best extends the argument of this book — from the human stories, to the technical foundations, to the geopolitical implications, to the conceptual frameworks that shaped my thinking. Walter Isaacson, *The Innovators: How a Group of Hackers, Geniuses, and Geeks Created the Digital Revolution* (2014) The most widely read narrative history of computing, and the book that covers the most overlapping territory with this one. Isaacson tells the story through people — Lovelace, Turing, Shockley, Gates, Jobs, Berners-Lee — and his thesis is that innovation is fundamentally

collaborative. If you want the human stories behind the substrate transitions described in this book, told with warmth and exceptional detail, start here. Where Isaacson and I diverge: he focuses on who built the machines. I focus on what the machines do to the distribution of power. He stops at the internet era. I continue through quantum and biological computing. Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software* (1999, revised 2022) If this book made you want to understand how computers actually work at a physical level — how switches become logic gates, how logic gates become processors, how binary becomes computation — this is the book to read next. Petzold starts with Morse code and Braille and builds, layer by painstaking layer, to a complete understanding of a functioning computer. It is the best pedagogical treatment of computer architecture I have encountered, accessible to anyone willing to follow the argument carefully. Chris Miller, *Chip War: The Fight for the World's Most Critical Technology* (2022) If Chapter 4 of this book — the geopolitics of TSMC, the Taiwan Strait, and semiconductor export controls — concerned you, this is the deep dive. Miller traces the geopolitical history of the semiconductor industry from its Cold War origins through the current US-China competition with the rigor of a historian and the urgency of a strategist. Essential reading for anyone who wants to understand why chips are now a matter of national security, and what the precedent means for quantum computing governance. James Gleick, *The Information: A History, A Theory, A Flood* (2011) Gleick traces the concept of information itself — from African talking drums to Claude Shannon's information theory to the DNA genetic code — with extraordinary intellectual range. Where this book traces the physical machines that process information, Gleick traces the abstract idea of information that the machines manipulate. The two perspectives are complementary: read Gleick to understand what information is. Read this book to understand who controls the machines that process it. Michael Riordan and Lillian Hoddeson, *Crystal Fire: The Birth of the Information Age* (1997) The definitive history of the transistor, told by two science historians with deep access to the Bell Labs archives and the personal stories of Shockley, Bardeen, and Brattain. If Chapter 4 of this book captured your interest, *Crystal Fire* provides the full, richly detailed account of the personalities, the physics, and the institutional dynamics behind the most important invention of the twentieth century. Ray Dalio, *Principles for Dealing with the Changing World Order: Why Nations Succeed and Fail* (2021) Not a computing book at all, but the methodological inspiration for this one. Dalio's approach — identifying repeating patterns across centuries of history, treating them as structural cycles, and using them to generate predictions about the present — is the

method I applied to computing history. If you are interested in pattern-based historical analysis applied to geopolitics, economics, and the rise and fall of empires, Dalio's work is the place to start. His intellectual rigor and willingness to make falsifiable predictions are qualities I have tried to emulate, however imperfectly.

Yuval Noah Harari, *Sapiens: A Brief History of Humankind* (2011) The book that proved deep history could be told accessibly, provocatively, and with a thesis that reframes how the reader sees the world. Harari covers 70,000 years of human history in 400 pages and makes it feel urgent. His *Nexus: A Brief History of Information Networks* (2024) is more directly related to this book's subject matter, tracing information networks from the Stone Age to AI. Harari's focus is on information's political and social consequences; mine is on the physical substrates and their power dynamics. Both books ask, from different angles, what it means when technology reshapes who holds power.

Dennis Shasha and Cathy Lazere, *Natural Computing: DNA, Quantum Bits, and the Future of Smart Machines* (2010) The book that comes closest to the territory of Chapters 9 and 10 of this series. Shasha and Lazere profile fifteen scientists working at the boundaries of computing and nature — DNA computing, quantum computing, bio-inspired robotics. It is profile-driven journalism rather than historical narrative, but it provides an accessible and engaging introduction to the research communities working on the substrates this book describes as the next phase of computing's evolution.

A Final Note I have learned from every book on this list and from many others not listed here. The history of computing is a collaborative achievement — built by thousands of researchers, engineers, historians, and writers whose work makes a synthesis like this one possible. I have tried to credit them fairly in the References document and here. Where I have fallen short, I welcome correction. The best thing a book can do is send you to other books. If this one has done that, it has succeeded.

Pumulo Sikaneta Curious by nature. Correctable by choice. Always learning. Often wrong. Let's discuss.

Supplementary References & Source Attribution Additional sources for claims added or expanded during the March 2026 rewrite of *Who Holds the Jar?* This document supplements the main References & Source Attribution document. It covers factual claims that were added, expanded, or newly sourced during the rewrite of all fourteen chapters in *The Last Keeper* voice.

REF-44: Frank Rosenblatt death. Frank Rosenblatt died on July 11, 1971, on his 43rd birthday, in a boating accident on Chesapeake Bay. He did not live to see his neural network ideas vindicated. Sources: Olazaran, M., "A Sociological Study of the Official History of the Perceptrons Controversy," *Social Studies of Science*, Vol. 26, No. 3 (1996), pp. 611–659; "Frank Rosenblatt," Cornell University Faculty Memorial

Statements.

REF-45: Zambia mobile money and digital economy. The Lusaka section references the scale of mobile money in Zambia's economy. President Hakainde Hichilema stated that 2024 digital payment transaction values accounted for more than 70% of Zambia's GDP. Mobile money transaction value reached K452 billion in 2023, a 50% increase from K295.8 billion in 2022, with 2.24 billion transactions. Approximately 58.4% of Zambia's adult population uses mobile money services. 12.6 million active mobile wallets were recorded by end of 2023 with over 340,000 mobile money agents across the country. Sources: Payments Association of Zambia / President Hichilema remarks, reported by Mobile Money Africa (2024); Bank of Zambia, Payment Systems Statistics; ZICTA ICT Sector 2024 Mid-Year Market Report; UNDP, "The Role of Digital Mobile Money in Catalyzing Financial Inclusion: Country Report Zambia" (2024); International Growth Centre, "How Zambia can leverage mobile money to increase tax revenue" (2024); Nkwazi Magazine, "The Currency of Progress" (2025).

REF-46: PIPEDA and Canadian data governance. Source: Office of the Privacy Commissioner of Canada, "PIPEDA in brief"; Government of Canada, Personal Information Protection and Electronic Documents Act, S.C. 2000, c. 5.

REF-47: South African telecoms in Zambia The Lusaka section notes that mobile networks in Zambia are operated by South African telecommunications companies. MTN Group (South Africa) operates in Zambia, alongside Airtel (India). Cloud infrastructure for these operators is contracted from American hyperscalers (primarily AWS and Microsoft Azure). Sources: MTN Group Annual Report (2024); Airtel Africa Annual Report (2024); GSMA, "The State of the Industry Report on Mobile Money 2025"; general industry reporting on cloud infrastructure contracts.

REF-48: Epic of Gilgamesh. Standard Babylonian version, compiled c. 1200 BCE from earlier Sumerian poems (c. 2100 BCE). George, Andrew R. The Babylonian Gilgamesh Epic: Introduction, Critical Edition and Cuneiform Texts. Oxford University Press, 2003. Tablet XI: the plant of rejuvenation stolen by a serpent.

REF-49: Samudra Manthan (Churning of the Ocean of Milk). Hindu mythology, described in the Bhagavata Purana (Canto 8) and the Vishnu Purana (Book 1, Ch. 9). Amrita (nectar of immortality); Mohini avatar of Vishnu; distribution exclusively to the Devas. Doniger, Wendy. Hindu Myths: A Sourcebook. Penguin, 2004.

REF-50: Global Positioning System (GPS). US Department of Defense, NAVSTAR programme, initiated 1973. Civilian access: Reagan directive following KAL 007 shootdown (September 1, 1983). Selective Availability removal: Clinton

executive order, May 1, 2000. GPS.gov; National Coordination Office for Space-Based Positioning, Navigation, and Timing.

REF-51: Life extension and biological ageing research. CRISPR-Cas9: Doudna, Jennifer A. & Charpentier, Emmanuelle. Nobel Prize in Chemistry, 2020. Senolytics: Kirkland, James L. & Tchkonja, Tamara. “Senolytic drugs: from discovery to translation.” *Journal of Internal Medicine* 288(5):518–536, 2020. Telomere biology: Blackburn, Elizabeth H. Nobel Prize in Physiology or Medicine, 2009. Negligible senescence organisms: Finch, Caleb E. *Longevity, Senescence, and the Genome*. University of Chicago Press, 1990.

REF-52: Presumption of innocence and autonomous weapons. Universal Declaration of Human Rights, Article 11 (1948). Geneva Conventions Additional Protocol I (1977), Articles 48, 51, 52. Distinction, proportionality, and precaution in attack. Application to algorithmic targeting: Scharre, Paul. *Army of None: Autonomous Weapons and the Future of War*. W.W. Norton, 2018. International Committee of the Red Cross position on autonomous weapons, 2021.

REF-53: Sikaneta, Tabo et al. “Ethnic and immigrant disparities in dialysis prevalence and chronic kidney disease trajectories in Toronto.” *BMJ Open*, 2025. Double observational cohort, 2,397 participants, Scarborough Health Network. Dialysis prevalence 4.2× higher in immigrants; birth country predicted outcomes more strongly than ethnicity. Published under CC BY-NC 4.0.

Note: The claim that agricultural pricing algorithms for Zambian smallholder farming run on US cloud infrastructure reflects the general architecture of platform-based agricultural pricing services (e.g., WFP/FEWS NET commodity pricing, Digital Green, Esoko) rather than a specific documented instance. The analytical framing of the Oracle at three altitudes (New York, Toronto, Lusaka) is original to this work.

Intellectual Debts and Inspirations

No book is built in isolation. The frameworks, voice, and ambition of this series owe specific debts to writers and thinkers whose work shaped my approach. I name them here not as a formality but as an act of honesty. The ideas in this book are my own synthesis and my own argument, but they were made possible by standing in the light of others.

**Ray Dalio. Dalio’s method — identifying repeating patterns in historical data, treating them as structural cycles rather than isolated events, and using them to*

*generate testable predictions — is the methodological foundation of this series. His Principles for Dealing with the Changing World Order** (2021) demonstrated that deep historical pattern-recognition could be applied to geopolitics with rigor and accessibility. The five-pattern framework in this book, the leading indicators in Chapter 13, and the structural approach to prediction throughout the series are directly inspired by Dalio's method, applied to a different domain. I owe him the approach; the conclusions are my own.

**Yuval Noah Harari. Harari's Sapiens: A Brief History of Humankind (2011) proved that deep history could be told accessibly, with a provocative thesis and a narrative arc that respects the reader's intelligence while refusing to assume specialist knowledge. His Nexus: A Brief History of Information Networks* (2024) traces information technology from the Stone Age to AI, and readers of both works will recognize thematic resonance with this series. Harari's focus is on information networks and their political consequences; mine is on physical computing substrates and their power dynamics. The approaches are complementary, not overlapping, but his example showed me that the scope I was attempting was achievable.*

**Walter Isaacson. Isaacson's The Innovators: How a Group of Hackers, Geniuses, and Geeks Created the Digital Revolution (2014) is the most widely read narrative history of computing, and many of the historical events in this book are events he also covers — because they are THE events. Babbage, Lovelace, Turing, Shockley, the personal computer revolution, and the internet are shared territory. Isaacson's thesis is that innovation is collaborative; mine is that computing substrates evolve and concentrate power. His book ends at roughly 2010; mine extends through quantum and biological computing. Readers who enjoyed this series will find his complementary and richly detailed. Additional Influences James Gleick, The Information: A History, A Theory, A Flood (2011) — for the treatment of Shannon, information theory, and DNA as information. Gleick traces the abstract concept of information; this series traces the physical machines that process it. Charles Petzold, Code: The Hidden Language of Computer Hardware and Software (1999, revised 2022) — for the pedagogical clarity of explaining binary, Boolean logic, and computer architecture from first principles. Chris Miller, Chip War: The Fight for the World's Most Critical Technology (2022) — for the geopolitical analysis of semiconductor manufacturing, TSMC, and the strategic significance of chip fabrication. Martin Campbell-Kelly and William Aspray, Computer: A History of the Information Machine (4th ed., 2023) — the standard scholarly history, consulted for factual verification throughout. George Dyson, Turing's Cathedral: The Origins of the*

Digital Universe* (2012) — for the institutional history of early electronic computing.

About the Author

Pumulo Sikaneta has spent his career in the technology industry, where a decade of work across partner strategy, platform architecture, and enterprise transformation gave him a front-row seat to the questions this trilogy examines: what happens when the systems we build outpace the governance we write for them, and who bears the cost when they do.

Born in Zambia, raised in Canada, and living in the United States, he has experienced three different relationships to the power structures his work examines — from a developing nation whose sovereignty depends on infrastructure it does not own, through a middle power navigating between hegemony, to the country whose technology companies are building the systems the trilogy documents.

He wrote it because the evidence demanded it and because nobody with more authority was writing it fast enough.